From Objects to Functions

Note: For this book, I use longer, detailed code examples. This is because they present each series of code transformations as a logical step-by-step process to an end state, and this isn't readily apparent from the big jumps from step 1 to step 2 in the actual code examples directory. You lose a lot of context without the intermediate steps.

Chapter 1 - Preparing a New Application

- In this book we will be working on a to-do-list web application named Zettai.
- We used user stories and *EventStorming* to plan our new application. In EventStorming, you write down on a sticky note each "thing" that happens in the future application. Events can be external to the application (e.g., you finished something you had to do) or internal (e.g., you checked off a new to-do item). You describe these events in the past tense. Then you arrange the sticky notes from left to right by timing and try to group together related events. Then with sticky notes of a different color, you write down the actions that caused the events. Finally, you identify what triggered the events, either the interaction with your application (ex: a button click) or an external event (ex: the end of the month). And we drew some loose, non-binding mockups.
- We also agreed to use *test driven development* to design and document our application. The main benefit of TDD is it forces us to define our *success criteria* before we start coding, thus thinking about what would be a solution to the problem we are trying to solve. In TDD, you start with a failing test, then write the code to make it pass, and then clean up the code until you're happy enough to switch to a new test. It is often called the *red-green-refactor* cycle. Software designed to be easy to test is also easy to maintain and improve.
- Our overall approach is to be "outside-in", working from the acceptance tests and finishing with the core domain. Alternatively, we could choose "inside out" where we focus on the unit tests and domain and build up functionality. The advantage to "outside-in" is we're guided by the external constraints of the user stories and don't risk implementing core functionality in a way that doesn't fit the problems we are trying to solve. The drawback is that it can take hours or days before you are able to fulfill the test since acceptance tests cover a full slice of functionality.
- We will be using JUnit for tests and Strikt as an assertion library with a convenient DSL.

- We will also use property-based testing. It is a good way to test pure functions which transform an input type A to a different type B.
- Here is a small example of testing addition to show off the testing and assertion libraries and present the idea of property based testing:

```
import kotlin.random.Random

class AdditionTest{
    fun randomNatural() = Random.nextInt(from = 1, until = 100_000_000)

    @Test
    fun `add two numbers`() {
        expectThat(5 + 6).isEqualTo(11)
        expectThat(7 + 42).isEqualTo(49)
        expectThat(9999 + 1).isEqualTo(10000)
    }

    @Test
    fun `zero identity`() {
        repeat(100) {
            val x = randomNatural()

            expectThat(x + 0).isEqualTo(x)
        }
    }

    @Test
    fun `commutative property`() {
        repeat(100) {
            val x = randomNatural()
            val y = randomNatural()

            expectThat(x + y).isEqualTo(y + x)
        }
    }

    @Test
    fun `associative property`() {
        repeat(100) {
            val x = randomNatural()
            val y = randomNatural()
            val z = randomNatural()

            expect {
                that((x + y) + z).isEqualTo(x + (y + z))
                that((y + z) + x).isEqualTo(y + (z + x))
                that((z + x) + y).isEqualTo(z + (x + y))
            }
        }
```

```
    }
}
```

- Note that we are using Strikt's soft assertions, which allow us to see all assertion results even if some assertions fail. That is why we are wrapping our assertions in an `expect` lambda block.
- With functional code, you often provide input and test output or else define some property and test it with any inputs, so there is no need to mock objects.
- An important principle of functional design is that instead of one big function, it is better to split functionality among many small functions and compose them together.

Chapter 2 - Handling HTTP Using Functions

- The fundamental rule of designing functional programs is to examine what we have (inputs) and what we want (outputs) to decide the function we need.
- A Web Server or `HttpHandler` in functional terms is just a function that transforms a request into a response. `(Request) -> Response`
- A function `(Request) -> Response` is a *service*. A *filter* is a function that takes a service and returns another service.
- We will be using http4k as our web server. This is written in a functional manner despite being built over Jetty which isn't particularly functional.

```
val greetingPage = """
<html>
    <body>
        <h1 style="text-align:center; font-size:3em;">
        </h1>
    </body>
</html>
"""

val handler: HttpHandler = { Response(Status.OK).body(greetingPage) }

fun main() {
    handler.asServer(Jetty(8080)).start()
}
```

- We can store functions in lambda values for each route or pass named functions directly.

```
val app: HttpHandler = routes(
    "/greetings" bind GET to ::greetings,
    "/data" bind POST to ::receiveData,
    "/todo/{user}/{list}" bind GET to ::showList
)

fun greetings(req: Request): Response = Response(OK).body(greetingPage)

fun receiveData(req: Request): Response = Response(CREATED)
    .body("Received: ${req.bodyString()} ")

fun showList(req: Request): Response {
    val user: String? = req.path("user")
    val list: String = req.path("list").orEmpty()
    val htmlPage = """
<html>
    <body>
        <h1>Zettai</h1>
        <p>Here is the list <b>$list</b> of user <b>$user</b></p>
    </body>
</html>"""
    return Response(OK).body(htmlPage)
}

fun main() {
    app.asServer(Jetty(8080)).start()
}
```

- Note that in creating our domain objects, we define data classes for `ToDoList`, `ToDoItem`, `ToDoStatus` and both `ListName` and `User`. The latter two only contain a string, but it is easier to validate them in an object and it keeps it clear what we are passing to other functions rather than just sending around lots of strings. Note that we also make an `HtmlPage` data class to wrap a string and remind us when we're using it to hold HTML content. We'll add more details to the types later.

```kotlin
// These are defined in domain.ToDoList.kt in example code
data class ListName(val name: String)
data class ToDoList(val listName: ListName, val items: List<ToDoItem>)
data class ToDoItem(val description: String)
// This is defined in domain.User.kt
data class User(val name: String)
// ToDoStatus is not in example code but is in book as an enum
enum class ToDoStatus { ToDo, InProgress, Done, Blocked }
```

- The built-in `TODO()` function throws a `NotImplementedError` and makes an excellent placeholder once you know your function types.
- Once we define the functions and types, we can compose them together even before writing the implementation:

```kotlin
fun extractListData(request: Request): Pair<User, ListName> = TODO()
fun fetchListContent(listId: Pair<User, ListName>): ToDoList = TODO()
fun renderHtml(list: ToDoList): HtmlPage = TODO()
fun createResponse(html: HtmlPage): Response = TODO()

// Kotlin's `let` function takes what would be an inside out, indented list
// of function calls and puts them in order from top to bottom. Each function
// in the chain is fed the output of the previous function
fun getToDoList(request: Request): Response =
  request
      .let(::extractListData)
      .let(::fetchListContent)
      .let(::renderHtml)
      .let(::createResponse)
// `let` works a bit like the pipe operator in shell or `|>` operator in
// JavaScript and F#. Without `let` this would be:
// fun getToDoList(request: Request) : Response =
//     createResponse(
//         renderHtml(
//             fetchListContent(
//                 extractListData(
//                     request
//                 )
//             )
//         )
//     )
```

- Using `let` like above only works for functions with a single input parameter which is why we made fetchListContent take a `Pair`.
- Here is the entire Zettai class for this chapter which uses a Map to keep track of users and lists:

```
// imports and package declaration omitted...
// Note: HtmlPage, renderHtml, and renderItems have been factored out to
// a ui.HtmlPage.kt file in the example code.
data class HtmlPage(val raw: String)

data class Zettai(val lists: Map<User, List<ToDoList>>) : HttpHandler {

    val routes = routes(
        "/todo/{user}/{list}" bind GET to ::getToDoList
    )

    override fun invoke(request: Request): Response =
        routes(request)

    private fun getToDoList(request: Request): Response =
        request.let(::extractListData)
            .let(::fetchListContent)
            .let(::renderHtml)
            .let(::createResponse)


    fun extractListData(request: Request): Pair<User, ListName> {
        val user = request.path("user") ?: error("User missing")
        val list = request.path("list") ?: error("List missing")
        return User(user) to ListName(list)
    }

    fun fetchListContent(listId: Pair<User, ListName>): ToDoList =
        lists[listId.first]
            ?.firstOrNull { it.listName == listId.second }
            ?: error("List unknown")

    fun renderHtml(todoList: ToDoList): HtmlPage =
        HtmlPage(
            """
        <html>
            <body>
                <h1>Zettai</h1>
                <h2>${todoList.listName.name}</h2>
                <table>
                    <tbody>${renderItems(todoList.items)}</tbody>
                </table>
            </body>
        </html>
        """.trimIndent()
```

```
        )

    private fun renderItems(items: List<ToDoItem>) =
        items.map {
            """<tr><td>${it.description}</td></tr>""".trimIndent()
        }.joinToString("")

    fun createResponse(html: HtmlPage): Response =
Response(Status.OK).body(html.raw)

}
```

- Note in the above code that throwing exceptions makes these functions partial. We'll learn better ways to deal with invalid input later. The way we are doing it here would return an unexplained 500 error to the user instead of a helpful 404 page.
- The beauty of the way we defined our `Zettai` class is it is easy to run both normally and in testing. Here is our first acceptance test:

```
// This appears in the example code in stories.SeeATodoListAT
@Test
fun `List owners can see their lists`() {

    val listName = "shopping"
    val foodToBuy = listOf("carrots", "apples", "milk")
    val userName = "frank"

    startTheApplication(userName, listName, foodToBuy)

    val list = getToDoList(userName, listName)

    expectThat(list.listName.name).isEqualTo(listName)
    expectThat(list.items.map { it.description }).isEqualTo(foodToBuy)
}

// Defined to start the server for testing.
// Notice how we're running our test server on a different port (8081).
private fun startTheApplication(
    user: String,
    listName: String,
    items: List<String>,
) {
    val toDoList = ToDoList(ListName(listName), items.map(::ToDoItem))
    val lists = mapOf(User(user) to listOf(toDoList))
    val server = Zettai(lists).asServer(Jetty(8081)) // different from main
    server.start()
}

// We're using http4k's `JettyClient`
private fun getToDoList(user: String, listName: String): ToDoList {
    val client = JettyClient()
    val response = client(Request(Method.GET,
"http://localhost:8081/todo/$user/$listName"))

    return if (response.status == Status.OK)
        parseResponse(response.bodyString())
    else
        fail(response.toMessage())
}

// for now we're parsing the response with regexes
private fun parseResponse(html: String): ToDoList {
    val nameRegex = "<h2>.*<".toRegex()
    val listName = ListName(extractListName(nameRegex, html))
```

```
    val itemsRegex = "<td>.*?<".toRegex()
    val items = itemsRegex.findAll(html)
        .map { ToDoItem(extractItemDesc(it)) }.toList()
    return ToDoList(listName, items)
}

private fun extractListName(nameRegex: Regex, html: String): String =
    nameRegex.find(html)?.value
        ?.substringAfter("<h2>")
        ?.dropLast(1)
        .orEmpty()

private fun extractItemDesc(matchResult: MatchResult): String =
    matchResult.value.substringAfter("<td>").dropLast(1)
```

- And here are the entire final contents of Main.kt which runs our app:

```
package com.ubertob.fotf.zettai.webservice

import com.ubertob.fotf.zettai.domain.ListName
import com.ubertob.fotf.zettai.domain.ToDoItem
import com.ubertob.fotf.zettai.domain.ToDoList
import com.ubertob.fotf.zettai.domain.User
import org.http4k.core.HttpHandler
import org.http4k.server.Jetty
import org.http4k.server.asServer

fun main() {
    val items = listOf("write chapter", "insert code", "draw diagrams")
    val toDoList =
        ToDoList(ListName("book"), items.map(::ToDoItem))
    val lists = mapOf(User("uberto") to listOf(toDoList))

    val app: HttpHandler = Zettai(lists)
    app.asServer(Jetty(8080)).start() //starting the server

    println("Server started at http://localhost:8080/todo/uberto/book")
}
```

- Chapter 2 had a pair of exercises that caught my eye. I was able to create a functional stack class easily enough, but for the reverse Polish notation calculator that uses it I was able to program with the immutable structure but only if I bound it to a var that I could update. The book solves this problem using fold. It's an intuition that I need to build: "Any time I think I

need a `for` loop; I probably need a fold." This really made me think about the fact that an accumulator is simply a very scope-limited `var` since it actually gets updated through the steps of a fold. Each step doesn't mutate the value in place; `fold` still works with immutable data structures, but `acc` gets rebound to a new immutable value on every iteration. Note that it is possible to reproduce `fold` without mutation through recursion and passing a new `acc` on every call.

```
// First, an immutable stack.
data class FunStack<T>(private val elements: List<T> = emptyList()) {
    fun push(element: T): FunStack<T> = FunStack(listOf(element) + elements)
    fun pop(): Pair<T, FunStack<T>> = elements.first() to
FunStack(elements.drop(1))
    fun size(): Int = elements.size
}

// Now the RPN Calculator that uses it.
object RpnCalc {

    val operationsMap = mapOf<String, (Double, Double) -> Double>(
        "+" to Double::plus,
        "-" to Double::minus,
        "*" to Double::times,
        "/" to Double::div
    )

    val funStack = FunStack<Double>()

    fun calc(expr: String): Double =
        expr.split(" ")
            .fold(funStack, ::reduce)
            .pop().first


    private fun reduce(stack: FunStack<Double>, token: String):
FunStack<Double> =
        if (operationsMap.containsKey(token)) {
            val (b, tempStack) = stack.pop()
            val (a, newStack) = tempStack.pop()
            newStack.push(operation(token, a, b))
        } else {
            stack.push(token.toDouble())
        }

    private fun operation(token: String, a: Double, b: Double) =
        operationsMap[token]?.invoke(a, b) ?: error("Unknown operation
$token")

}
```

Chapter 3 - Defining the Domain and Testing It

- Goal: make our acceptance tests better.
    - Our acceptance tests should only use domain terms and should not mention any technical details like UI elements ("click the X button") or the transmission protocol ("assert that header Y is present").
    - Each test should represent a use-case scenario; all tests in a file form a user-story.
    - Tests should focus on use-case actors and their interactions.
    - Assertions should be represented by methods with clear domain names, leaving the actual implementation out of the test itself, since they can be quite complicated to read.
- Let's clean up our one acceptance test by defining a type of `ScenarioActor`, the `ToDoListOwner`.

```
interface ScenarioActor {
    val name: String
}

class ToDoListOwner(override val name: String): ScenarioActor {
    // this replaces the assertions in our old test
    fun canSeeTheList(listName: String, items: List<String>) {
        val expectedList = createList(listName, items)
        val list = getToDoList(name, listName)
        expectThat(list).isEqualTo(expectedList)
    }

    fun cannotSeeTheList(listName: String) {
        expectThrows<AssertionFailedError> {
            app.getToDoList(name, listName)
        }
    }

    private fun getToDoList(user: String, listName: String): ToDoList {
        // moved from the test to the actor
    }
}

// Utility functions
private fun createList(listName: String, items: List<String>) =
    ToDoList(ListName(listName), items.map(::ToDoItem))

// Then we can rewrite our test.
@Test
fun `List owners can see their lists`() {
    val listName = "shopping"
    val foodToBuy = listOf("carrots", "apples", "milk")
    val frank = ToDoListOwner("Frank")

    startTheApplication(frank.name, createList(listName, foodToBuy))

    frank.canSeeTheList(listName, foodToBuy)
}
```

- This is already a lot clearer to read, but we can make it even better by extracting a class to work as an abstraction for the application so we pass the Actor an application reference and it only works when it has such a reference.

- We will be using the Facade design pattern. The Facade is a design pattern in which an object stands as a facade or proxy for a complex system, hiding the more complex innards and making it easy to interact with. We want to use it to extract information and give commands to our application without cluttering the test with technical details. Our goal is to create an interface that allows us to query the application state and change it. It will act as a contract that outlines all possible user actions on our application.
- To start, we move `getToDoList` and other private methods of the test to a new class `ApplicationForAT` which we pass to our `ToDoListOwner`. We're also going to pass in a `client` and `server` reference to our `ApplicationForAT` so it handles opening and closing the connection and doesn't need to manage host and port.

```
class ApplicationForAT(val client: HttpHandler, val server: AutoCloseable) {
    // this is implementation specific, so belongs here; not in ToDoListOwner
    fun getToDoList(user: String, listName: String): ToDoList {

        val response = client(Request(Method.GET, "/todo/$user/$listName"))

        return if (response.status == Status.OK)
            parseResponse(response.bodyString())
        else
            fail(response.toMessage())
    }

    fun runScenario(steps: (ApplicationForAT) -> Unit) {
        server.use {
            steps(this)
        }
    }

// rest of methods: parseResponse, extractListName, extractItemsDesc
}

fun startTheApplication(lists: Map<User, List<ToDoList>>): ApplicationForAT {

    val port = 8081
    val server = Zettai(lists).asServer(Jetty(port))
    server.start()

    val client = ClientFilters
        .SetBaseUriFrom(Uri.of("http://localhost:$port/"))
        .then(JettyClient())
    return ApplicationForAT(client, server)
}

// Assuming we modified `cannotSeeTheList` to take an `app`,
// how would we use this new Facade
@Test
fun `Only owners can see their lists`() {

    val app = startTheApplication(list)
    app.runScenario {
        frank.cannotSeeTheList("gardening", it)
        bob.cannotSeeTheList("shopping", it)
    }
}
```

- The main remaining problem with the above is we need to pass app into our actor methods that require the application. The best way to fix that is to treat the step function as if it were data. Note that we aren't really adding a dependency here when we add the application as parameter, because the Actor already required that we start the application before any of his methods to see his list would work; we're just making the dependency explicit.
- Instead of having a method that takes a parameter (in this case, actor methods that take an application), we can create a new function that requires that parameter and returns the function.
    - The first step is defining an interface with all the Actions an Actor can use on our App. That's simple for now, it's just: `getToDoList`. This is implemented by our `ApplicationForAT`.

```
interface Actions {
    fun getToDoList(user: String, listName: String): ToDoList?
}

// And a typealias for this unwieldy type. This is an extension function
// with `Action` as a receiver.
typealias Step = Actions.() -> Unit

// And modify runScenario in ApplicationForAT to take a variable
// number of steps.
fun runScenario(vararg steps: Step) {
  server.use {
    // We pass each step an `Actions` interface as a  parameter.
    steps.onEach { step  -> step(this) }
  }
}

// And back in our Actor, we return a step as a new function:
fun cannotSeeTheList(listName: String): Step = {
  getToDoList(name, listName)
}

// Now our tests don't need to be passed an ApplicationForAT
@Test
fun `List owners can see their lists`() {
    val app = startTheApplication(lists)
    app.runScenario(
        frank.canSeeTheList("shopping", shoppingItems),
        bob.canSeeTheList("gardening", gardenItems)
    )
}

@Test
fun `Only owners can see their lists`() {
    val app = startTheApplication(lists)
    app.runScenario(
        frank.cannotSeeTheList("gardening"),
        bob.cannotSeeTheList("shopping")
    )
}
```

- Don't read too much into the typealias syntax even though it's unfamiliar. Basically, it lets the lambda acts a receiver on `Actions` to run inside the context of the `Actions` instance Any class implementing the Actions interface becomes eligible to serve as the

receiver when a Step is invoked on it — that eligibility comes from the interface, but the actual binding happens at the call site, not at implementation time. We're simply limiting ourselves to Steps that take no input and produce no output, hence they're executed for side effects only.

    - Functional programming is the art of minimizing the code depending on side effects and keeping it at the outer edge of your program.
    - A higher-order function is pure if it preserves referential transparency when given a pure function as an input. If it has a side effect when you give it an impure function, it is still pure and the signature makes clear that it is calling this impure function which has a type that goes to Unit.

- Each module should have only one reason to change, so business logic (anything you can discuss without any technical terms) should be kept separate from the technical implementation.
- The Port and Adapter Pattern (or Hexagonal Architecture) involves a layer of abstraction between the core logic and its external dependencies known as ports and adapters. Ports define the interfaces through which the core logic communicates with the external dependencies, while adapters implement those interfaces and provide the actual communication.
    - In what we've done so far, `User`, `ListName`, and `ToDoList` are part of our domain model, while `HtmlPage`, `Request`, and `Response` are technical details of our implementation. So functions that have a domain type as input and output are part of our domain, or the hub, and others are part of the spokes that connect the hub with the external world.
        - `extractListData`, `(Request)->Pair<User, ListName>`, spoke
        - `fetchListContent`, `(Pair<User, ListName>)->ToDoList`, hub
        - `renderHtml`, `(ToDoList)->HtmlPage`, spoke
        - `createResponse`, `(HtmlPage)->Response`, spoke
    - The hub functions stay within the business domain are part of the "hub", and functions that cross from domain to technical details are "spokes".
    - Imagine how we want to use our `ZettaiHub`:

```
@Test
// Technical, normally the world reaches into the Hub, or the Hub reaches
// out through adapters. `getList` is our adapter.
fun `get list by user name`() {
    val hub = ZettaiHub(listMap)
    val myList = hub.getList(user, list.listName)
    expectThat(myList).isEqualTo(list)
}

// So we need one function on our Interface. Note this is Actions above.
interface ZettaiHub {
    fun getList(user: User, listName: ListName): ToDoList?
}

// So we implement this as follows
class ToDoListHub(val lists: Map<User, List<ToDoList>>): ZettaiHub {
    override fun getList(user: User, listName: ListName): ToDoList? =
        lists[user]
        ?.firstOrNull { it.listName == listName }
}

// Then we pass the hub to the Zettai class constructor instead of a Map
// of ToDoLists
data class Zettai(val hub: ZettaiHub): HttpHandler {
    // rest of the methods...

    fun fetchListContent(listId: Pair<User, ListName>): ToDoList =
        hub.getList(listId.first, listId.second)
        ?: error("List unknown")
}
```

    - With the hub, the inside of the hub should stay functionally pure, and we provide external functions from the outside.
    - Note that if we are keeping our tests close to the domain, the `Actions` available will closely match our Hub methods.
- Domain-Driven Tests (DDT) capture the basic idea that acceptance tests should be written using business domain terms. It's also a pun on DDT the pesticide being good at killing bugs. Programmers love puns.
- We can take tests written in domain terms and run them several times using different methods of connecting to the domain, which we'll refer to as *protocols*. Basically, we may use an in-memory implementation of the hub that tests basic domain logic, then run the

tests again using a locally started application that services http calls, then a third time running the tests again an externally deployed service to test the network and deployment scripts. So a "domain-only protocol", a "local HTTP protocol", and a "remote HTTP protocol" will exist for all the tests. To do this, we want a single interface that represents our actions and abstracts different implementations. The overall goal of the approach is to describe our stories as interactions between human actors and the system domain. Advantages include:

    1. Fast failure feedback on the domain-only implementation
    2. End-to-end functionality testing on the HTTP implementation
    3. verification that there is no business logic in the infrastructure layer
    4. Verification that there is no infrastructure dependency in the business layer.
    5. Another advantage is this forces us to implement our domain layer without relying on technical details. It is too easy to make a business rule based upon the value of an HTTP header when you don't have to define it without relying on details of the technical implementation.

- In this book we will use the lightweight library Pesticide, written by the author, to implement DDT, but it really just saves us some simple boilerplate code. It isn't unwieldy to manually start each version of the app and run each test against the app with each backend, but it becomes a bit of boilerplate for each test that adds up. But never forget, DDT does not require Pesticide.

```
typealias ZettaiDDT = DomainDrivenTest<ZettaiActions>

fun allActions() = setOf(
    DomainOnlyActions(),
    HttpActions()
)

// each test class has all the scenario to define a single user story.
class SeeATodoListDDT : ZettaiDDT(allActions()) {

    @DDT
    fun `List owners can see their lists`() = ddtScenario {
        // here put the actors steps...
    }
}

interface ZettaiActions : DdtActions<DdtProtocol> {

    fun getToDoList(user: User, listName: ListName): ToDoList?
}

class DomainOnlyActions() : ZettaiActions {
    override val protocol: DdtProtocol = DomainOnly
    override fun prepare() = Ready

    private val lists: Map<User, List<ToDoList>> = emptyMap()

    private val hub = ToDoListHub(lists)

    override fun getToDoList(user: User, listName: ListName): ToDoList? =
        hub.getList(user, listName)
}

class HttpActions(val env: String = "local"): ZettaiActions {
    override val protocol: DdtProtocol = Http(env)
    override fun prepare(): DomainSetUp = TODO("launch the app")

    override fun getToDoList(user: User, listName: ListName): ToDoList =
        TODO("not implemented yet")
}
```

- Remember, we use Actors in our tests and do the assertions within the Actors. This keeps us focused on Actions performed by real users of our software and also keeps us from scattering assertions all over the place.

```
// The Actor can only act on the application using the DDT actions.
Therefore,
// we pass the ZettaiAction interface as a generic parameter to the Actor.
data class ToDoListOwner(override val name: String): DdtActor<ZettaiActions>
()
```

- We describe the user story a list of steps, each one a method on an Actor, and inside each step the actor interacts with the application using the current actions. A good rule of thumb is to separate the steps that query the application from the steps that operate a change in the application. For example, the step to add an item here won't contain any assertions, and the step to verify the list won't change anything in the application. We also keep the assertions in the steps, so the same assertions work for domain-only and HTTP actions. If we need different assertions for different protocols we have either leaked technical details or have some business logic in our adapters.

```
class SeeATodoListDDT : ZettaiDDT(allActions()) {

    val frank by NamedActor(::ToDoListOwner)

    val bob by NamedActor(::ToDoListOwner)

    val shoppingListName = "shopping"
    val shoppingItems = listOf("carrots", "apples", "milk")

    val gardenListName = "gardening"
    val gardenItems = listOf("fix the fence", "mowing the lawn")

    @DDT
    fun `List owners can see their lists`() = ddtScenario {

        setUp {
            // This step associates a list with a user in the map in the
            // hub, but it is a temporary hack, and we will see a better
            // solution in the next chapter.
            frank.`starts with a list`(shoppingListName, shoppingItems)
            bob.`starts with a list`(gardenListName, gardenItems)
        }.thenPlay(
            frank.`can see #listname with #itemnames`(
                shoppingListName, shoppingItems),
            bob.`can see #listname with #itemnames`(
                gardenListName, gardenItems)
        )
    }

    @DDT
    fun `Only owners can see their lists`() = ddtScenario {

        setUp {
            frank.`starts with a list`(shoppingListName, shoppingItems)
            bob.`starts with a list`(gardenListName, gardenItems)
        }.thenPlay(
            frank.`cannot see #listname`(gardenListName),
            bob.`cannot see #listname`(shoppingListName)
        )
    }
}

// In the above list, we use our redesigned ToDoListOwner.
data class ToDoListOwner(override val name: String):
        DdtActor<ZettaiActions>() {
```

```
    val user = User(name)

    // the hash signs in the method names will be substituted with the
    // parameters we pass to the step function.
    fun `can see #listname with #itemnames`(
            listName: String,
            expectedItems: List<String>) =

        step(listName, expectedItems) {

            val list = getToDoList(user, ListName(listName))

            expectThat(list)
                .isNotNull()
                .itemNames
                .containsExactlyInAnyOrder(expectedItems)
        }

    private val AssertionBuilder<ToDoList>.itemNames
        get() = get { items.map { it.description } }
}

// Now we need some HttpActions to test. Go back and complete those TODO's!
// We're only testing locally now, but this will work for a remote deployment
// later. We'll do that later!
data class HttpActions(val env:String="local"): ZettaiActions {

    override val protocol: DdtProtocol = Http(env)

    val zettaiPort = 8000 // test on a different port than we normally run
    val server = Zettai(hub).asServer(Jetty(zettaiPort))

    val client = JettyClient()

    override fun prepare(): DomainSetUp {
        server.start()
        return Ready
    }

    // `also` runs a method for its side effects then returns `this`.
    override fun tearDown(): HttpActions =
        also { server.stop() }

    private fun callZettai(method: Method, path: String) : Response =
        client(log(Request(
```

```
            method,
            "http://localhost:$zettaiPort/$path")))
}
```

- Note that code in the repository for this chapter is more fleshed out that the code in the book and these notes. It includes more actions such as adding things to a list and parsing the html results and a way to initialize the Hub from a `ToDoListFetcher` and includes validations on `ListName` values. The test for adding new items and the route to handle posts for new items are the biggest differences. It also has some random generators and basic property based tests such as random user's not being able to access lists of other random users and being able to get their own lists. Here the "add a list code":

```
class ModifyAToDoListDDT : ZettaiDDT(allActions()) {

    val ann by NamedActor(::ToDoListOwner)

    @DDT
    fun `the list owner can add new items`() = ddtScenario {

        setUp {
            ann.`starts with a list`("diy", emptyList())
        }.thenPlay(
            ann.`can add #item to #listname`("paint the shelf", "diy"),
            ann.`can add #item to #listname`("fix the gate", "diy"),
            ann.`can add #item to #listname`("change the lock", "diy"),
            ann.`can see #listname with #itemnames`(
                "diy", listOf(
                    "fix the gate", "paint the shelf", "change the lock"
                )
            )
        )
    }
}

// This requires that the Hub be initialized with a `mutableMapOf()`
// in both `DomainOnlyActions` and `HttpActions`.
// It also requires additional actions. This is from `DomainOnlyActions`:
override fun addListItem(user: User, listName: ListName, item: ToDoItem) {
  hub.addItemToList(user, listName, item)
}
// While I'm in `DomainOnlyActions` it occurs to me we have used
// `starts with a list` multiple time but it isn't defined in any
// examples.
override fun ToDoListOwner.`starts with a list`(listName: String, items:
List<String>) {
    val newList = ToDoList.build(listName, items)
    fetcher.assignListToUser(user, newList)
}

// And here are both from `HttpActions`, along with submitToZettai which
// is required for the post.
override fun addListItem(user: User, listName: ListName, item: ToDoItem) {

    val response = submitToZettai(
        todoListUrl(user, listName),
        listOf("itemname" to item.description, "itemdue" to
item.dueDate?.toString())
```

```
    )

    expectThat(response.status).isEqualTo(Status.SEE_OTHER)
}

private fun todoListUrl(user: User, listName: ListName) =
      "todo/${user.name}/${listName.name}"

private fun submitToZettai(path: String, webForm: Form): Response =
    client(log(Request(Method.POST,
"http://localhost:$zettaiPort/$path").body(webForm.toBody())))
// While we're looking at functions in `HttpActions` note that in the
// repository we are using cssItems to parse responses instead of Regex.
  private fun extractItemsFromPage(html: HtmlPage): List<ToDoItem> =
      html.parse()
          .select("tr")
          .filter { it.select("td").size == 3 }
          .map {
              Triple(
                  it.select("td")[0].text().orEmpty(),
                  it.select("td")[1].text().toIsoLocalDate(),
                  it.select("td")[2].text().orEmpty().toStatus()
              )
          }
          .map { (name, date, status) ->
              ToDoItem(name, date, status)
  }


override fun ToDoListOwner.`starts with a list`(listName: String, items:
List<String>) {
    fetcher.assignListToUser(
        user,
        ToDoList(ListName.fromUntrustedOrThrow(listName), items.map {
ToDoItem(it) })
    )
}

// And the function from `ToDoListOwner` for adding:
fun `can add #item to #listname`(itemName: String, listName: String) =
step(itemName, listName) {
    val item = ToDoItem(itemName)
    addListItem(user, ListName.fromUntrustedOrThrow(listName), item)
}

// And for completeness here is the `ZettaiActions` interface:
```

```
interface ZettaiActions : DdtActions<DdtProtocol> {
    fun ToDoListOwner.`starts with a list`(listName: String, items:
List<String>)

    fun getToDoList(user: User, listName: ListName): ToDoList?
    fun addListItem(user: User, listName: ListName, item: ToDoItem)
}

// And of course we need a little more functionality in our Hub.
// Here is the whole `ToDoListHub` file.
interface ZettaiHub {
    fun getList(user: User, listName: ListName): ToDoList?
    fun addItemToList(user: User, listName: ListName, item: ToDoItem):
ToDoList?
}

class ToDoListHub(private val fetcher: ToDoListUpdatableFetcher) : ZettaiHub
{

    override fun getList(user: User, listName: ListName): ToDoList? =
        fetcher(user, listName)

    // notice that this uses an extension function to upsert the item.
    override fun addItemToList(user: User, listName: ListName, item:
ToDoItem): ToDoList? =
        fetcher(user, listName)?.run {
            val newList = copy(items = items.upsertItem(item))
            fetcher.assignListToUser(user, newList)
        }

    private fun List<ToDoItem>.upsertItem(item: ToDoItem) = filterNot {
it.description == item.description } + item
}

// Note that a ToDoListUpdatableFetcher just assigns a store to `val store`
// and provides `invoke` and `assignToUser` methods:
typealias ToDoListFetcher = (user: User, listName: ListName) -> ToDoList?

interface ToDoListUpdatableFetcher : ToDoListFetcher {

    override fun invoke(user: User, listName: ListName): ToDoList?

    fun assignListToUser(user: User, list: ToDoList): ToDoList?

}
```

```
typealias ToDoListStore = MutableMap<User, MutableMap<ListName, ToDoList>>

data class ToDoListFetcherFromMap(
    private val store: ToDoListStore
) : ToDoListUpdatableFetcher {
    override fun invoke(user: User, listName: ListName): ToDoList? =
        store[user]?.get(listName)

    override fun assignListToUser(user: User, list: ToDoList): ToDoList? =
        store.compute(user) { _, value ->
            val listMap = value ?: mutableMapOf()
            listMap.apply { put(list.listName, list) }
        }?.let { list }

}
```

- Note that most of the code above lives in the test folder except the hub and fetcher. We'll flesh out or kotlin folder later. Mostly now it is hub, fetch, and utility classes like `HtmlPage` or `User` plus a simple `Main.kt` and `Routes.kt`. It is notable that in our routes we are chaining methods like the earlier example but pairing them with a `?:` to do `Status.NOT_FOUND` or `Status.BAD_REQUEST` in case of null.

- In my on going, "Authors Cool Use of Fold in the Exercises" expose, I'd like to focus on this elegant fold in a `renderTemplate` function:

```
data class StringTag(val text: String)

infix fun String.tag(value: String): Pair<String, StringTag> =
    this to StringTag(value)

fun renderTemplate(template: String, data: Map<String, StringTag>): String =
    data.entries.fold(template) { acc, (key, value) ->
        acc.replace("{$key}", value.text)
    }

// And you can see it exercised in this test:
class E04_SimpleTemplateTest {

    @Test
    fun `happy birthday template`() {
        val template = """
            Happy Birthday {name} {surname}!
            from {sender}.
            """.trimIndent()
        val data = mapOf(
            "name" tag "Uberto",
            "surname" tag "Barbini",
            "sender" tag "PragProg"
        )
        val actual = renderTemplate(template, data)
        val expected = """
            Happy Birthday Uberto Barbini!
            from PragProg.
            """.trimIndent()
        expectThat(actual).isEqualTo(expected)
    }
}
```

Chapter 4 - Modeling the Domains and the Adapters

- We actually implement that "add to list" test first in this chapter, but you can see all the final code above. He has an excellent suggestion that to implement a new DDT and get it compiling, just write out the Actor steps as empty functions until we're satisfied with how the scenario reads.
- The book has an aside about why we test the way we do. We want our unit test to test properties hold with random values. They are quick and easy to run, so this is natural. For

our scenarios, we use hand picked, concrete values since it's easier to talk to business stakeholders about concrete examples rather than universal properties.

- The Domain-Driven Test Process looks like a V:
    1. We start with writing the Http version first so we can sort out the "plumbing" of our architecture.
    2. When we arrive at the point where we need some domain logic, we switch to InMemory/DomainOnly DDTs and let them guide us in modeling the domain.
    3. Then, we develop the needed components one by one, using unit tests.
    4. After that, we fix the DomainOnly DDTs until they pass.
    5. Finally, we return to the Http DDT and make sure the final infrastructure is working as expected.
- Note that the author's test library has a `wip` method that lets you mark a test as a work in process so you don't have to ignore it failing until you build out all the functionality.

```
class ModifyAToDoListDDT: ZettaiDDT(allActions()) {

    val ann by NamedActor(::ToDoListOwner)

    @DDT
    fun `The list owner can add new items`() = ddtScenario {
        setUp {
            ann.`starts with a list`("diy", emptyList())
        }.thenPlay(
            ann.`can add #item to #listname`("paint the shelf", "diy"),
            ann.`can add #item to #listname`("fix the gate", "diy"),
            ann.`can add #item to #listname`("change the lock", "diy"),
            ann.`can see #listname with #itemnames`("diy", listOf(
                "fix the gate", "paint the shelf", "change the lock"))
        ).wip(LocalDate.of(2026, 7, 12), "Not implemented yet")
    }
}
```

- Note that we started with HTTP first since we are going "outside-in" and user actions normally take place at the external layer of the system.
- I got ahead of myself and listed all this "add to list" code last chapter, since I didn't understand the step folders of the project didn't map to the chapters 1-to-1, since Chapter 2 was all of step 1.

- Let's spike on using higher order functions to decide routes. Imagine we have each of the CRUD operations a c, r, u, and d. We could define a `routes` function as follows:

```
// Here we are taking the different handlers and returning a handler that
// handles them all through a higher order function.
// Remember and HttpHandler is just a function (Request) -> Response
fun routes(c: HttpHandler, r: HttpHandler,
           u: HttpHandler, d: HttpHandler
): HttpHandler = { request ->
    when ("${request.method} ${request.url.path}") {
        "POST /data" -> c(request)
        "GET /data" -> r(request)
        "PUT /data" -> u(request)
        "DELETE /data" -> d(request)
        else -> Response(404, "NOT FOUND")
    }
}
```

- The good news is, we don't have to do that, because http4k has a Domain Specific Language that lets us define all routes in a declarative way.

- I didn't list the new routes that make the HTTP POST work last chapter:

```
class Zettai(val hub: ZettaiHub): HttpHandler {

    val httpHandler = routes(
        "/todo/{user}/{listname}" bind GET to ::getToDoList,
        "/todo/{user}/{listname}" bind POST to ::addNewItem
    )

    fun addNewItem(request: Request): Response {
        val user = request.path("user")
            ?.let(::User)
            ?: return Response(Status.BAD_REQUEST)
        val listName = request.path("listname")
            ?.let(::ListName)
            ?: return Response(Status.BAD_REQUEST)
        val item = request.form("itemname")
            ?.let(::ToDoItem)
            ?: return Response(Status.BAD_REQUEST)

        // Now we have everything we need from HTTP, so we let the
        // hub handle the domain action.
        hub.addItemToList(user, listName, item)
            ?.let { Response(Status.SEE_OTHER)
                .header("Location", "/todo/${user.name}/${listName.name}") }
            ?: Response(Status.NOT_FOUND)
    }
}

// We saw ToDoListHub.addItemToList earlier.
```

- The domain action simply delegates to the hub, since the hub handles everything within the domain.
- Dependency Injection is as important in FP as OOP. If our code needs to use some external dependencies, it should avoid creating them directly, and it should receive them from outside instead. In FP, we pass dependencies as functions to the main function by using another function that accepts dependencies as arguments and generates a new function with those dependencies embedded within. Let's see a concrete example:

```
// We need a function that retrieves a particular list given a `User`
// and a `ListName`.
typealias ToDoListFetcher = (User, ListName) -> ToDoList?

typealias ToDoListMap = Map<User, Map<ListName, ToDoList>>

fun mapFetcher(map: ToDoListMap,
               user: User,
               list: ListName): ToDoList? =
    map[user]?.get(list)
// Note how mapFetcher looks suspiciously like `ToDoListFetcher` if the
// `map` parameter was set. We need a way to partially apply a function.

fun <A, B, C, R> partial(f: (A, B, C) -> R, a: A): (B, C) -> R = {
    b, c -> f(a, b, c)
}

val fetcher: ToDoListFetcher = partial(::mapFetcher, map)

// So how do we use this in the Hub?
class ToDoListHub(val fetcher: ToDoListFetcher) : ZettaiHub {

    override fun getList(user: User, listName: ListName): ToDoList? =
        fetcher(user, listName)

    override fun addItemToList(user: User,
        listName: ListName, item: ToDoItem): ToDoList? = TODO()
}

// Now to use it in a test.
class ToDoListHubTest {

    val fetcher = TODO("we need an implementation!")

    val hub = ToDoListHub(fetcher)

    @Test
    fun `get list by user name name`() {
        repeat(10) {
            val user = randomUser()
            val list = randomToDoList()

            // TODO ("assign the list to the user!")

            val myList = hub.getList(user, list.listName)
```

```
            expectThat(myList).isEqualTo(list)
        }
    }
}
```

- I know I spoiled some of the surprise by listing these complete classes earlier, but it's good to see how their built up. Now, we don't really need `mapFetcher` or `partial` because of features of Kotlin. A class can inherit from a function type and be invokable as if it was a function. You can pass a member of this invokable class anywhere a function with the same signature if needed. We just put the function class in the type declaration where the superclass would have been, and then override the invoke method. There are three advantages to this:

    1. Any constructor parameter will be partially applied to the needed function.
    2. Complex code can be neatly contained in private functions.
    3. A state needed for an API can be stored between calls.

```
typealias ToDoListStore = MutableMap<User, MutableMap<ListName, ToDoList>>

data class ToDoListFetcherFromMap(
    private val store: ToDoListStore
): ToDoListFetcher {

    override fun invoke(user: User, listName: ListName): ToDoList? =
        store[user]?.get(listName)

    // This looks complicated, but it just gets the map for the
    // user if it exists and otherwise starts a new `mutableMap` for
    // the user then adds listName, list to the user's internal map
    // and returns the list it was passed.
    // This is idiomatic Kotlin for get-or-create-then-mutate on
    // a map, so a useful pattern to remember.
    fun assignListToUser(user: User, list: ToDoList): ToDoList? =
        store.compute(user) { _, value ->
            val listMap = value ?: mutableMapOf()
            listMap.apply { put(list.listName, list) }
            }?.let { list }
}

// So now we can create out Test
class ToDoListHubTest {

    fun emptyStore(): ToDoListStore = mutableMapOf()

    val fetcher = ToDoListFetcherFromMap(emptyStore())

    val hub = ToDoListHub(fetcher)

    @Test
    fun `get list by user and name`() {
        repeat(10) {
            val user = randomUser()
            val list = randomToDoList()

            fetcher.assignListToUser(user, list)

            val myList = hub.getList(user, list.listName)

            expectThat(myList).isEqualTo(list)
        }
    }
```

```
    @Test
    fun `don't get list from other users`() {
        repeat(10) {
            val firstList = randomToDoList()
            val secondList = randomToDoList()
            val firstUser = randomUser()
            val secondUser = randomUser()

            fetcher.assignListToUser(firstUser, firstList)
            fetcher.assignListToUser(secondUser, secondList)

            expect {
                that(hub.getList(firstUser, secondList.listName)).isNull()
                that(hub.getList(secondUser, firstList.listName)).isNull()
            }
        }
    }
}
```

- I already listed the code for `ToDoListUpdatableFetcher` and the `ToDoListhub` with `addItemToList` method earlier. Notice how we added a new `ToDoListUpdatableFetcher` interface for this method.
- Note that backing this by a mutable map is a little cheating. The fetcher acts as an adapter to an external mutable state, but our hub remains pure. We'll use a better solution in the following chapters.
- In general, extracting smaller functions can clarify code by introducing new explanatory names for intent and having additional type signatures to help if the code isn't compiling.
- One useful debugging trick is a `printIt` extension function. Here we are also using an intermediate variable to hold the new list while testing. It can be helpful to break a functional chain into intermediate variables and steps and is still purely functional.

```
// Thanks to `also`, `printIt` returns the same value it extends so it
// can participate in chains without breaking anything.
fun <T> T.printIt(prefix: String = ">"): T = also{ println("$prefix $this") }

// Now we can debug our fetcher by printing the lists content before
// and after replacing the item.
fetcher(user, listName)?.run {
    val newList = copy(items = items.printIt("orig")
        .upsertItem(item).printIt("upserted"))
            .printIt("newList")
    fetcher.assignListToUser(user, newList)
}
```

- Functional Domain Modeling is based on transformations (arrows) of data types and their compositions. So for mapping a business process, we focus on the data that gets exchanged and its transformation. We define functions that take some data (ex: CustomerOrder), perform some computation (calculate the total cost), and return some other data as output (the final Invoice). The transition from object-oriented design to functional design in a nutshell is a transition from looking at your domain as entities collaborating with each other to looking at it as a network of transformations of immutable pieces of information.

- So far, we've discussed two principles of defining good functional data structures:

    1. Immutability - operating with immutable data structures allows us to use them inside pure functions.
    2. Naming - denoting our types with precise names instead of using primitives makes the signature of our function clearer and their intent easier to understand.
    3. A third important property of a type is cardinality: the number of possible values. The less values a type can have, the harder it is for it to express values that don't have a meaning in the domain.
- To limit the cardinality of our `ListName`, we will make the constructor private and add two public static constructors for when it is being constructed from a trust or untrusted source. We want our `ListName` values to be alphanumeric so they can be part of the URL and of reasonable length, say no more than 40 characters. For our untrusted constructor, we choose to return `null` for invalid values. Since we don't care about the reason for failure, it is simplest to return `null` and handle it through Kotlin language features. When we care about the source of error, there are better ways to handle it we will learn later.

```
data class ListName internal constructor(val name: String) {
    companion object {
        fun fromTrusted(name: String): ListName = ListName(name)
        fun fromUntrusted(name: String): ListName? = TODO("not defined yet")
    }
}
```

- So Property testing gives us a ready made way to test our new `fromUntrusted` function. For this, we are going to need to use Kotlin's `generateSequence`. Note: this appear in `tooling.Generators.kt` in the sample code.

```
const val uppercase = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
const val lowercase = "abcdefghijklmnopqrstuvwxyz"
const val digits = "0123456789"

fun stringsGenerator(charSet: String, minLen: Int, maxLen: Int)
        : Sequence<String> = generateSequence {
    randomString(charSet, minLen, maxLen)
}

fun randomString(charSet: String, minLen: Int, maxLen: Int) =
    StringBuilder().run {
        repeat(Random.nextInt(maxLen - minLen) + minLen) {
            append(charSet.random())
        }
        toString()
}

fun substituteRandomChar(fromCharset: String, intoString: String): String =
  intoString
      .toCharArray()
      .apply { set(Random.nextInt(intoString.length), fromCharset.random()) }
      .joinToString(separator = "")

// These appear in ToDoListTest.kt.
val validCharset = uppercase + lowercase + digits + "-"
val invalidCharset = " !@#$%^&*()_+={}[]|:;'<>,./?\u2202\u2203\u2204\u2205"

@Test
fun `Valid names are alphanum+hyphen between 3 and 40 chars length`() {

    stringsGenerator(validCharset, 3, 40)
        .take(100)
        .forEach {
            expectThat(ListName.fromUntrusted(it))
                .isEqualTo(ListName.fromTrusted(it))
        }
}

@Test
fun `Name cannot be empty`() {
    expectThat(ListName.fromUntrusted("")).isEqualTo(null)
}

@Test
fun `Names longer than 40 chars are not valid`() {
```

```
    stringsGenerator(validCharset, 41, 200)
        .take(100)
        .forEach {
            expectThat(ListName.fromUntrusted(it)).isEqualTo(null)
        }
}

@Test
fun `Invalid chars are not allowed in the name`() {

    stringsGenerator(validCharset, 1, 30)
        .map { substituteRandomChar(invalidCharset, it) }
        .take(1000).forEach {
            expectThat(ListName.fromUntrusted(it)).isEqualTo(null)
        }
}

// Now we can write our `fromUntrusted` function. Note this lives in
// ToDoList.kt. I'm going to reproduce the entire `ListName` and `ToDoList`
// definitions for convenience since it's short. The example code also
// defines `fromUntrustedOrThrow` which doesn't substitute `null` and is
// used by the actual `ToDoList` which the book hasn't discussed yet.
fun String.capitalize() = replaceFirstChar {
    if (it.isLowerCase()) it.titlecase(
        Locale.getDefault()
    ) else it.toString()
}

val pathElementPattern = Regex(pattern = "[A-Za-z0-9-]+")

data class ListName internal constructor(val name: String) {
    companion object {
        fun fromTrusted(name: String): ListName = ListName(name)
        fun fromUntrustedOrThrow(name: String): ListName =
            fromUntrusted(name) ?: throw IllegalArgumentException("Invalid
list name $name")

        fun fromUntrusted(name: String): ListName? =
            if (name.matches(pathElementPattern) && name.length in 1..40)
fromTrusted(name) else null
    }
}

// Note the book hasn't discussed `fromUntrustedOrThrow` yet, so the book
// `ToDoList` is just the data class with basic constructor and no ``
```

```
data class ToDoList(val listName: ListName, val items: List<ToDoItem>) {
    companion object {
        fun build(
            listName: String, items: List<String>
        ): ToDoList = ToDoList(ListName.fromUntrustedOrThrow(listName),
            items.map { ToDoItem(it) })
    }
}

// It is our intention here to make dueDate nullable since it isn't required.
data class ToDoItem(
    val description: String,
    val dueDate: LocalDate? = null,
    val status: ToDoStatus = ToDoStatus.Todo
)

enum class ToDoStatus { Todo, InProgress, Done, Blocked }
```

- We will improve the property tests in the next chapter to not stop at the first failure and generate a sample of failure case, in addition to testing corner cases first.
- Our HTML is pretty basic at this point, so we're also going to update `renderPage` to be a little nicer and use Bootstrap since it's the centerpiece of a TODO App. Here is all of `ui.HtmlPage` which was omitted in the book.

```
package com.ubertob.fotf.zettai.ui

import com.ubertob.fotf.zettai.domain.ToDoItem
import com.ubertob.fotf.zettai.domain.ToDoList
import com.ubertob.fotf.zettai.domain.ToDoStatus
import com.ubertob.fotf.zettai.fp.unlessNullOrEmpty
import java.time.LocalDate
import java.time.format.DateTimeFormatter

data class HtmlPage(val raw: String)


fun renderPage(todoList: ToDoList): HtmlPage =
    HtmlPage(
        """
        <!DOCTYPE html>
        <html>
        <head>
            <link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.cs
s" integrity="sha384-
ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T"
crossorigin="anonymous">
            <title>Zettai - a ToDoList application</title>
        </head>
        <body>
        <div id="container">
        <div class="row justify-content-md-center">
        <div class="col-md-center">
            <h1>Zettai</h1>
            <h2>ToDo List ${todoList.listName.name}</h2>
            <table class="table table-hover">
                <thead>
                    <tr>
                      <th>Name</th>
                      <th>Due Date</th>
                      <th>Status</th>
                    </tr>
                </thead>
                <tbody>
                ${todoList.renderItems()}
                </tbody>
            </table>
            </div>
        </div>
```

```
        </div>
        </body>
        </html>
    """.trimIndent()
    )

private fun ToDoList.renderItems() =
    items.map(::renderItem).joinToString("")

private fun renderItem(it: ToDoItem): String = """<tr>
              <td>${it.description}</td>
              <td>${it.dueDate?.toIsoString().orEmpty()}</td>
              <td>${it.status}</td>
            </tr>""".trimIndent()


fun LocalDate.toIsoString(): String =
format(DateTimeFormatter.ISO_LOCAL_DATE)

fun String?.toIsoLocalDate(): LocalDate? =
    unlessNullOrEmpty { LocalDate.parse(this,
DateTimeFormatter.ISO_LOCAL_DATE) }

fun String.toStatus(): ToDoStatus = ToDoStatus.valueOf(this)
```

- Note the new web page breaks our simple Regex parsing functions, so we should use a real parser now. That's why my earlier code example shows it using css selectors instead, but here is that code too. Remember to add `jsoup` to your Gradle dependencies.

```
private fun HtmlPage.parse(): Document = Jsoup.parse(raw)

fun extractItemsFromPage(html: HtmlPage): List<ToDoItem> =
    html.parse()
        .select("tr")
        .filter { it.select("td").size == 3 }
        .map {
            Triple(
                it.select("td")[0].text().orEmpty(),
                it.select("td")[1].text().toIsoLocalDate(),
                it.select("td")[2].text().orEmpty().toStatus()
            )
        }
        .map { (name, date, status) ->
            ToDoItem(name, date, status) }
```

- The book then covers the useful `unlessNullOrEmpty` extension we saw above.

```
fun <U : Any> CharSequence?.unlessNullOrEmpty(f: (CharSequence) -> U): U? =
    if (this.isNullOrEmpty()) null else f(this)
```

- Oh, they weren't mentioned in the book, but since we used them above, I included the contents of DomainGenerators.kt with `randomUser` and `randomToDoList`:

```
import com.ubertob.fotf.zettai.domain.tooling.digits
import com.ubertob.fotf.zettai.domain.tooling.lowercase
import com.ubertob.fotf.zettai.domain.tooling.randomString
import kotlin.random.Random.Default.nextInt

fun emptyStore(): ToDoListStore = mutableMapOf()

fun usersGenerator(): Sequence<User> = generateSequence {
    randomUser()
}

fun randomUser() = User.fromTrusted(randomString(lowercase, 3,
6).capitalize())

fun itemsGenerator(): Sequence<ToDoItem> = generateSequence {
    randomItem()
}

fun randomItem() = ToDoItem(randomString(lowercase + digits, 5, 20), null,
ToDoStatus.Todo)


fun toDoListsGenerator(): Sequence<ToDoList> = generateSequence {
    randomToDoList()
}

fun randomToDoList(): ToDoList = ToDoList(
    randomListName(),
    itemsGenerator().take(nextInt(5) + 1).toList()
)

fun randomListName(): ListName = ListName.fromTrusted(randomString(lowercase,
3, 6))
```

Chapter 5 - Using Events to Modify the State

- So our app has worked well so far with a single user having a single list. Our next goal is to add scenarios for multiple lists and creating a new one.

```
class UserListPageDDT: ZettaiDDT(allActions()) {

    val carol by NamedActor(::ToDoListOwner)
    val emma by NamedActor(::ToDoListOwner)

    @DDT
    fun `new users have no lists`() = ddtScenario {

        play(
            emma.`cannot see any list`()
        ).wip(LocalDate.of(2026, 7, 13))
    }

    @DDT
    fun `only owners can see all their list`() = ddtScenario {

        val expectedLists = generateSomeToDoLists()

        setUp {
            carol.`starts with some lists`(expectedLists)
        }.thenPlay(
            carol.`can see the lists: #listName`(expectedLists.keys)
            emma.`cannot see any list`()
        )
    }.wip(LocalDate.of(2026, 7, 13))

    private fun generateSomeToDoLists(): Map<String, List<String>> {
        return mapOf(
            "work" to listOf("meeting", "spreadsheet"),
            "home" to listOf("buy food"),
            "friends" to listOf("buy present", "book restaurant"),
        )
    }
}
```

- So we need some new actor methods as well as domain and HTTP actions.

```
data class ToDoListOwner(override val name: String): DdtActor<ZettaiActions>
() {

    fun `cannot see any list`() = step {
        val lists = allUserLists(user)
        expectThat(lists).isEmpty()
    }

    fun `can see the lists: #listName`(expectedLists: Set<String>) =
        step(expectedList) {
            val lists = allUserLists(user)
            expectThat(list)
                .map(ListName::name)
                .containsExactly(expectedLists)
        }
}
```

- So now we need Actions `starts with some lists` (remember: our setUp methods are actions) and `allUserLists`.

```
interface ZettaiActions : DdtActions<DdtProtocol> {

    fun ToDoListOwner.`starts with some lists`(
            lists: Map<String, List<String>>) =
        lists.forEach { (listName, items) ->
            `starts with a list`(listName, items)
        }

    fun allUserLists(user: User): List<ListName> = TODO()

    // rest of the methods...
}
```

- So to implement the HTTPActions, we need a new route in our HttpHandler

```
class Zettai(val hub: ZettaiHub) : HttpHandler {

    val httpHandler = routes(
        "/todo/{user}/{listname}" bind GET to ::getToDoList,
        "/todo/{user}/{listname}" bind POST to ::addNewItem,
        "/todo/{user}" bind GET to ::getAllLists
    )

    private fun getAllLists(req: Request): Response {
        val user = req.extractUser()

        return hub.getLists(user)
            ?.let { renderListsPage(user, it) }
            ?.let(::toResponse)
            ?: Response(Status.BAD_REQUEST)
    }

    fun Request.extractUser(): User = path("user").orEmpty().let(::User)

    // rest of the methods...
}
```

- So now we need a method on the hub to `getLists`. Note in the above code we aren't yet verifying the users do exist. Let's add a `allUserLists` to HTTPActions and `getLists` to our hub.

```
// HTTPActions.kt
override fun allUserLists(user: User): List<ListName> {
    val response = callZettai(Method.GET, allUserListsUrl(user))
    expectThat(response.status).isEqualTo(Status.OK)
    TODO("parsing not implemented yet")
}

private fun allUserListsUrl(user: User) =
    "/todo/${user.name}"

// ToDoListHub.kt
interface ZettaiHub {
    fun getList(user: User, listName: ListName): ToDoList?
    fun addItemToList(user: User
                      listName: ListName, item: ToDoItem): ToDoList?
    fun getLists(user: User): List<ListName>?
}

class ToDoListHub(val fetcher: ToDoListUpdatableFetcher): ZettaiHub {

    override fun getLists(user: User): List<ListName>? =
        fetcher.getAll(user)

    // other methods...
}
```

- Notice in the above we are declaring the `ToDoList` nullable so we can distinguish between a user without any to-do list (empty list) and a user that doesn't exist or for which there was an error (null).
- Remember to add `allUserLists` to our `ZettaiActions` interface also.

```kotlin
// ZettaiActions.kt
interface ZettaiActions : DdtActions<DdtProtocol> {

    fun allUserLists(user: User): List<ListName>
}

// And add to DomainOnlyActions.kt
override fun allUserLists(user: User): List<ListName> =
        hub.getLists(user) ?: emptyList()

// And to HttpActions.kt
override fun allUserLists(user: User): List<ListName> {
    val response = callZettai(Method.GET, allUserListsUrl(user))
    expectThat(response.status).isEqualTo(Status.OK)
    val html = HtmlPage(response.bodyString())
    val names = extractListNameFromPage(html)
    return names.map { name -> ListName.fromTrusted(name) }
}
```

- At this point, our new DDT's pass! So we're going to move on to "Storing the State Changes". The next story is letting people make a new list, which we could easily continue to with our MutableMap, but we've been promised a different approach, so we're going to do Event Sourcing.

- The main idea behind event sourcing is that instead of persisting the new state at every change, we persist only the change while specifying the business reason - the business event. This gives us an audit flow on who did what at every step so we can reconstruct the history up to the current state. We can also calculate any past state. It is also possible to collect statistics that are hard to track otherwise, such as what has the most changes. Here are the benefits we gain:

    1. The ability to maintain the information about the business event behind the change.
    2. The ability to collect all the audit data.
    3. The ability to reconstruct the state at a certain time in the past.
    4. The ability to easily create multiple different views of the same data.
- The downsides of Event Sourcing are its complexity and learning curve. Its complexity can also be a drawback where performance is critical, and it doesn't bring much benefit in domains where there is little business logic or we don't care much about the business logic.

- The goal in functional programming is to put solid boundaries around the mutable and effects part of our code, so we can easily re-use and compose the majority of our functions.
- Here is an operation with a mutable variable:

```
fun readStream(stream: Stream): Int {

    var totByteRead = 0
    while (true) {
        val bytesRead = stream.read()
        totByteRead += bytesRead
        if (bytesRead == 0)
            break
    }
}
```

- We could clearly track this operation through a series of immutable values: say `bytesRead1` through `bytesRead10`, but we only care about the final `bytesRead10` value, and we're just lucky if it happens to take exactly 10 reads. But this is the general problem an accumulator solves.

```
fun reduceOperations(accumulator: Int,
                     operation: (Int) -> Int): Int = TODO()

// `it` in the lambda is our accumulator that we're combining with the
// read results.
val bytesRead = reduceOperations(0) { it + stream.read() }

// So we could implement `reduceOperations` through recursion.
tailrec fun reduceOperations(
    accumulator: Int,
    operation: (Int) -> Int
): Int {
    // Here we are just calling `operation` on `accumulator` until it stops
    // changing. The combining is left up to our lambda.
    val newAccumulator = operation(accumulator)
    return if (newAccumulator == accumulator) {
        accumulator
    } else {
        reduceOperations(newAccumulator, operation)
    }
}
```

- Now consider this function that concatenates a series of webpages into a single string

```
fun concatenateHtml(urls: Iterable<URI>): String {
    var html = ""
    for (url in urls) {
        html += fetchHtml(url)
    }
    return html
}

// Now with recursion.
tailrec fun Iterable<URI>.concatenateHtml(
        initial: String,
        operation: (acc: String, URI) -> String): String {
    val head = firstOrNull()

    return if (head == null)
        initial
    else {
        val html = operation(initial, head)
        drop(1).concatenateHtml(html, operation)
    }
}

// Now make it generic.
tailrec fun <T, R> Iterable<T>.composeOver(initial: R,
                  operation: (acc: R, element:T) -> R): R{

    val head = firstOrNull()

    return if (head == null)
        initial // if the list is empty
    else {
        drop(1).composeOver(operation(initial, head), operation)
    }
}

// In fact, this generic already exists in the standard library
// as fold. This is its signature. It's actually implemented without
// recursion for performance reasons.
inline fun <T, R> Iterable<T>.fold(
    initial: R,
    operation: (acc: R, T) -> R
): R
```

- The `fold` function is the most generic abstraction over the imperative programming loop. There is also a `foldRight` in the standard library with the same signature. `fold` is more powerful than a simple loop.
- Imagine a door as a finite state machine. It has the following states:
  - A door can be open or closed (unlocked) or locked.
  - It can be opened only when it's closed (unlocked).
  - It can be locked only when it's closed (unlocked).
  - It can be unlocked only when it's locked.
  - When open, it can be swung at different angles, but it stays open.
  - When locked, the key can be turned again but it stays locked.

```
sealed class Door {
    data class Open(val angle: Double) : Door() {
        fun close() = Closed
        fun swing(delta: Double) = Open(angle+delta)
    }
    object Closed : Door() {
        fun open(degrees: Double) = Open(degrees)
        fun lock() = Locked(1)
    }
    data class Locked(val turns: Int) : Door() {
        fun unlock() = Closed
        fun turnKey(delta: Int) = Locked(turns + delta)
    }
}

val door = Door.Closed.lock().turnKey(3).unlock().open(12.4).swing(34.5)
               .swing(34.5).close()
```

- Note above we use an object for Closed since there is no information to save.
- The last chain of actions was correct and checked by the compiler. We could not have closed the door twice or turned the key on an open door.
- Imagine if we defined these state transitions as separate functions. An advantage is we can combine multiple functions easily since they take and return the same type.

```
typealias DoorEvent = (Door) -> Door

val unlockDoor: DoorEvent = { aDoor: Door ->
    when (aDoor) {
        is Door.Locked -> aDoor.unlock()
        else -> aDoor // if the state is invalid, ignore it
    }
}
```

- All `DoorEvent`s share the same signature, `(Door) -> Door`, so they can be composed with `andThen` into one bigger `DoorEvent`, and there's a do-nothing `DoorEvent` — the identity function, `{ door -> door }` — that changes nothing when composed in, exactly like the `else` branch above that just hands back `aDoor` unchanged. A set of things with a combining operation (`andThen`) and an identity element that operation leaves unchanged is a *monoid*. That's exactly the shape `fold` needs: seed the accumulator with the identity `DoorEvent`, then compose each event from a list onto it in turn, and you end up with a single `DoorEvent` representing the whole sequence, ready to apply to a `Door` once at the end.
- Today's exercise that uses `fold` is a lot more straightforward, but a chance to see another sealed class. It's good preparation for folding up a list of events.

```
data class Elevator(val floor: Int)

sealed class Direction

object Up : Direction()

object Down : Direction()

@Test
fun `fold elevator events`() {

    val values = listOf(Up, Up, Down, Up, Down, Down, Up, Up, Up, Down)

    val tot = values.fold(Elevator(0)) { elevator, direction ->
        when (direction) {
            is Up -> Elevator(elevator.floor + 1)
            is Down -> Elevator(elevator.floor - 1)
        }
    }

    expectThat(tot).isEqualTo(Elevator(2))
}
```

- There's also a `Monoid` class in the exercises.

```
data class Monoid<T : Any>(val zero: T, val combination: (T, T) -> T) {
   fun List<T>.fold(): T = fold(zero, combination)
}

val zeroMoney = Money(0.0)

data class Money(val amount: Double) {
   fun sum(other: Money) = Money(this.amount + other.amount)
}

with(Monoid(0, Int::plus)) {
    expectThat(listOf(1, 2, 3, 4, 10).fold())
         .isEqualTo(20)
}

with(Monoid("", String::plus)) {
    expectThat(listOf("My", "Fair", "Lady").fold())
         .isEqualTo("MyFairLady")
}

with(Monoid(zeroMoney, Money::sum)) {
     expectThat(
         listOf(
             Money(2.1),
             Money(3.9),
             Money(4.0)
         ).fold()
    )
    .isEqualTo(Money(10.0))
}
```

Chapter 6 - Executing Commands to Generate Events

- We will be testing by sending the HTML form directly rather than using system resources to drive a headless browser.
- Our first task is a new DDT for `can create a new list called #listname`.
- We're going to use a command-oriented interface for our internal API. This gives us more control of what will change our state and allows us to "easily add common behavior to commands by decorating the command executor. This is very handy for handling transactions, logging, and the like." (Martin Fowler) Instead of more methods on the hub,

we'll define command data classes with all information inside and pass them to the command executor. We start with two commands:

```
sealed class ToDoListCommand

data class CreateToDoList(val user: User, val name: ListName)
        : ToDoListCommand() {

    // The ToDoListId is randomly minted.
    val id: ToDoListId = ToDoListId.mint()
}

data class AddToDoItem(val user: User,
                       val name: ListName, val item: ToDoItem)
        : ToDoListCommand()
```

- Instead of modifying the state directly, a command will generate one or more events. It may generate zero events if the system is already in the correct state.
- Commands are messages from the outside that can fail and are usually expressed in imperative form: CloseDoor. Events can't fail (it's already happened) and they are usually expressed in past form: DoorClosed.
- To model something as an entity in our domain, we need to make sure that we can safely update a list without affecting other entities outside the scope of a user transaction. We can do that with a list, but not with a ToDoItem, because updating or removing an item requires a change in the list. Items are contained in a ToDoList, but lists aren't contained inside a user, they simply have a reference to the UserId. A set of related entities that can be changed together is called an aggregate. In this book, an aggregate (structured data that will change together inside an atomic transaction) will be referred to as a transactional entity or just entity. Our entities are immutable. We need a basic `ToDoListEvent` that holds an `EntityId` of the entity it relates to. Before creating events, we had a brainstorming session where we came up with possible states and transitions.

```
typealias ToDoListId = EntityId

data class EntityId(val raw: UUID) {
    companion object {
        fun mint() = EntityId(UUID.randomUUID())
    }
}

interface EntityEvent {
    val id: EntityId // this is the entity's id that the event relates to
}

sealed class ToDoListEvent: EntityEvent

data class ListCreated(val id: ToDoListId, val owner: User,
        val name: ListName): ToDoListEvent()

data class ItemAdded(val id: ToDoListId,
        val item: ToDoItem): ToDoListEvent()

data class ItemRemoved(val id: ToDoListId,
        val item: ToDoItem): ToDoListEvent()

data class ItemModified(val id: ToDoListId,
        val prevItem: ToDoItem, val item: ToDoItem): ToDoListEvent()

data class ListPutOnHold(val id: ToDoListId,
        val reason: String): ToDoListEvent()

data class ListReleased(val id: ToDoListId): ToDoListEvent()

data class ListClosed(val id: ToDoListId,
        val closedOn: Instant): ToDoListEvent()
```

- Next we need all the possible states of our entity. It must have a `combine` method to combine events. Having such a method is a strong indication that there is a monoid hidden somewhere in our events.

```
interface EntityState<in E : EntityEvent> {
    fun combine(event: E): EntityState<E>
}

sealed class ToDoListState : EntityState<ToDoListEvent> {
    abstract override fun combine(event: ToDoListEvent): ToDoListState
}

object InitialState: ToDoListState() {
    override fun combine(event: ToDoListEvent) = this // for the moment
}

fun Iterable<ToDoListEvent>.fold(): ToDoListState =
    fold(InitialState as ToDoListState) { acc, e -> acc.combine(e) }
```

- To define all the states, we can start from the unit tests.

```
internal class ToDoListEventTest {

    val id = ToDoListId.mint()
    val name = randomListName()
    val user = randomUser()
    val item1 = randomItem()
    val item2 = randomItem()
    val item3 = randomItem()

    @Test
    fun `the first event create a list`() {

        val events = listOf(
            ListCreated(id, user, name)
        )

        val list = events.fold()

        expectThat(list).isEqualTo(
            ActiveToDoList(id, user, name, emptyList())
        )
    }

    @Test
    fun `adding and removing items to active list`() {

        val events: List<ToDoListEvent> = listOf(
            ListCreated(id, user, name),
            ItemAdded(id, item1),
            ItemAdded(id, item2),
            ItemAdded(id, item3),
            ItemRemoved(id, item2)
        )

        val list = events.fold()

        expectThat(list)
            .isEqualTo(ActiveToDoList(id, user, name, listOf(item1, item3)))
    }

    @Test
    fun `putting the list on hold`() {
        val reason = "not urgent anymore"
        val events: List<ToDoListEvent> = listOf(
            ListCreated(id, user, name),
```

```
            ItemAdded(id, item1),
            ItemAdded(id, item2),
            ItemAdded(id, item3),
            ListPutOnHold(id, reason)
        )

        val list = events.fold()

        expectThat(list).isEqualTo(
            OnHoldToDoList(id, user, name,
                listOf(item1, item2, item3), reason))
    }

    // For brevity we are showing three tests, but it is good to make
    // sure all the events and all possible states are covered.
}
```

- For the above tests to pass, we need all the `ToDoListState` subtypes and the function to switch from one state to another. Note that all `ToDoListState` values except `InitialState` have an internal constructor so we're forced to start from an `InitialState` and combine events to reach any other states.

```
object InitialState: ToDoListState() {
    override fun combine(event: ToDoListEvent): ToDoListState =
        when(event) {
            is ListCreated -> create(event.id, event.owner, event.name,
emptyList())
            else -> this // ignore other events
        }
}

data class ActiveToDoList internal constructor(
    val id: ToDoListId,
    val owner: User,
    val name: ListName,
    val items: List<ToDoItem>
) : ToDoListState() {
    override fun combine(event: ToDoListEvent): ToDoListState =
        when (event) {
            is ItemAdded -> copy(items = items + event.item)
            is ItemRemoved -> copy(items = items - event.item)
            is ItemModified -> copy(items = items - event.prevItem +
event.item)
            is ListPutOnHold -> onHold(event.reason)
            is ListClosed -> close(event.closedOn)
            else -> this //ignore other events
        }
}

data class OnHoldToDoList internal constructor(
    val id: ToDoListId,
    val owner: User,
    val name: ListName,
    val items: List<ToDoItem>,
    val reason: String
) : ToDoListState() {
    override fun combine(event: ToDoListEvent): ToDoListState =
        when (event) {
            is ListReleased -> release()
            else -> this //ignore other events
        }
}


data class ClosedToDoList internal constructor(val id: ToDoListId, val
closedOn: Instant) : ToDoListState() {
    override fun combine(event: ToDoListEvent): ToDoListState = this //ignore
```

```
other events

}

// These could be class methods but the author prefers extension functions
// defined in the same file so they can access internal constructors.
fun InitialState.create(id: ToDoListId, owner: User, name: ListName,
                        items: List<ToDoItem>) =
    ActiveToDoList(id, owner, name, items)

fun ActiveToDoList.onHold(reason: String) =
    OnHoldToDoList(id, owner, name, items, reason)

fun OnHoldToDoList.release() =
    ActiveToDoList(id, owner, name, items)

fun ActiveToDoList.close(closedOn: Instant) =
    ClosedToDoList(id, closedOn)
```

- So a Command is passed to the CommandHandler and produces zero or more events (usually one). First let's look at our signatures.

```
typealias CommandHandler<CMD, EVENT> = (CMD) -> List<EVENT>?

class ToDoListCommandHandler:
        CommandHandler<ToDoListCommand, ToDoListEvent> {
    override fun invoke(command: ToDoListCommand) = TODO()
}

// We don't need the entire `EventStore` but just a way to retrieve Lists
typealias ToDoListRetriever =
        (user: User, listName: ListName) -> ToDoListState?
```

- Then let's define a test.

```
internal class ToDoListCommandsTest {

    // We're mocking the retriever to just return InitialState
    val fakeRetriever: ToDoListRetriever = { _: User, _: ListName ->
InitialState }

    @Test
    fun `CreateToDoList generate the correct event`() {

        val cmd = CreateToDoList(randomUser(), randomListName())

        val handler = ToDoListCommandHandler(fakeRetriever)
        val res = handler(cmd)?.single()

        expectThat(res).isEqualTo(
            ListCreated(cmd.id, cmd.user, cmd.name)
        )
    }
}
```

- So now we need to implement a `ToDoListCommandHandler`.

```
class ToDoListCommandHandler(
    val entityRetriever: ToDoListRetriever
) : (ToDoListCommand) -> List<ToDoListEvent>? {

    override fun invoke(command: ToDoListCommand): List<ToDoListEvent>? =
        when (command) {
            is CreateToDoList -> command.execute()
            else -> null // ignore for the moment
        }

    private fun CreateToDoList.execute(): List<ToDoListEvent>? =
        entityRetriever(user, name)
            ?.let { listState ->
                when (listState) {
                    InitialState -> {
                        ListCreated(id, user, name).toList()
                    }
                    else -> null // command fail
                }
            }
}
```

- Something you'll come to appreciate is the fact that the extension function `CreateToDoList.execute()` makes it easy to navigate from the command class to the handler method using the IDE. Let's write a test to make sure creating a list twice is an error (a null):

```
// I've replaced the books code with code from the repo, since
// the books code wouldn't compile since storeEvents wasn't defined anywhere.
  val streamer = ToDoListEventStreamerInMemory()
  val eventStore = ToDoListEventStore(streamer)

  val handler = ToDoListCommandHandler(eventStore)

  fun handle(cmd: ToDoListCommand): List<ToDoListEvent>? =
      handler(cmd)?.let(eventStore)

@Test
fun `Add list fails if the user has already a list with same name`() {

    val cmd = CreateToDoList(randomUser(), randomListName())
    val res = handle(cmd)?.single()

    expectThat(res).isA<ListCreated>()

    val duplicatedRes = handle(cmd)
    expectThat(duplicatedRes).isNull()
}
```

- This test looks simple, but for it to pass we need to create an Event Store. It won't work with the simple approach where we've been mocking the retriever. We are going to have a whole chapter on the Event Store, but for now, we're going to store events in memory using an atomic reference to an immutable list. We want to define our `EventStreamer` independent of how we persist things so we can easily change that later. We also need to be able to be both an `EventPersister` and a `ToDoListRetriever` but a class can only have one invoke method per signature, so we'll make retriever an interface:

```
interface ToDoListRetriever {

    fun retrieveByName(user: User, listName: ListName): ToDoListState?

}

typealias EventStreamer<E> = (EntityId) -> List<E>?

typealias EventPersister<E> = (Iterable<E>) -> List<E>

class ToDoListEventStore(
    val eventStreamer: ToDoListEventStreamer
): ToDoListRetriever, EventPersister<ToDoListEvent> {

    private fun retrieveById(id: ToDoListId): ToDoListState? =
        eventStreamer(id)
            ?.fold()

    override fun retrieveByName(user: User,
                                listName: ListName): ToDoListState? =
        eventStreamer.retrieveIdFromName(user, listName)
            ?.let(::retrieveById)
            ?: InitialState

    override fun invoke(events: Iterable<ToDoListEvent>): List<ToDoListEvent>
=
        eventStreamer.store(events)
}

// And now we add our in-memory `EventStreamer`
// We're extending a function type again, so we are bound to implement
// a proper invoke with type (EntityId) -> List<E>?
interface ToDoListEventStreamer : EventStreamer<ToDoListEvent> {
    fun retrieveIdFromName(user: User, listName: ListName): ToDoListId?
    fun store(newEvents: Iterable<ToDoListEvent>): List<ToDoListEvent>
}

class ToDoListEventStreamerInMemory : ToDoListEventStreamer {

    val events = AtomicReference<List<ToDoListEvent>>(emptyList())

    override fun retrieveIdFromName(user: User, listName: ListName):
ToDoListId? =
        events.get()
            .firstOrNull { it == ListCreated(it.id, user, listName) }
```

```
            ?.id

    override fun store(newEvents: Iterable<ToDoListEvent>):
List<ToDoListEvent> =
        newEvents.toList().also { ne -> events.updateAndGet { it + ne } }

    override fun invoke(id: ToDoListId): List<ToDoListEvent> =
        events.get()
            .filter { it.id == id }
}
```

- That's enough for our test to run. Note that we use event sources but are not event driven. They share similar terminology, but event driven is another architectural pattern where we use events to communicate among different applications or services and it uses different events. Our events are not meant to be shared outside the application. If we share Domains Events and Integration Events, it leads to tight coupling across service boundaries.
- So we already saw a `handle` method method in our tests, but we need a new `handle` method on our hub. Since `handle` requires the command handlers and the event store, we are going to pass those as constructor parameters to our hub.
- The book has a good picture of the "big picture" at this point. We receive a request and convert it to a command, we then feed the command into the hub which feeds it to the command handler. At that point, the command handler sends events to the event streamer which sends them to an entity retriever that feeds an entity into the command handler. This loosely matches what we saw in our CommandHandler code. It receives a command and calls command.execute, which for our one command used an event retriever to fetch the list and when it was in `InitialState` sent out a list containing a `ListCreated` event. Also, in the diagram the command handler returns the command which gets transformed to a response. We're actually going to return our command from the hub. Why are we returning the command? Well, because events are private to our application and we don't want them becoming part of the API of the domain, but it is useful to return some information on what has been handled. So sending back the command is a good response. Note that our Hub will return null in the case the command errored out.

```
class ToDoListHub(
    val fetcher: ToDoListFetcher,
    val commandHandler: ToDoListCommandHandler,
    val persistEvents: EventPersister<ToDoListEvent>
) : ZettaiHub {

    override fun handle(command: ToDoListCommand): ToDoListCommand? =
        commandHandler(command)
            ?.let( persistEvents )
            ?.let { command } // return the command
}
```

- With the Hub finished (it really was that small a change), we can complete the route function to create a new list:

```
class Zettai(val hub: ZettaiHub) : HttpHandler {
    //...
    fun createNewList(request: Request): Response {
        val user = request.extractUser()
        return request.extractListNameFromForm("listname")
            ?.let { CreateToDoList(user, it) }
            ?.let(hub::handle)
            ?.let { Response(Status.SEE_OTHER)
                        .header("Location", "/todo/${user.name}") }
            ?: Response(Status.BAD_REQUEST)
    //...
    }
}
```

- At this point, we've completely wired up creating a new list, but we won't see it if we run the app, because the fetcher is still using the mutable map we gave it. We will see later about how to use events to create specific views on the system, but for now, we're going to promote our fetcher to our *read model* and will update it within the hub. While we're at it, we're going add "Add an Item to a List" as a command. Obviously we'd add tests to verify the happy path and test all the failure cases.

```
class ToDoListCommandHandler(
    val entityRetriever: ToDoListRetriever,
    val readModel: ToDoListUpdateableFetcher // temporary!
) : (ToDoListCommand) -> List<ToDoListEvent>? {
    //...
    override fun invoke(command: ToDoListCommand): List<ToDoListEvent>?=
        when (command) {
            is CreateToDoList -> command.execute()
            is AddToDoItem -> command.execute()
        }

    private fun CreateToDoList.execute(): List<ToDoListEvent>? =
        entityRetriever.retrieveByName(user, name)
            ?.let { listState ->
                when (listState) {
                    InitialState -> {
                        readModel.assignListToUser(
                            user,
                            ToDoList(name, emptyList())
                        )
                        ListCreated(id, user, name).toList()
                    }
                    else -> null // command fail
                }
            }

    private fun AddToDoItem.execute(): List<ToDoListEvent>? =
        entityRetriever.retrieveByName(user, name)
            ?.let { listState ->
                when (listState) {
                    is ActiveToDoList -> {
                        if (listState.items.any { it.description ==
item.description })
                            null //cannot have 2 items with same name
                        else {
                            readModel.addItemToList(user, listState.name,
item)
                            ItemAdded(listState.id, item).toList()
                        }
                    }

                    InitialState,
                    is OnHoldToDoList,
                    is ClosedToDoList -> null //command fail
                }
```

```
            }
        // other methods
    }
```

- Note in the above `InitialState` is a singleton so doesn't need `is`.
- And we'll update `Zettai` with the new route:

```
class Zettai(val hub: ZettaiHub): HttpHandler {
    //...
    "/todo/{user}/{listname}" bind POST to ::addNewItem,
    //...
    private fun addNewItem(request: Request): Response {
        val user = request.extractUser()
        val listName = request.extractListName()
        return request.extractItem()
            ?.let { AddToDoItem(user, listName, it) }
            ?.let(hub::handle)
            ?.let { Response(Status.SEE_OTHER).header("Location",
                        "/todo/${user.name}/${listName.name}") }
            ?: Response(Status.BAD_REQUEST)
    }
}
```

- Command names matching event names (`AddToDoItem` -> `ItemAdded` and `CreateToDoList` -> `ListCreated`) is a common pattern but is not always applicable. Commands are linked to the external API and use-cases, while events reflect the strict logic of the finite state machine. As a rule of thumb, be generous creating events but stingy with commands. If you want to share events between applications, keep those separate from your internal Event Sourcing events. Trying to use the same events to both reconstruct state and share information is usually a source of many headaches. It is also possible to keep track of changes by storing state in a traditional database and all changes as events in a separate audit persistance, but this "half-hearted event sourcing" gets all the complications of Event Sourcing and normal state persistence. It makes sense as a way to improve a legacy application.
- The identity is central to the concept of an entity. Using immutable entities makes the object identity problem much simpler to solve since they represent a frozen image in time instead of an actually mutable piece of reality.

- In Event Sourcing applications, it is a good practice to have events to revert other events so users can solve errors by themselves. If this is not possible, we may want a special "reset" event that allows us to change the state of the system. We want to avoid changing the data directly.
- There are no good examples of fold in this chapter's exercises, so here is a finite state machine of an elevator that can break down.

```
sealed class ElevatorCommand {
    data class CallElevator(val floor: Int) : ElevatorCommand()
    data class GoToFloor(val floor: Int) : ElevatorCommand()
}

sealed class ElevatorState {
    data class DoorsOpenAtFloor(val floor: Int) : ElevatorState()
    data class TravelingAtFloor(val floor: Int) : ElevatorState()
}

fun handleCommand(state: ElevatorState, command: ElevatorCommand):
ElevatorState {
    return when (command) {
        is ElevatorCommand.CallElevator -> {
            when (state) {
                is DoorsOpenAtFloor -> state // if doors are open, no need to
do anything
                is TravelingAtFloor -> DoorsOpenAtFloor(command.floor) // if
traveling, open doors at the called floor
            }
        }

        is ElevatorCommand.GoToFloor -> {
            when (state) {
                is DoorsOpenAtFloor -> TravelingAtFloor(command.floor) // if
doors are open, start traveling
                is TravelingAtFloor -> state // if already traveling, no need
to do anything
            }
        }
    }
}

sealed class ElevatorEvent {
    data class ButtonPressed(val floor: Int) : ElevatorEvent()

    data class ElevatorMoved(val fromFloor: Int, val toFloor: Int) :
ElevatorEvent()
}


fun foldEvents(events: List<ElevatorEvent>): ElevatorState =
    events.fold(DoorsOpenAtFloor(0) as ElevatorState) { state, event ->
        when (event) {
            is ButtonPressed ->
```

```
                if (state != DoorsOpenAtFloor(event.floor))
                    TravelingAtFloor(event.floor)
                else
                    state

            is ElevatorMoved -> DoorsOpenAtFloor(event.toFloor)
        }
    }
}

fun handleCommandEvents(state: ElevatorState, command: ElevatorCommand):
List<ElevatorEvent> {
    return when (command) {
        is CallElevator -> {
            when (state) {
                is DoorsOpenAtFloor -> listOf(ButtonPressed(command.floor),
ElevatorMoved(state.floor, command.floor))
                is TravelingAtFloor -> listOf(ElevatorMoved(state.floor,
command.floor))
            }
        }

        is GoToFloor -> {
            when (state) {
                is DoorsOpenAtFloor -> listOf(ButtonPressed(command.floor),
ElevatorMoved(state.floor, command.floor))
                is TravelingAtFloor -> emptyList() //ignore
            }
        }
    }
}
```

- Note in the above exercise that when doors are open in one version, it won't move to another floor and open doors; it ignores ElevatorCalled. In the event version it generates a noop when you call it from a floor it is already on. It will fold over this noop harmlessly, but you might not want the spurious events in your log.

Chapter 7 - Handling Errors Functionally

- In this chapter, we will learn what functors can do for our error handling. We'll also start returning nice error messages to users and keep error details for our logs. Returning `null` works well when we don't care much about why an error occurred, but often there is more than one thing that can go wrong and they should be handled in different ways. For

example, if a user didn't exist then the end user might want to double check the name they put in, but if the user details call failed because the web API is down then they might want to try again later. There are generally three cases where `null` isn't good enough:

1. We need to distinguish between different failures with different consequences.
2. We need to record details about which step failed in a complex calculation.
3. We can't use `null` as an error indication, because `null` is a valid result of the calculation. (Ex: due date of tasks is optional)

- We could be like GoLang and return pairs of (string, result) where string is empty when there's no error, but this leads to verbose and error prone code plus we still need to check for nullability before using the result. It also forces us to represent all errors as strings when we might want to include a faulty `Request`.

- The problem with exceptions is it removes the totality of our functions. We are declaring a function that returns say an `Email` but our code might give you an email or might throw an exception, failing its contract. We only use exceptions when there is no possible recovery.

- Put simply, a *category* is a collection of dots (objects) and arrows between them (morphisms) such that we can always combine two arrows, the order in which we combine arrows isn't important, and there is an identity arrow from each dot to itself. The concept is very abstract and can work with anything that involves relations that can be combined. A *functor* is something that maps two categories C and D together. It preserves the relationships such that if a morphism f in C connects object a to object b, the image of f in D will connect the image of a to the image of b. There's more of this theory in Appendix 3. A valid functor can take different objects in the original category and "squash" them together, and it can completely ignore parts that aren't relevant, but all the objects in the source category need to be mapped to the destination category in a way that preserves their properties.

- A useful way to consider a category is programming is for each dot to represent our types and the arrows representing pure functions. Consider just three types: `Int`, `String`, and `Double` with an identity function and another function that combines two other functions. Let me point out something first: the goal was never to treat `Int` as one category and `String` as another and hunt for morphisms between them. Really, the category is "all types" — it covers `String`, `Int`, `Double`, everything — with pure functions as the morphisms between them. `String::length`, of type `(String) -> Int`, is one such morphism. What a functor gives us is `map`: a way to take that morphism and produce a corresponding morphism between `List<String>` and `List<Int>`, operating on the containers instead of the bare values.

```
fun <T> identity(a: T): T = a

infix fun <A, B, C> ((A)->B).andThen(f: (B)->C): (A)->C =
    { a: A -> f(this(a)) }

// Rule 1: We can always combine two arrows.
val anyString = randomString
expectThat(identity(anyString)).isEqualTo(anyString)
// Rule 2: the order in which we combine arrows isn't important.
val l = anyString.length
val h = half(l) // divide by 2
val halfLength = String::length andThen ::half
expectThat(halfLength(anyString)).isEqualTo(h)
// Rule 3: there is always an identity arrow from a dot to itself
val halfLengthStr1 = (String::length andThen ::half) andThen ::toString
val halfLengthStr2 = String::length andThen (::half andThen ::toString)
expectThat(halfLengthStr1(anyString)).isEqualTo(halfLengthStr2(anyString))
```

- Kotlin's type system kind of forms a category in itself if we ignore exceptions, impure functions, mutable singletons, etc.
- So functors in code... We need a way to transform a set of types into other types while conserving their relations. If we have a category with types `String` and `Int` plus a function `(String) -> Int` then our functor must create new types based on `Int` and `String` and allow us to use our function on the new types. We can do that with generics, consider

```
data class Holder<T>(private val value: T)
```

  It is helpful to think of `Holder<T>` not as a generic type but as a *type builder*. Basically, `Holder<Int>` and `Holder<String>` are both types, but `Holder<T>` is a type builder. Generics are type builders and we can use type builders to implement functors in our code.

- But our function only works with `(String)->Int`. So there are two ways to deal with that:
  1. We apply the function to transform our functor into a `Holder<Int>`.
  2. We use the functor to *lift* our function so that it can operate directly on functors.

```
// Method 1: Define a transform method that returns a new `Holder` with
// the value inside.
data class Holder<T>(private val value: T) {
    fun <U> transform(f: (T) -> U): Holder<U> = Holder( f(value) )
}
// Method 2: higher-order function to "lift" our function to work
// with `Holder` type.
data class Holder<T>(private val value: T) {

    companion object {
        fun <T, U> lift(f: (T) -> U): (Holder<T>) -> Holder<U> =
            { c: Holder<T> -> c.transform(f) }
    }
}

// Let's make this concrete. We have a `Holder` with a `String` and want
// to apply `String::length` to return an `Int`:
val a: Holder<String> = Holder("this is a string")
val b1: Holder<Int> = a.transform(String::length)
// using `lift` instead, we create `strLenLifted` of type
// `(Holder<String>)->Holder<Int>` which we can use directly.
val strLenLifted = Holder.lift(String::length)
val b2: Holder<Int> = strLenLifted(a)
expectThat(b1 == b2)
```

- Note that `transform` is usually more convenient than `lift` unless we're working with multiple functors.
- Our `transform` method is `fmap` in Haskell but unfortunately defined as `map` in most JVM functional libraries. It is unclear using the same function `map` for collections and functors, since collections can be considered functors but that isn't their primary attribute. It's especially unclear when we operate on collections of functors or functors of collections.
- The Laws of Functors
    1. Functors must preserve identity morphisms.
    2. Functors must preserve composition of morphisms.

```
// Law 1: Functors must preserve identity morphisms.
fun <T> identity(a: T): T = a
val a: Holder<String> = Holder("this is a string")
val a1 = a.transform(::identity)
expectThat(a==a1)
// Law 2 says concatenating two transformations gives us the same result
// as combining the two functions and applying the result
infix fun <A, B, C> ((A)->B).andThen(f: (B)->C): (A)->C =
    { a: A -> f(this(a)) }
val splitIntoWords: (String) -> List<String> = { s:String -> s.split(' ') }
val c1 = a.transform(splitIntoWords).transform(List<String>::size)
val NumOfWords: (String) -> Int = splitIntoWords andThen List<String>::size
val c2 = a.transform(NumOfWords)
expectThat(c1 == c2)
```

- We only deal with endofunctors here — functors that map a category back onto itself, rather than into some different category. Since our category is (roughly) Kotlin's own types and functions, our functors take Kotlin types to other Kotlin types — they don't leave the language. Basically the category is Kotlin types and the morphisms are functions `(A)->B`.
- `Holder` seems worthless, but collections types are well-known functors. `List` is the simplest nontrivial functor. We can always create a `List` from a value, so a function of type `(T) -> List<T>` is our functor constructor. It is also possible to create lists of functions, such as

```
val f: List< (String)->Int > = listOf(String::length)

// The equivalent of `transform` is `List::map`
fun<T, R> liftList(f: (T) -> R): (List<T>) -> List<R> =
    { c: List<T> -> c.map(f) }
```

- With `liftList` we can transofrm a function working on elements to a function working on lists.

```
val strFirstLifted = liftList(String::first)
val words: List<String> = listOf(
    "Cuddly", "Acrobatic", "Tenacious", "Softly-purring")

val initials: List<Char> = strFirstLifted(words)
println(initials) // ['C', 'A', 'T', 'S']
```

- If you feel a little lost by all that theory, remember that in practice, in this book, a functor is just: a generic wrapper type, plus a way to apply a function to what's inside without unwrapping it first. `Holder<T>` is the example. You have a `Holder<String>`. You have a function `String -> Int` (like `String::length`). You can't directly call that function on a `Holder<String>` — the types don't match. A functor gives you `transform` (a.k.a. `map`, `fmap`), which says: "let me apply your `String -> Int` function to the thing wrapped inside, and hand you back a `Holder<Int>`, still wrapped." You never had to manually unwrap, apply, rewrap. That's the entire mechanical trick. The two "laws" are just sanity checks that this "apply without unwrapping" trick doesn't secretly do something weird:
    - Law 1 (identity): if you transform with a do-nothing function, you get the same value back (still wrapped) — transform can't sneakily alter things on its own.
    - Law 2 (composition): transforming twice in a row gives the same result as combining the two functions first and transforming once — transform can't reorder or interfere with your logic.
- We're going to use a union type for the outcome of our calculations, combining success or failure.

```
sealed class Outcome<T>

data class Success<T>(val value: T): Outcome<T>()
data class Failure(val errorMessage: String) : Outcome<Nothing>()

fun <U> transform(f: (T) -> U): Outcome<U> =
    when (this) {
        is Success -> Success(f(value))
        is Failure -> this
    }

fun String.asFailure(): Outcome<Nothing> = Failure(this)
fun <T> T.asSuccess(): Outcome<T> = Success(this)
```

- How do we use it?

```
fun readTextFile(filename: String): Outcome<String> =
    if (fileExists && canRead)
        readfile(File(filename)).asSuccess()
    else
        "$filename not found".asFailure()

fun getPersonDetails(nickname: String): Outcome<Person> {

    val resp = remoteRead(nickname)

    return if (resp.status == OK)
        Person.parse(resp.bodyString()).asSuccess()
    else if (resp.status == NOT_FOUND)
        "$nickname not found!".asFailure()
    else
        resp.bodyString().asFailure()
}
```

- But we have a problem where if we can end up with nested outcomes for multiple failures, so we need an `exitBlock` to return early.

```
inline fun <E: OutcomeError, T> Outcome<E, T>.onFailure(
                                    exitBlock: (E) -> Nothing): T =

    when (this) {
        is Success<T> -> value
        is Failure<E> -> exitBlock(this)
    }

fun prepareGreetingsEmail(userName: String): Outcome<Email> {

    val user = getPersonDetails(userName)
        .onFailure { return it } // no local return

    return readTextFile("myTemplate.txt")
        .transform { templ -> replace(templ, user) }
        .transform { text -> Email(user.emailAddress, text) }
}
```

- But our `Failure` is too generic. So we can improve `Outcome`:

```
interface OutcomeError {
    val msg: String
}

sealed class Outcome<out E : OutcomeError, out T> {

    fun <U> transform(f: (T) -> U): Outcome<E, U> =
        when (this) {
            is Success -> f(value).asSuccess()
            is Failure -> this
        }
}

data class Success<T> internal constructor(val value: T):
    Outcome<Nothing, T>()
data class Failure<E : OutcomeError> internal constructor(val error: E):
    Outcome<E, Nothing>()

fun <T, E : OutcomeError> T.asFailure(): Outcome<E, Nothing> = Failure(this)
fun <T> T.asSuccess(): Outcome<Nothing, T> = Success(this)
fun <T:Any, E:OutcomeError> T?.failIfNull(error: E): Outcome<E, T>
    = this?.asSuccess() ?: error.asFailure()

fun <T,U,E:OutcomeError> lift(f: (T)->U): (Outcome<E,T>)->Outcome<E,U> =
    { o -> o.transform { f(it) } }

fun <T,E:OutcomeError> Outcome<E,T>.recover(recoverError: (E)->T): T =
    when (this) {
        is Success -> value
        is Failure -> recoverError(error)
    }
```

- We could use recover with `readTextFile` to provide a fallback template:

```
val template = readTextFile("MyTemplate.txt")
    .recover { "hello {user_name}!" }

fun generatePage(request: Request): Response =
    request.parseJsonRequest()
        .transform { it.toHtmlPage() }
        .transform { Response(Status.OK).body(it) }
        .recover { Response(Status.BAD_REQUEST).body(it.msg) }

// `failOrNull` is also easy to use
private fun Request.extractListName(): ZettaiOutcome<ListName> =
    path("listname")
        .orEmpty()
        .let(ListName.Companion::fromUntrusted)
        .failIfNull(InvalidRequestError("Invalid list name in path: $this"))
```

- Here is how to use `Outcome` to clean up some of our nullable types:

```
sealed class ZettaiError: OutcomeError
data class InvalidRequestError(override val msg: String): ZettaiError()
data class ToDoListCommandError(override val msg: String): ZettaiError()
//...
typealias ZettaiOutcome<T> = Outcome<ZettaiError, T>

// eliminate nullable in the type signatures
interface ZettaiHub {
    fun getList(user: User, listName: ListName): ZettaiOutcome<ToDoList>
    fun getLists(user: User): ZettaiOutcome<List<ListName>>
    fun handle(command: ToDoListCommand): ZettaiOutcome<ToDoListCommand>
}

class ToDoListHub(...) : ZettaiHub {

    override fun handle(command: ToDoListCommand) =
        commandHandler(command)
            .transform(persistEvents)
            .transform{ command }
}

// So our ToDoListCommandHandler needs to return an `Outcome`.
typealias ToDoListCommandOutcome = ZettaiOutcome<List<ToDoListEvent>>

class ToDoListCommandHandler(...): (ToDoListCommand) ->
ToDoListCommandOutcome {
//...
    override fun invoke(command: ToDoListCommand): ToDoListCommandOutcome =
        when (command) {
            is CreateToDoList -> command.execute()
//...
    private fun CreateToDoList.execute(): ToDoListCommandOutcome {

        val listState = entityRetriever.retrieveByName(user, name) ?:
InitialState

        return when (listState) {
            InitialState ->
                ListCreated(ToDoListId.mint(), user, name).asCommandSuccess()

            is ActiveToDoList,
            is OnHoldToDoList,
            is ClosedToDoList -> InconsistentStateError(this,
listState).asFailure()
        }
```

```
    }
}

data class InconsistentStateError(
                val command: ToDoListCommand,
                val state: ToDoListState): ZettaiError() {
    override val msg = "Command $command cannot be applied to state $state"
}

fun ToDoListEvent.asCommandSuccess(): ZettaiOutcome<List<ToDoListEvent>> =
    listOf(this).asSuccess()

// Now we can update some of our Hub calls in Routes.kt
private fun addNewItem(request: Request): Response {
    val user = request.extractUser()
        .recover { User("anonymous") }
    val listName = request.extractListName()
        .onFailure { return Response(Status.BAD_REQUEST).body(it.msg) }
    val item = request.extractItem()
        .onFailure { return Response(Status.BAD_REQUEST).body(it.msg) }

    return hub.handle(AddToDoItem(user, listName, item))
        .transform { Response(Status.SEE_OTHER)
            .header("Location", "/todo/${user.name}/${listName.name}") }
        .recover { Response(Status.UNPROCESSABLE_ENTITY).body(it.msg) }
}

// And we can test with `Outcome` - let's define some helper assertions
fun <E : OutcomeError, T> Outcome<E, T>.expectSuccess(): T =
    onFailure { error -> fail { "$this expected success but was $error" } }

fun <E : OutcomeError, T> Outcome<E, T>.expectFailure(): E =
    onFailure { error -> return error }
        .let { fail { "Expected failure but was $it" } }

@Test
fun `Add list fails if the user has already a list with same name`() {

    val cmd = CreateToDoList(user, name)
    val res = handle(cmd).expectSuccess().single()

    expectThat(res).isA<ListCreated>()

    val duplicatedRes = handler(cmd).expectFailure()
```

```
    expectThat(duplicatedRes).isA<InconsistentStateError>()
}
```

Chapter 8 - Using Functors to Project Events

- We run into the problem that events don't contain the whole information of an entity but only what changed. That's why we need projections. Instead of querying the events, we will *project* the events in a stateful data structure that we can query like it was a database table. Projections can be stored in a persistence layer or recreated in memory every time the application starts. This chapter we will work with them in memory.
- A projection is like a database table; it's a collection of records, known as rows, that can be queried. One nice advantage is we can tailor the row with precisely the data that we require, denormalized or aggregated, without worrying about update issues since they are read-only. Projections depend on two types:
    1. The type of events that generate the projection.
    2. The type of stored state; in simpler terms, the projection row.

```
interface Projection<ROW: Any, EVENT: EntityEvent> {

    // This will fetch new events since last update.
    val eventFetcher: FetchStoredEvents<EVENT>

    // This function returns the last event that has been projected.
    fun lastProjectedEvent(): EventSeq

    // This method fetches new events and uses them to update the
    // projection.
    fun update() {
         eventFetcher(lastProjectedEvent())
            .forEach{ TODO("project events here") }
    }
}
```

- To keep track of which events have been processed, we need a sequential progressive number associated to each event. To store this progressive number, we create the `EventSeq` type and the new generic type `StoredEvent<E>` to keep the event with its progressive number.

```
data class EventSeq(val progressive: Int) {
    operator fun compareTo(other: EventSeq): Int =
        progressive.compareTo(other.progressive)
}

data class StoredEvent<E: EntityEvent>(val eventSeq: EventSeq, val event: E)

typealias FetchStoredEvents<E> = (EventSeq) -> Sequence<StoredEvent<E>>
```

- The `projector` is a function that updates the projection rows by creating a `DeltaRow` from each event representing the information needed to update projection. A `DeltaRow` is similar to a SQL CREATE/UPDATE/DELETE.

```
typealias ProjectEvents<R, E> = (E) -> List<DeltaRow<R>>

data class RowId(val id: String)

sealed class DeltaRow<R: Any>

data class CreateRow<R: Any>(val rowId: RowId,
                             val row: R): DeltaRow<R>()

data class DeleteRow<R: Any>(val rowId: RowId): DeltaRow<R>()

data class UpdateRow<R: Any>(val rowId: RowId,
                             val updateRow: R.()->R): DeltaRow<R>()
```

- Note that `UpdateRow` takes a receiver-typed lambda, which allows it to call methods on its receiver without `this`, `it`, or a named parameter.

```
val updateRow: (Row) -> Row = { it.copy(name = "new name") }

val updateRow2: Row.() -> Row = { copy(name = "new name") }
```

- The `RowId` can just be the `EntityId` or can be something more complicated.
- Now we can complete our `update` method:

```
typealias FetchStoredEvents<E> = (EventSeq) -> Sequence<StoredEvent<E>>
typealias ProjectEvents<R, E> = (E) -> List<DeltaRow<R>>

interface Projection<R: Any, E: EntityEvent> {

    // The eventProjector will be injected. It is a pure function that
    // only depends on the projection logic.
    val eventProjector: ProjectEvents<R, E>

    // The eventFetcher will also be injected.
    val eventFetcher: FetchStoredEvents<E>

    fun lastProjectedEvent(): EventSeq

    fun update() {
        eventFetcher(lastProjectedEvent())
            .forEach{ storedEvent ->
                applyDelta(storedEvent.eventSeq,
                    eventProjector(storedEvent.event))
            }
    }

    fun applyDelta(eventSeq: EventSeq, deltas: List<DeltaRow<R>>)
}
```

- For this chapter, we will store our projections in a `HashMap`. Next chapter we will store them in a database.

```
interface InMemoryProjection<R: Any, E: EntityEvent> : Projection<R, E> {
    fun allRows(): Map<RowId, R>
}

data class ConcurrentMapProjection<R: Any, E: EntityEvent>(
    override val eventFetcher: FetchStoredEvents<E>,
    override val eventProjector: ProjectEvents<R, E>
) : InMemoryProjection<R, E> {

    private val rowsReference: AtomicReference<Map<RowId, R>> =
        AtomicReference(emptyMap())

    private val lastEventRef: AtomicReference<EventSeq> =
        AtomicReference(EventSeq(-1))

    override fun allRows(): Map<RowId, R> = rowsReference.get()


    override fun applyDelta(eventSeq: EventSeq, deltas: List<DeltaRow<R>>) {

        deltas.forEach { delta ->
            rowsReference.getAndUpdate { rows ->
                when (delta) {
                    is CreateRow -> rows + (delta.rowId to delta.row)
                    is DeleteRow -> rows - delta.rowId
                    is UpdateRow -> rows[delta.rowId]?.let { oldRow ->
                        rows - delta.rowId +
                        (delta.rowId to delta.updateRow(oldRow))
                    }
                }
            }.also { lastEventRef.getAndSet(eventSeq) }
        }
    }

    override fun lastProjectedEvent(): EventSeq = lastEventRef.get()
}
```

- Our goal is to have a collection of rows that can be search, filtered, and aggregated for analysis. Each row should represent a to-do list with details. If we were using a traditional approach rather than event sourcing, we would have a database table that resembles this projection. Advantages of event sourcing include being able to reconstruct the state of the system at any given point in time and being able to easily modify the projection in the future

if we need a different view or creating additional projections for specific tasks as needed. We start with our row.

```
data class ToDoListProjectionRow(val user: User,
                                 val active: Boolean,
                                 val list: ToDoList) {

    fun addItem(item: ToDoItem): ToDoListProjectionRow =
        copy(list = list.copy(items = list.items + item))

    fun removeItem(item: ToDoItem): ToDoListProjectionRow =
        copy(list = list.copy(items = list.items - item))

    fun replaceItem(prevItem: ToDoItem, item: ToDoItem):
ToDoListProjectionRow =
        copy(list = list.copy(items = list.items - prevItem + item))

    fun putOnHold(): ToDoListProjectionRow = copy(active = false)

    fun release(): ToDoListProjectionRow = copy(active = true)
}
```

- And we're ready to code our `eventProjector`.

```
fun eventProjector(e: ToDoListEvent): List<DeltaRow<ToDoListProjectionRow>> =
    when (e) {
        is ListCreated -> CreateRow(e.rowId(),
            ToDoListProjectionRow(e.owner, true, ToDoList(e.name,
emptyList())))
        is ItemAdded -> UpdateRow(e.rowId()) { addItem(e.item) }
        is ItemRemoved -> UpdateRow(e.rowId()) { removeItem(e.item) }
        is ItemModified -> UpdateRow(e.rowId()) { replaceItem(e.prevItem,
e.item) }
        is ListPutOnHold -> UpdateRow(e.rowId()) { putOnHold() }
        is ListReleased -> UpdateRow(e.rowId()) { release() }
        is ListClosed -> DeleteRow(e.rowId())
    }.toSingle()
}

private fun ToDoListEvent.rowId(): RowId = RowId(id.toString())

fun <T : Any> DeltaRow<T>.toSingle(): List<DeltaRow<T>> = listOf(this)
```

- So we need to figure out how to query our projection, and tests are the best way to guide our decision.

```
@Test
fun `findAll returns all the lists of a user`() {

    val user = randomUser()
    val listName1 = randomListName()
    val listName2 = randomListName()
    val events = listOf(
        ListCreated(ToDoListId.mint(), user, listName1),
        ListCreated(ToDoListId.mint(), user, listName2),
        ListCreated(ToDoListId.mint(), randomUser(), randomListName())
    )

    val projection = events.buildListProjection()

    expectThat(projection.findAll(user))
        .isEqualTo(listOf(listName1, listName2))
}

private fun List<ToDoListEvent>.buildListProjection(): ToDoListProjection =
    ToDoListProjection { after ->
        mapIndexed { i, e ->
            StoredEvent(EventSeq(after.progressive + i + 1), e) }
                .asSequence()
    }.also(ToDoListProjection::update)
```

- But we need a `ToDoListProjection` and `findAll` query for this to work.

```
class ToDoListProjection(eventFetcher: FetchStoredEvents<ToDoListEvent>):
    InMemoryProjection<ToDoListProjectionRow, ToDoListEvent>
        by ConcurrentMapProjection(eventFetcher, ::eventProjector) {

    fun findAll(user: User): List<ListName>? =
        allRows().values
            .filter { it.user == user }
            .map { it.list.listName }

    // here's some other queries from the code repo
    fun findList(user: User, name: ListName): ToDoList? =
        allRows().values
            .firstOrNull { it.user == user && it.list.listName == name }
            ?.list

    fun findAllActiveListId(user: User): List<EntityId> =
        allRows()
            .filter { it.value.user == user && it.value.active }
            .map { ToDoListId.fromRowId(it.key) }
}
```

- And now we can add our `ToDoListProjection` to the Hub and remove our MutableMap:

```
class ToDoListHub(...) {
    //...
    override fun getList(user: User, listName: ListName):
            ZettaiOutcome<ToDoList> =
        listProjection.findList(user, listName)
            .failIfNull(
                InvalidRequestError("List $listName of user $user not
found!"))

    override fun getLists(user: User): ZettaiOutcome<List<ListName>> =
        listProjection.findAll(user)
            .failIfNull(InvalidRequestError("User $user not found!"))
    //...
}
```

- Unfortunately it doesn't work yet since we need to update the projection before querying it. We also need a new component with the responsibility of keeping all the projections together and updating them when needed so that responsibility doesn't fall to the Hub. So we will write a `QueryRunner`. The goal is to have a function that take a `QueryRunner` and

returns a result of an arbitrary type T. To ensure the `QueryRunner` is in the proper state when the function is called, we'll create a functor named `ProjectionQuery` that wraps a function of type `(QueryRunner) -> T`. A functor gives us control over when the query is executed. We only want to run it when it's correct and at minimum the projections are up-to-date. Functors allow us to separate technical infrastructure concerns from the business requirements.

```
interface QueryRunner<Self : QueryRunner<Self>> {
    operator fun <R> invoke(f: Self.() -> R): ProjectionQuery<R>
}

class ToDoListQueryRunner(eventFetcher: FetchStoredEvents<ToDoListEvent>):
                                        QueryRunner<ToDoListQueryRunner> {
    internal val listProjection = ToDoListProjection(eventFetcher)

    override fun <R> invoke(f: ToDoListQueryRunner.() -> R) =
        ProjectionQuery(setOf(listProjection)) { f(this) }
}
```

- That `Self` reference plugs in the concrete class as the type reference so we have access to stuff contained in `ToDoListQueryRunner` like `listProjection` with it as receiver. We want our internal `listProjection` reference accessible from our DSL.
- The functor also interacts well with laziness. We are going to use Kotlin lazy sequences so the projection is only calculated once when used. This could make resources hard to plan for since we don't know when it will be accessed though. Functors allow us to control both the laziness and how we use external resources and are easy to compose together.

```
data class ProjectionQuery<T>(
        val projections: Set<Projection<*, *>>,
        val runner: () -> T) {

    fun <U> transform(f: (T) -> U): ProjectionQuery<U> =
        ProjectionQuery(projections) { f(runner()) }

    fun runIt(): T {
        projections.forEach(Projection<*, *>::update)
        return runner()
    }
}
```

- It's a good sign that we're dealing with a functor because it's a generic type with a transform method that applies a function and returns a new instance of a different generic type.
- We can use this to replace our `fetcher` and we don't even need new tests since we already have them to cover the fetching functionality.

```
class ToDoListHub(
    val queryRunner: ToDoListQueryRunner,
    val commandHandler: ToDoListCommandHandler,
    val persistEvent: EventPersister<ToDoListEvent>
) : ZettaiHub {

    // other methods...

    override fun getList(user: User, listName: ListName):
                                    ZettaiOutcome<ToDoList> =
        queryRunner {
            listProjection
            .findList(user, listName)
            .failIfNull( InvalidRequestError(
                "List $listName of user $user not found!"))
        }.runIt()

    override fun getLists(user: User): ZettaiOutcome<List<ListName>> =
        queryRunner {
            listProjection.findAll(user)
                .failIfNull(InvalidRequestError("User $user not found!"))
        }.runIt()
}
```

- When are functors useful?
    1. We have some data that's "bound" to something else - for example, a number in a list of numbers, or a result and its failure.
    2. We want to operate on the data, ignoring it context.
- Functors we've seen so far: `Holder`, `List`, and `Outcome` all look like a container if you squint hard enough, but other functors like `ProjectorQuery` can't be thought of that way.
- We haven't implemented a story in a while, so we want to display a page with the most urgen items still pending for a user. We're going to gloss over the details but focus on the new projection required, since it will include items from multiple lists that have a due date.

```
data class ItemProjectionRow(val item: ToDoItem, val listId: EntityId)

fun eventProjector(e: ToDoListEvent): List<DeltaRow<ItemProjectionRow>> =
    when (e) {
        is ListCreated -> emptyList()
        is ListPutOnHold -> emptyList()
        is ListReleased -> emptyList()
        is ListClosed -> emptyList()
        is ItemAdded -> CreateRow(
            e.itemRowId(e.item),
            ItemProjectionRow(e.item, e.id)).toSingle()
        is ItemRemoved -> DeleteRow(e.itemRowId(e.item)).toSingle()
        is ItemModified -> listOf(
            DeleteRow(e.itemRowId(e.prevItem)),
            CreateRow(e.itemRowId(e.item), ItemProjectionRow(e.item, e.id)),
        )
}

private fun ToDoListEvent.itemRowId(item: ToDoItem): RowId =
    RowId("${id}_${item.description}")

class ToDoItemProjection(eventFetcher: FetchStoredEvents<ToDoListEvent>):
    InMemoryProjection<ItemProjectionRow, ToDoListEvent> by
        ConcurrentMapProjection(
            eventFetcher,
            ::eventProjector
        ) {

    fun findWhatsNext(maxRows: Int, lists: List<EntityId>):
        List<ItemProjectionRow> =
            allRows().values
                .filter { it.listId in lists }
                .filter { it.item.dueDate != null
                          && it.item.status == ToDoStatus.Todo }
                .sortedBy { it.item.dueDate }
                .take(maxRows)
}

// Now for the Hub method
class ToDoListHub(...)
//...
    override fun whatsNext(user: User): Outcome<ZettaiError, List<ToDoItem>>
=
        queryRunner {
            listProjection.findAllActiveListId(user)
```

```
                .failIfEmpty(InvalidRequestError("User $user not found!"))
                .transform { ul -> itemProjection.findWhatsNext(10, ul) }
                .transform { it.map(ItemProjectionRow::item) }
        }.runIt()
}
```

- The basic idea behind CQRS (what we just implemented) is to keep the read data model (query) separate from the write data model (command). Event Sourcing and CQRS are often used together, but can appear apart.
- And we're back with the elevators. That was a fun state machine, so here we are making projections.

```
package com.ubertob.fotf.exercises.chapter8

import com.ubertob.fotf.exercises.chapter8.ElevatorEvent.*
import com.ubertob.fotf.exercises.chapter8.ElevatorState.*


sealed class ElevatorEvent {

    abstract val elevatorId: Int

    data class ButtonPressed(override val elevatorId: Int, val floor: Int) :
ElevatorEvent()
    data class ElevatorMoved(override val elevatorId: Int, val fromFloor:
Int, val toFloor: Int) : ElevatorEvent()
    data class ElevatorBroken(override val elevatorId: Int) : ElevatorEvent()
    data class ElevatorFixed(override val elevatorId: Int) : ElevatorEvent()
}


sealed class ElevatorState {

    abstract val floor: Int

    data class DoorsOpenAtFloor(override val floor: Int) : ElevatorState()
    data class TravelingAtFloor(override val floor: Int) : ElevatorState()
    object OutOfOrder : ElevatorState() {
        override val floor: Int = 0
    }
}

data class ElevatorProjectionRow(val elevatorId: Int, val floor: Int, val
state: ElevatorState)

fun foldEvents(events: List<ElevatorEvent>): ElevatorState =
    events.fold(DoorsOpenAtFloor(0) as ElevatorState) { state, event ->
        when (event) {
            is ButtonPressed ->
                if (state != DoorsOpenAtFloor(event.floor))
                    TravelingAtFloor(event.floor)
                else
                    state

            is ElevatorMoved -> DoorsOpenAtFloor(
                event.toFloor
            )
```

```
            is ElevatorBroken -> OutOfOrder
            is ElevatorFixed -> DoorsOpenAtFloor(0) // assume elevator goes
back to ground floor when fixed

        }
    }


interface ElevatorProjection {
    fun allRows(): List<ElevatorProjectionRow>
    fun getRow(elevatorId: Int): ElevatorProjectionRow?
}

class ElevatorProjectionInMemory(events: List<ElevatorEvent>) :
ElevatorProjection {
    //process all events in a map of elevatorstate
    val stateMap: Map<Int, ElevatorProjectionRow> = events.groupBy {
it.elevatorId }
        .mapValues {
            println(it)

            foldEvents(it.value).toProjectionRow(it.key)
        }

    override fun allRows(): List<ElevatorProjectionRow> =
        stateMap.values.toList()


    override fun getRow(elevatorId: Int): ElevatorProjectionRow? =
        stateMap[elevatorId]

}

private fun ElevatorState.toProjectionRow(elevatorId: Int) =
    ElevatorProjectionRow(elevatorId, this.floor, this)
```

Chapter 9 - Using Monads to Persist Data Safely

- So far we've used unit tests to check logic and DDT's to test a slice of functionality end-to-end. Now we will add integration tests to test our integration with an external service. We will be using PostgreSQL as our database backend and start with a spike to evaluate the technology for our use. We'll use PostgreSQL via Docker. Dockerfile is in the repo, or you

can always install PostgreSQL the old fashioned way. We're using the Exposed library for database access. It is by the same people at JetBrains responsible for Kotlin. Exposed is not "functional", but that lets it serve as a good example of how to use a non-functional library in our functional code. We will use PostgreSQL's document feature to store the entire event in a `jsonb` field. We aren't using our uuid as key, since we need to sort events in order and an increasing key makes that easy.

```
data class PgEventTable(override val tableName: String) : Table(tableName) {

    val id = long("id").autoIncrement()
    override val primaryKey = PrimaryKey(id, name = "${tableName}_pkey")

    val recorded_at = timestamp("recorded_at")
        .defaultExpression(CurrentTimestamp())
    val entity_id = uuid("entity_id")
    val event_type = varchar("event_type", 100)
    val event_version = integer("event_version")
    val event_source = varchar("event_source", 100)
    val json_data = jsonb("json_data")
}
```

- `jsonb` isn't supported by Exposed, so it took about fifty lines of code to define it as a field type in the database. All that is present in the sample code in `com.ubertob.fotf.zettai.db.jdbc.PgJson.kt`.
- So we need a `Parser` to convert events and a type with all the fields we are going to write to the database.

```
data class Parser<A, S>(
    val render: (A) -> S,
    val parse: (S) -> Outcome<OutcomeError, A>
) {

    fun parseOrThrow(encoded: S) =
        parse(encoded).onFailure { error("Error parsing $encoded ${it.msg}") }
}
}

data class PgEvent(
    val entityId: EntityId,
    val eventType: String,
    val jsonString: String,
    val version: Int,
    val source: String
)
```

- We need to convert our ToDoListEvent objects to PgEvent objects, but first we should write a test. We are just going to generate a good number of random events, then convert them to PgEvent and back, and make sure they are identical afterwards.

```
class ToDoListEventParserTest {

    val eventParser = toDoListEventParser()

    @Test
    fun `convert events to and from`() {

        eventsGenerator().take(100).forEach { event ->

            val conversion = eventParser.render(event)
            val newEvent = eventParser.parse(conversion).expectSuccess()

            expectThat(newEvent).isEqualTo(event)

        }
    }
}

fun randomEvent(): ToDoListEvent =
    when (val kClass = ToDoListEvent::class.sealedSubclasses.random()) {
        ListCreated::class -> ListCreated(
            ToDoListId.mint(), randomUser(), randomListName())
        ItemAdded::class -> ItemAdded(ToDoListId.mint(), randomItem())
        ItemRemoved::class -> ItemRemoved(ToDoListId.mint(), randomItem())
        ItemModified::class -> ItemModified(
            ToDoListId.mint(), randomItem(), randomItem())
        ListPutOnHold::class -> ListPutOnHold(
            ToDoListId.mint(), randomText(20))
        ListReleased::class -> ListReleased(ToDoListId.mint())
        ListClosed::class -> ListClosed(ToDoListId.mint(), Instant.now())
        else -> error("Unexpected class: $kClass")
    }

fun eventsGenerator(): Sequence<ToDoListEvent> = generateSequence {
    randomEvent()
}
```

- We're using Klaxon as our JSON library since it supports reflection to transform a Kotlin class into JSON. This is a temporary solution until we learn to safely handle serialization and deserialization later.

```
fun toDoListEventParser(): Parser<ToDoListEvent, PgEvent> =
    Parser(::toPgEvent, ::toToDoListEvent)

fun toPgEvent(event: ToDoListEvent): PgEvent =
    PgEvent(
        entityId = event.id,
        eventType = event::class.simpleName.orEmpty(),
        version = 1,
        source = "event store",
        jsonString = event.toJsonString()
    )

fun toToDoListEvent(pgEvent: PgEvent): ZettaiOutcome<ToDoListEvent> =
    Outcome.tryOrFail {
        when (pgEvent.eventType) {
            ListCreated::class.simpleName ->
                klaxon.parse<ListCreated>(pgEvent.jsonString)
            ItemAdded::class.simpleName ->
                klaxon.parse<ItemAdded>(pgEvent.jsonString)
            ItemRemoved::class.simpleName ->
                klaxon.parse<ItemRemoved>(pgEvent.jsonString)
            ItemModified::class.simpleName ->
                klaxon.parse<ItemModified>(pgEvent.jsonString)
            ListPutOnHold::class.simpleName ->
                klaxon.parse<ListPutOnHold>(pgEvent.jsonString)
            ListClosed::class.simpleName ->
                klaxon.parse<ListClosed>(pgEvent.jsonString)
            ListReleased::class.simpleName ->
                klaxon.parse<ListReleased>(pgEvent.jsonString)
            else -> null
        } ?: error("type not known ${pgEvent.eventType}")
    }.transformFailure { ZettaiParsingError(
        "Error parsing ToDoListEvent: ${pgEvent} with error: $it ") }
```

- We also need a table to store our Projection where we record RowId and timestamp of the last update with the `jsonb` field with the row data. All rows are the same type, so we can go directly from a Row to a JSON string. And we need another table to store the sequentially incrementing counter of the last event processed in each projection and when it was updated.

```
data class PgProjectionTable<ROW : Any>(
        override val tableName: String,
        val parser: Parser<ROW, String>): Table(tableName) {

    val id = varchar("id", 50)

    override val primaryKey = PrimaryKey(id, name = "${tableName}_pkey")

    val updated_at = timestamp("recorded_at")
        .defaultExpression(CurrentTimestamp())

    val row_data = jsonb("row_data",
        parser::parseOrThrow,
        parser::render
    )
}

data class PgLastEventTable(override val tableName: String):Table(tableName)
{

    val last_event_id = long("last_event_id")
    val updated_at = timestamp("recorded_at")
        .defaultExpression(CurrentTimestamp())
}
```

- So we just define our actual tables, along with some database helper functions (found in `com.ubertob.fotf.zettai.db.ZettaiTables`):

```
val toDoListEventsTable = PgEventTable("todo_list_events")

val toDoListProjectionTable = PgProjectionTable(
    "todo_list_projection", toDoListProjectionParser)

val toDoListLastEventTable = PgLastEventTable(
    "${toDoListProjectionTable.tableName}_last_processed_event")

fun resetDatabase(datasource: DataSource) {

    val db = Database.connect(datasource)

    transaction(db) {
        addLogger(StdOutSqlLogger)

        dropTables()
        prepareDb()
    }
}

fun prepareDatabase(datasource: DataSource) {

    val db = Database.connect(datasource)

    transaction(db) {
        addLogger(StdOutSqlLogger)

        prepareDb()
    }
}

private fun Transaction.prepareDb() {
    SchemaUtils.createMissingTablesAndColumns(
        toDoListEventsTable,
        toDoListProjectionTable,
        toDoListLastEventTable,
    )
}

private fun dropTables() {
    SchemaUtils.drop(
        toDoListEventsTable,
        toDoListProjectionTable,
        toDoListLastEventTable,
```

```
    )
}
```

- At this point, the focus on purity comes across as a little obsessive, but we'll see how it works out. The plan is that since the Exposed database library works primarily through side effects, we will hack on the library source code a bit to make it more functional. Specifically the Transaction is stored in a hidden singleton, and we want to pass it as an argument. There are a full set of tests based around this in the source repo, but I'll list the example from the book:

```
val dataSource = pgDataSourceForTest()

@Test
fun `can read and write events from db`() {

    val db = Database.connect(dataSource)

    transaction(db) {

        val listId = ToDoListId.mint()
        val event = ListCreated(listId, user, list.listName)
        val pgEvent = toPgEvent(event)

        val eventId = toDoListEventsTable.insertIntoWithReturn(this,
                stored(event)) {
            newRow ->
                newRow[entity_id] = pgEvent.entityId.raw
                newRow[event_source] = pgEvent.source
                newRow[event_type] = pgEvent.eventType
                newRow[json_data] = pgEvent.jsonString
                newRow[event_version] = pgEvent.version
            }.eventSeq

        expectThat(eventId.progressive).isGreaterThan(0)

        val row = toDoListEventsTable.selectWhere(this,
                toDoListEventsTable.id eq eventId.progressive)
                .single()


expectThat(row.get(toDoListEventsTable.entity_id)).isEqualTo(listId.raw)
    }
}
```

- So we provide some extensions functions to database library objects that take Transaction as an argument to get our tests working. The goal here is to abstract away the transaction and create a mechanism for operating in any "context" that facilitates the reading and writing of data. The code is mostly extension functions in the code repo at: `com.ubertob.fotf.zettai.db.jdbc.TransactionProvider.kt`.

- We want to move data access into operations that run inside a transaction. A higher-order function remains referentially transparent only when its evaluation does not introduce effects. Passing an impure callback such as `context::read` makes the resulting operation effectful, even if the surrounding composition is otherwise clean.

- We currently have two issues. One is passing the context explicitly each time, and we will focus on that first. The other is the fact that remote access can fail for a lot of reasons, and there is no indication of what can fail and why. For errors we will define a `ContextProvider` that correctly handles the context even in case of failures, closing file handlers or database connections, then return an Outcome with the result or error.

- First let's sketch out how all this will work:

```
// simple sketch
val context = remoteStorageContext("config")      //impure computation
val a = readA(context::read, id)                  //pure function
val b = calculateB(a)                             //pure function
writeB(context::write, b)                         //pure function

// sketchy definition - wrap any computation that needs the context with
// a `ContextReader` and let them run together at the end with the context.
fun readA(id: String): ContextReader<CTX, A> = //...
fun calculateB(a: A): B = //...
fun writeB(b: B): ContextReader<CTX, Unit> = //...
reader = readA(id)
    .transform{ a -> calculateB(a) }
    .transform{ b -> write(b) }
reader.runWith{ TODO("Here we need the context") }
```

- Note that although we're thinking about the database, we aren't committed to that context. A `ContextReader` could work with a connection to a database, a CSV file, or a structure in memory. We are using a functor to inject the context into the chain of transformations.

```
data class ContextReader<CTX, out T>(val runWith: (CTX) -> T) {

    fun <U> transform(f: (T) -> U): ContextReader<CTX, U> =
        ContextReader { t -> f(runWith(t)) }
}

interface ContextProvider<CTX> {
    fun <T> tryRun(reader: ContextReader<CTX, T>): Outcome<ContextError, T>
}
```

- If you squint hard enough, you can see this is what is in the repo at `com.ubertob.fotf.zettai.db.fp.{ContextProvider, ContextReader}`, although both implementations have gotten considerably more complicated when finished. The `ContextProvider` doesn't require much elaboration, but the `ContextReader` has grown to 50 lines. So let's move back to our domain. We want our functor to store projections. We'll start with the types. Note that we're returning a ContextReader with a Row inside and returning Unit inside a ContextReader.

```
fun readRow(id: String):
    ContextReader<Transaction, ToDoListProjectionRow> = TODO()

fun writeRow(row: ToDoListProjectionRow):
    ContextReader<Transaction, Unit> = TODO()

fun remoteStorageProvider: ContextProvider<Transaction> = TODO()

// In pseudocode, we could use this as follows:
val listUpdater = readRow("myRowId")
        .transform { r -> r.copy(active = false) }
        .transform { r -> writeRow(r) }

remoteStorageProvider.tryRun(listUpdater).expectSuccess()
```

- The code above compiles, but it has a problem. `listUpdater` is of types `ContextReader<Transaction, ContextReader<Transaction, Unit>>` when we wanted `ContextReader<Transaction, Unit>`.

- So now we reach Monads. Contrary to opinions that they are impossible to explain, a Monad is merely what is formed by a Functor that can be combined with another Functor of the same type. If you think of a Functor as a generic type `Foo<T>` that has a method like `Foo<T>.bar(f: (T) -> U): Foo<U>`, then a Monad could be simplified to a generic type

`Foo<T>` that has a method like `Foo<T>.barbar(f: (T) -> Foo<U>): Foo<U>`. There's more to it than that, but that's a good rule of thumb. We're going to call `barbar` just `bind`, but you might see it as `flatmap` or even `collect` in other languages.

- To combine two endofunctors, they must be of the same type. We can flatten a `List<List<Int>>` or an `Outcome<E, Outcome<E, Int>>` but in general we cannot do much with a `List<Outcome <E, Int>>` except maybe change it into an `<Outcome<E, List<Int>>`.
- Monad Laws
    1. Left Identity - If we apply a plain monad to a value and we bind a function that creates a new monad, the result would be the same as applying that function to our initial value.
    2. Right Identity - If we have a monadic value and we bind it to a function that returns the plain monad, the result would be the same as our initial monadic value.
    3. Associativity - If we have a chain of functions to bind, it shouldn't matter how we nest them.
- We are going to make `Outcome` into a monad by defining `bind` and `join`. `join` flattens an `Outcome` nested inside another one.

```kotlin
// Start with the tests
val genericFail = DivisionError("generic error").asFailure()
val divFail = DivisionError("You cannot divide by zero").asFailure()

private fun divide100by(x: Int): Outcome<DivisionError, Int> =
    if (x == 0)
        divFail
    else
        (100 / x).asSuccess()

@Test
fun `binding two outcome together`() {
    val valid = 5.asSuccess()
    expectThat(valid.bind(::divide100by)).isEqualTo(20.asSuccess())

    val invalid = DivisionError("generic error").asFailure()
    expectThat(genericFail.bind(::divide100by)).isEqualTo(divFail)

    val zero = 0.asSuccess()
    expectThat(zero.bind(::divide100by)).isEqualTo(divFail)
}

@Test
fun `joining two outcome together`() {
    val valid = 10.asSuccess().asSuccess()
    expectThat(valid.join()).isEqualTo(10.asSuccess())

    val invalid = genericFail.asSuccess() {
    expectThat(invalid.join()).isEqualTo(genericFail)
}

// An the implementations are quite simple:
inline fun <T, U, E : OutcomeError>
        Outcome<E, T>.bind(f: (T) -> Outcome<E, U>): Outcome<E, U> =
    when (this) {
        is Success -> f(value)
        is Failure -> this
    }

fun <T, E: OutcomeError> Outcome<E, Outcome<E, T>>.join(): Outcome<E, T> =
    bind { it }

// Let's also finish our ContextReader, This is also known as the Reader
monad.
data class ContextReader<CTX, out T>(val runWith: (CTX) -> T) {
```

```
    fun <U> transform(f: (T) -> U): ContextReader<CTX, U> =
        ContextReader { ctx -> f(runWith(ctx)) }

    fun <U> bind(f: (T) -> ContextReader<CTX, U>): ContextReader<CTX, U> =
        ContextReader { ctx -> f(runWith(ctx)).runWith(ctx) }
}
```

- We can't extract these out to interfaces since we can't express their types in Kotlin. We need type classes, which are unavailable.
- The exercises this chapter were to implement `join` without `bind` and then define `bind` in terms of `join`.

```
data class ContextReader<CTX, out T>(val runWith: (CTX) -> T) {

    fun <U> transform(f: (T) -> U): ContextReader<CTX, U> = ContextReader {
ctx -> f(runWith(ctx)) }


    fun <U> bind(f: (T) -> ContextReader<CTX, U>): ContextReader<CTX, U> =
        this.transform(f).join()
// original
//        ContextReader { ctx -> f(runWith(ctx)).runWith(ctx) }
}

fun <CTX, T> ContextReader<CTX, ContextReader<CTX, T>>.join():
        ContextReader<CTX, T> =
    ContextReader { ctx ->
        this.runWith(ctx).runWith(ctx)
    }
```

Chapter 10 - Read Context to Handle Commands

- Now that our ContextReader is a monad, we can combine read and write in the same transaction.

```
val listUpdater = readRow("myRowId")
    .transform { r -> r.copy(active = false) }
    .bind { r -> writeRow(r) }

runInTransaction(listUpdater).exxpectSuccess()
```

- It is worth pointing out explicitly that all the database changes take place in the last line above. Until we run the ContextReader in a transaction, it is simply maintaining a record of operation we intend to carry out on the database. We do still need to implement `readRow` and `writeRow`.

```
typealias TxReader<T> = ContextReader<Transaction, T>

fun readRow(id: String): TxReader<ToDoListProjectionRow> = TxReader { tx ->
    toDoListProjectionTable.selectWhere(tx, toDoListProjectionTable.id eq id)
        .map { it[toDoListProjectionTable.row_data] }
        .single()
}

fun writeRow(row: ToDoListProjectionRow): TxReader<Unit> = TxReader { tx ->
    toDoListProjectionTable.insertInto(tx) { newRow ->
        newRow[id] = row.id.toRowId()
        newRow[row_data] = row
    }
}

data class TransactionProvider(
        private val dataSource: DataSource,
        val isolationLevel: TransactionIsolationLevel,
        val maxAttempts: Int = 10): ContextProvider<Transaction> {

    override fun <T> tryRun(
            reader: ContextReader<Transaction, T>): Outcome<ContextError, T>
=

        inTopLevelTransaction(
               db = Database.connect(dataSource),
               transactionIsolation = isolationLevel.jdbcLevel,
               repetitionAttempts = maxAttempts) {

            addLogger(StdOutSqlLogger)

            try {
                reader.runWith(this).asSuccess()
            } catch (t: Throwable) {
                rollback()
                TransactionError("Transaction rolled back: ${t.message}", t)
                    .asFailure()
            }
        }
}

// We can now successfully run the full test on the projection row
class TxContextReaderTest {

    @Test
```

```
    fun `write and read from a table`() {

        val user = randomUser()
        val expectedList = randomToDoList()
        val listId = ToDoListId.mint()
        val row = ToDoListProjectionRow(listId, user, true, expectedList)

        val listReader: TxReader<ToDoList> =
            writeRow(row)
              .bind { readRow(listId.toRowId()) }
              .transform { row -> row.list }

        val list = transactionContextForTest().tryRun(listReader)
            .expectSuccess()

        expectThat(list).isEqualTo(expectedList)
    }
}
```

- We still want to support in-memory storage, so we modify our in-memory EventStreamer.

```
// Before - no ContextReader
interface EventStreamer<E: EntityEvent, NK: Any> {
    fun fetchByEntity(entityId: EntityId): List<E>?
    fun fetchAfter(eventSeq: EventSeq): Sequence<StoredEvent<E>>
    fun retrieveIdFromNaturalKey(key: NK): EntityId?
    fun store(newEvents: Iterable<E>): List<StoredEvent<E>>
}

// After - ContextReader
interface EventStreamer<CTX, E : EntityEvent, NK : Any> {
    fun fetchByEntity(entityId: EntityId): ContextReader<CTX, List<E>>
    fun fetchAfter(eventSeq: EventSeq):
            ContextReader<CTX, Sequence<StoredEvent<E>>>
    fun retrieveIdFromNaturalKey(key: NK): ContextReader<CTX, EntityId?>
    fun store(newEvents: Iterable<E>):
            ContextReader<CTX, List<storedEvent <E>>>
}

typealias ToDoListInMemoryRef = AtomicReference<List<ToDoListStoredEvent>>
typealias InMemoryEventsReader<T> = ContextReader<ToDoListInMemoryRef, T>

class InMemoryEventsProvider() : ContextProvider<ToDoListInMemoryRef> {

    val events = AtomicReference<List<ToDoListStoredEvent>>(listOf())

    override fun <T> tryRun(reader: InMemoryEventsReader<T>) =
        try {
            reader.runWith(events).asSuccess()
        } catch (e: Exception) {
            ToDoListEventsError("Operation failed: ${e.message}", e)
                .asFailure()
        }
}

class EventStreamerInMemory : ToDoListEventStreamer<ToDoListInMemoryRef> {

    override fun store(newEvents: Iterable<ToDoListEvent>) =
        InMemoryEventsReader { events ->
            newEvents.toSavedEvents(events.get().size.toLong())
                .also { ne -> events.updateAndGet { it + ne } }
        }
    //... similar changes to rest of the methods
}
```

- Next we implement `EventStreamerTx`, an implementation of the `EventStreamer` working with the database and transactions. Start with tests!

```
fun transactionProviderForTest() = TransactionProvider(
        pgDataSourceForTest(), ReadCommitted)
fun createToDoListEventStreamerOnPg(): EventStreamerTx = TODO()

class ToDoListEventStreamerOnPgTest {

    val user = randomUser()

    val streamer = createToDoListEventStreamerOnPg()

    private val txProvider = transactionProviderForTest()

    @Test
    fun `store some events and then fetch them by entity`() {

        val newList1 = randomListName()
        val newList2 = randomListName()
        val newList3 = randomListName()
        val listId1 = ToDoListId.mint()
        val listId2 = ToDoListId.mint()
        val listId3 = ToDoListId.mint()
        val item1 = randomItem()
        val item2 = randomItem().copy(dueDate = LocalDate.now())
    }

    val eventsToStore = listOf(
        ListCreated(listId1, user, newList1),
        ListCreated(listId2, user, newList2),
        ItemAdded(listId2, item1),
        ItemAdded(listId2, item2),
        ListCreated(listId3, user, newList3)
    )

    val storeAndFetch = streamer.store(
        eventsToStore
    ).bind {
        streamer.fetchByEntity(listId2)
    }

    val events = txProvider.tryRun(storeAndFetch).expectSuccess()

    expectThat(events).isEqualTo(
        listOf(
            ListCreated(listId2, user, newList2),
            ItemAdded(listId2, item1),
```

```
            ItemAdded(listId2, item2),
        )
    )
}

// For this test to run, we need to implement EventStreamerTx
// But first let's add a method to the PgEventTable
fun PgEventTable.queryEvents(
        condition: Op<Boolean>): TxReader<List<StoredEvent<PgEvent>>> =
    TxReader { tx ->
        selectWhere(tx, condition, id).map(::rowToPgEvent)
    }

class EventStreamerTx<E : EntityEvent, NK : Any>(
    private val table: PgEventTable,
    private val eventParser: Parser<E, PgEvent>,
    private val naturalKeySql: (NK) -> String
) : EventStreamer<Transaction, E, NK> {

    private fun pgEventsToEvents(pgEvents: List<StoredEvent<PgEvent>>) =
        pgEvents.map {
            StoredEvent(it.eventSeq, it.storedAt,
                eventParser.parseOrThrow(it.event))
        }

    override fun fetchByEntity(entityId: EntityId): TxReader<List<E>> =
        table.queryEvents(table.entity_id eq entityId.raw)
            .transform { pgEvents ->
                pgEventsToEvents(pgEvents).map(StoredEvent<E>::event) }

    override fun fetchAfter(
            eventSeq: EventSeq): TxReader<List<StoredEvent<E>>> =
        table.queryEvents(table.id greater eventSeq.progressive)
            .transform(this::pgEventsToEvents)

    override fun retrieveIdFromNaturalKey(key: NK): TxReader<EntityId?> =
        table.getEntityIdBySql(naturalKeySql(key))

    override fun store(events: Iterable<E>): TxReader<List<StoredEvent<E>>> =
        table
            .insertEvents(events.map { eventParser.render(it) })
            .transform(this::pgEventsToEvents)
}

typealias ToDoListEventStreamerTx =
        EventStreamer<Transaction, ToDoListEvent, UserListName>
```

```
fun createToDoListEventStreamerOnPg(): ToDoListEventStreamerTx =
    EventStreamerTx(toDoListEventsTable, toDoListEventParser()) { natKey ->
    """
    SELECT    entity_id
    FROM      todo_list_events
    WHERE     event_type = 'ListCreated'
              and json_data ->> 'owner' = '${natKey.user.name}'
              and json_data ->> 'name' = '${natKey.listName.name}'
    """.trimIndent()
}
```

- Let's backup and take in the big picture now. We have:
  - `EventStreamer` - knows the events and the database (or in-memory list), and it knows how to search for data in it, but it doesn't know the domain entities.
  - `EventStore` - knows how to convert events to domain entities and how to use the `EventStreamer`. It doesn't know about actual persistence and can work with both the database and in-memory streamers, and it doesn't know about commands.
  - `CommandHandler` - manages the logical transactions; it know how to use the `EventStore`, but it doesn't know anything about the streamer.
- So thinking about our `CommandHandler`, it needs a `ContextProvider` and an `EventStore` passed as constructor dependencies. The `CommandHandler` must be generic over the context type. The `Command::execute` methods will return a `ContextReader`, so they need to be bound together with the storing of events at the end of the operation:

```
class ToDoListCommandHandler<CTX>(
    private val contextProvider: ContextProvider<CTX>,
    private val eventStore: ToDoListEventStore<CTX>
) : (ToDoListCommand) -> ToDoListCommandOutcome {

    override fun invoke(command: ToDoListCommand): ToDoListCommandOutcome =
        contextProvider.tryRun(
            when (command) {
                is CreateToDoList -> command.execute()
                is AddToDoItem -> command.execute()
            }.bindOutcome(eventStore)
        ).join()
            .transform { storedEvents -> storedEvents.map { it.event } }
            .transformFailure { it as? ZettaiError
                                ?: ToDoListCommandError(it.msg) }

    private fun CreateToDoList.execute() =
        eventStore.retrieveByNaturalKey(UserListName(user, name))
            .transform { listState ->
                when (listState) {
                    null -> ListCreated(ToDoListId.mint(),
                            user, name).asCommandSuccess()
                    else -> InconsistentStateError(this,
listState).asFailure()
                }
            }

    private fun AddToDoItem.execute() =
        eventStore.retrieveByNaturalKey(UserListName(user, name))
            .transform { listState ->
                when (listState) {
                    is ActiveToDoList ->
                        if (listState.items.any {
                                it.description == item.description })
                            ToDoListCommandError("two items with same name!")
                                .asFailure()
                        else {
                            ItemAdded(listState.id, item).asCommandSuccess()
                        }
                    else -> InconsistentStateError(this,
listState).asFailure()
                }
            }
}
```

- `bindOutcome` is worth calling out, because it works as a monad transformer.

```
fun <CTX, E : OutcomeError, T, U> ContextReader<CTX, Outcome<E, T>>
        .bindOutcome(f: (T) -> ContextReader<CTX, U>):
                                ContextReader<CTX, Outcome<E, U>> =
    ContextReader { ctx ->
        val outcome = runWith(ctx)
        when (outcome) {
            is Success -> f(outcome.value).runWith(ctx).asSuccess()
            is Failure -> outcome
        }
    }
```

- While it is generally not possible to combine monads of different types, it is possible to write specific ad hoc functions like this. They are called monad transformers and you need to write on for each pair of monads you want to combine.
- It's important that no two commands on the same entity are processed at the same time, but we are going to handle that by setting the PostgreSQL isolation level to *Serializable*. This could be a bottleneck and need to be replaced if we had very high traffic on our service.
- And now a Main.kt to pull it all together:

```
// lib/source/kotlin/com/ubertob/fotf/zettai/webserver/Main.kt
fun main() {

    val dataSource = prepareProductionDatabase()

    val streamer = createToDoListEventStreamerOnPg()
    val eventStore = ToDoListEventStore(streamer)
    val txProvider = TransactionProvider(dataSource,
                        TransactionIsolationLevel.Serializable)

    val commandHandler = ToDoListCommandHandler(txProvider, eventStore)

    //...
}
```

- We're going to continue using the in-memory event store for our tests involving `DomainOnlyActions` and use the database for `HttpActions`.

- You could store projections in-memory or in the database. It really depends on performance needs and memory costs. We're going to convert one projection to store in the database and leave the other in-memory so you see both ways. There are no changes necessary to the `ToDoListProjection` interface or the projector, since they are independent of persistence details. We're going to make the test run twice, once for in-memory and once for the database.

```
// queries.ToDoListProjectionAbstractTest.kt
abstract class ToDoListProjectionAbstractTest {

    abstract fun buildListProjection(
            events: List<ToDoListEvent>): ToDoListProjection

    val user = randomUser()

    @Test
    fun `findAll returns all the lists of a user`() {

        val listName1 = randomListName()
        val listName2 = randomListName()

        val projection = buildListProjection(
            listOf(
                ListCreated(ToDoListId.mint(), user, listName1),
                ListCreated(ToDoListId.mint(), user, listName2),
                ListCreated(ToDoListId.mint(), randomUser(),
randomListName())
            )
        )

         expectThat(projection.findAll(user).expectSuccess())
               .isEqualTo(listOf(listName1, listName2))
    }

    @Test
    fun `findList get list with correct items`() {

        val listName = randomListName()
        val id = ToDoListId.mint()
        val item1 = randomItem()
        val item2 = randomItem()
        val item3 = randomItem()

        val projection: ToDoListProjection = buildListProjection(
            listOf(
                ListCreated(id, user, listName),
                ItemAdded(id, item1),
                ItemAdded(id, item2),
                ItemModified(id, item2, item3),
                ItemRemoved(id, item1)
            )
        )
```

```
        expectThat(projection.findList(user, listName).expectSuccess())
                    .isNotNull()
                .isEqualTo(ToDoList(listName, listOf(item3)))
    }
}

//
integrationtesting.com.ubertob.fotf.zettai.db.ToDoListProjectionOnPgTest.kt
internal class ToDoListProjectionOnPgTest : ToDoListProjectionAbstractTest()
{

    val dataSource = pgDataSourceFortest()
    val txProvider = TransactionProvider(dataSource,
                         TransactionIsolationLevel.ReadCommitted)
    val streamer = createToDoListEventStreamerOnPg()
    val projection = ToDoListProjectionOnPg(txProvider)
                    { txProvider.tryRun(streamer.fetchAfter(it)) }

    override fun buildListProjection(events: List<ToDoListEvent>) =
        projection.apply {
            txProvider.tryRun(streamer.store(events)).expectSuccess()
            update()
        }
}

// queries.ToDoListProjectionInMemoryTest
internal class ToDoListProjectionInMemoryTest
                    : ToDoListProjectionAbstractTest() {

    override fun buildListProjection(
                    events: List<ToDoListEvent>): ToDoListProjection =
        ToDoListProjectionInMemory { after ->
            events.mapIndexed { i, e ->
                StoredEvent(EventSeq(after.progressive + i + 1),
                Instant.now(),
                e)
            }.asSuccess()
        }.also(ToDoListProjectionInMemory::update)
}
```

- So now we have tests, so we need to update our `Projection` to work with `ContextReader` and `ContextProvider`.

```
typealias FetchStoredEvents<E> =
            (EventSeq) -> Outcome<OutcomeError, List<StoredEvent<E>>>
typealias ProjectEvents<R, E> = (E) -> List<DeltaRow<R>>

interface Projection<CTX, R : Any, E : EntityEvent> {

    val contextProvider: ContextProvider<CTX>
    val eventProjector: ProjectEvents<R, E>
    val eventFetcher: FetchStoredEvents<E>

    fun readRow(rowId: RowId): ContextReader<CTX, R?>
    fun saveRow(rowId: RowId, row: R): ContextReader<CTX, Unit>
    fun deleteRow(rowId: RowId): ContextReader<CTX, Unit>
    fun updateRow(rowId: RowId, updateFn: (R) -> R): ContextReader<CTX, Unit>
    fun lastProjectedEvent(): ContextReader<CTX, EventSeq>
    fun updateLastProjectedEvent(eventSeq: EventSeq): ContextReader<CTX,
Unit>

    fun update() {
        contextProvider.tryRun { ctx ->
            eventFetcher(lastProjectedEvent()
                .runWith(ctx))
                .transform {
                    it.onEach { storedEvent ->
                        applyDelta(eventProjector(storedEvent.event))
                            .runWith(ctx)
                    }.lastOrNull()?.apply {
                        updateLastProjectedEvent(eventSeq)
                            .runWith(ctx)
                    }
                }
        }.recover {
            println("Error during update! $it")
        }
    }

    fun applyDelta(deltas: List<DeltaRow<R>>): ContextReader<CTX, Unit> =
        deltas.fold(ContextReader {}) { reader, delta ->
            reader composeWith delta.transformation()
        }

    fun DeltaRow<R>.transformation() =
        when (this) {
            is CreateRow -> saveRow(rowId, row)
            is DeleteRow -> deleteRow(rowId)
```

```
            is UpdateRow -> updateRow(rowId) { oldRow -> updateRow(oldRow) }
        }

    infix fun <CTX, T> ContextReader<CTX, T>.composeWith(
                            other: ContextReader<CTX, T>) = bind { other }
}
```

- We have options on when to update our Projections. We probably don't want to do it on each query like we did when they were in-memory, because query performance is important. We're going to update them when we save events to the database, but not in the same transaction so failing to update a projection won't roll back the new events. We could also update the projections on a timer is "reasonably fresh" is good enough for our purposes.
- Calling `runWith` multiple times if an example of the functional imperative approach. To make things a little nicer add a `doRun` method to our `ContextProvider` interface and overload the (+) operator to `runWith(ctx)`.

```
interface ContextProvider<CTX> {
    fun <T> tryRun(reader: ContextReader<CTX, T>): Outcome<ContextError, T>

    fun <T> doRun(block: ContextWrapper<CTX>.() -> T) =
        tryRun(ContextReader { ctx -> block(ContextWrapper(ctx)) } )
}

data class ContextWrapper<CTX>(val context: CTX) {
    operator fun <T> ContextReader<CTX, T>.unaryPlus(): T = runWith(context)
}

// now rewrite update
fun update() {
    contextProvider.doRun {
        eventFetcher(+lastProjectedEvent())
            .transform {
                it.onEach { storedEvent ->
                    +applyDelta(eventProjector(storedEvent.event))
                }.lastOrNull()?.apply {
                    +updateLastProjectedEvent(eventSeq)
                }
            }
    }.recover {
        println("Error during update! $it")
    }
}
```

- To complete our projection, we need to implement queries in SQL. Here is `findAll` and other queries are similar and in repo:

```
class ToDoListProjectionOnPg(
    txProvider: ContextProvider<Transaction>,
    readEvents: FetchStoredEvents<ToDoListEvent>
) : ToDoListProjection,
    Projection<Transaction, ToDoListProjectionRow, ToDoListEvent> by
        PgProjection(
            txProvider,
            readEvents,
            ToDoListProjection.Companion::eventProjector,
            projectionTable = toDoListProjectionTable,
            lastEventTable = toDoListLastEventTable
        ) {

    override fun findAll(user: User): Outcome<OutcomeError, List<ListName>> =

        contextProvider.tryRun(
            toDoListProjectionTable.selectRowsByJson(findAllByUserSQL(user))
        ).transform { it.map { row -> row.list.listName } }

    private fun findAllByUserSql(user: User): String =
        """SELECT *
           FROM   ${toDoListProjectionTable.tableName}
           WHERE  row_data ->> 'user' = '${user.name}'
        """.trimIndent()

    // other methods...
}
```

- Our fetcher is a little cumbersome, so we'll replace it with a `runWith` method of the `ContextProvider` interface and expand out our main:

```
fun <A, T> runWith(readerBuilder: (A) -> ContextReader<CTX, T>):
    (A) -> Outcome<ContextError, T> =
        { input -> tryRun( readerBuilder( input)) }

fun main() {

    val dataSource = prepareProductionDatabase()

    val streamer = createToDoListEventStreamerOnPg()
    val eventStore = ToDoListEventStore(streamer)
    val txProvider = TransactionProvider(dataSource, Serializable)

    val commandHandler = ToDoListCommandHandler(txProvider, eventStore)

    val fetcher = txProvider.runWIth(streamer::fetchAfter)

    val queryHandler = ToDoListQueryRunner(
        ToDoListProjectionOnPg(txProvider, fetcher),
        ToDoItemProjection(fetcher)
    )

    val hub = ToDoListHub(queryHandler, commandHandler)

    Zettai(hub).asServer(Jetty(8080)).start()
}
```

- At this point, Zetta has full persistence and is a perfectly usable application!
- CQRS and Event Sourcing best practices.

    1. Each event and each command should work on a single entity. For this reason, the entity of an event sourcing models should coincide with the *transactional scope* so we can modify it inside a single transaction.
    2. We need to list all the commands that we need to complete our requirements. If we have the commands, we can identify all the possible states of our entity. Each state must represent a behavior, which is simply a different way our entity will respond to inputs. Different behaviors should be mapped on different states. In other words, there shouldn't be if's in our code where the same state would do different things based on internal attributes. States should be determined by what can happen afterwards, their behavior, not by their history.
    3. Draw and keep updating the state diagram for each entity. An event must have a single destination, but it can have multiple origins. In other words, in a drawing the arrows can

fan in but they shouldn't fan out.

4. Names can be challenging. Commands should be in imperative form. Try to avoid *meek commands* like `TryToPublish` or `EnsureCorrectPayment`. Just call them `Publish` or `ProcessPayment`. States are situations with a definite behavior. If there is no good noun, they can be expressed like verb+ing like "Waiting for xxx" or "Listening at xxx".

- Events should be versioned with the version stored in the database. Read *Versioning in an Event Sourced System* by Greg Young for more on that. If an event can't be migrated to the new version, it's better to consider the new event a completely new event type and stop using the old event. The old event would be kept only for compatibility purposes.

- Rather than migrating stored projections, it is easier to recreate them using events from the start and create a new table for the updated projection.

- Chapter 10 had several interesting Monad exercises: a Logger monad, a ConsoleContext monad, and an implementation of the RPN calculator using the ConsoleContext monad. I'm listing all three here since they're instructive.

```
// bind joins the two log values, so it respects monadic laws
data class Logger<T>(val value: T, val log: List<String>) {
    fun <U> transform(f: (T) -> U): Logger<U> = Logger(f(value), log)

    fun <U> bind(f: (T) -> Logger<U>): Logger<U> = f(value).let {
        Logger(it.value, log + it.log)
    }
}

// ConsoleContext is a Reader interface for console IO
interface ConsoleContext {
    fun printLine(msg: String): String
    fun readLine(): String
}

class SystemConsole : ConsoleContext {
    override fun printLine(msg: String): String = msg.also { println(msg) }

    val reader = BufferedReader(InputStreamReader(System.`in`))
    override fun readLine(): String = reader.readLine()
}

fun contextPrintln(msg: String) =
    ContextReader<ConsoleContext, String> { ctx -> ctx.printLine(msg) }
fun contextReadln() =
    ContextReader<ConsoleContext, String> { ctx -> ctx.readLine() }

fun greetingsOnConsole() {
    contextPrintln("Hello, what's your name?")
        .bind { _ -> contextReadln() }
        .bind { name -> contextPrintln("Hello, $name!") }
        .runWith(SystemConsole())
}

fun main() {
  greetingsOnConsole()
}

// For ConsoleRPNCalculator using ConsoleContext
fun main() {
    consoleRpnCalculator().runWith(SystemConsole())
}

tailrec fun consoleRpnCalculator(): ContextReader<ConsoleContext, String> =
    contextPrintln("Write an RPN expression to calculate the result or Q to
```

```
quit.")
        .bind { _ -> contextReadln() }
        .bind { input ->
            if (input == "Q")
                contextPrintln("Bye!")
            else
                contextPrintln("The result is: ${RpnCalc.calc(input)}")
        }
        .bind { msg ->
            if (msg == "Bye!")
                contextPrintln("")
            else
                consoleRpnCalculator()
        }

// The provided tests for the RPN calculator show how to use it, how to
// redirect input, and how to use multi-line strings to define expectation.
class E03_ConsoleRPNCalculatorTest {

    @Test
    fun `RPN calculator read and write from console`() {
        val output = ByteArrayOutputStream()
        val input = ByteArrayInputStream(
            """
            4 3 2 1 - + *
            1 2 3 * 4 - +
            Q
        """.trimIndent().toByteArray()
        )

        val stdOut = System.out
        val stdIn = System.`in`

        try {
            System.setIn(input)
            System.setOut(PrintStream(output))

            consoleRpnCalculator().runWith(SystemConsole())

        } finally {
            System.setOut(stdOut)
            System.setIn(stdIn)
        }

        val expected =
            """Write an RPN expression to calculate the result or Q to quit.
```

```
                |The result is: 16.0
                |Write an RPN expression to calculate the result or Q to quit.
                |The result is: 3.0
                |Write an RPN expression to calculate the result or Q to quit.
                |Bye!
                |
                |""".trimMargin()
        expectThat(output.toString()).isEqualTo(expected)
    }
}
```

Chapter 11 - Validating Data with Applicatives

- To start this chapter off, we're picking a new user story: Renaming a List.

```
@DDT
fun `the list owner can rename a list`() = ddtScenario {
    setup {
        ben.`starts with a list`("shopping", emptyList())
    }.thenPlay(
        ben.`can add #item to the #listname`("carrots", "shopping"),
        ben.`can rename the list #oldname as #newname`(
            origListName = "shopping",
            newListName = "grocery"
        ),
        ben.`can add #item to the #listname`("potatoes", "grocery"),
        ben.`can see #listname with #itemnames`(
            "grocery", listOf("carrots", "potatoes")
        )
    ).wip(LocalDate.of(2026,07,25))
}
```

- To get this compiling, we need to create the rename step on the `ToDoListOwner` and add the `renameList` method to `ZettiaAction`.

```
// tooling.ZettaiActions.kt
interface ZettaiActions : DomainActions<DdtProtocol> {
    // other methods...
    fun renameList(user: User, oldName: ListName, newName: ListName)
}

// commands.ToDoListCommand.kt
data class RenameToDoList(
        val user: User,
        val oldName: ListName,
        val newName: ListName): ToDoListCommand()

// tooling.DomainOnlyActions.kt
class DomainOnlyActions: ZettaiActions {
    // other methods...
    override fun renameList(
            user: User,
            oldName: ListName,
            newName: ListName) {
        hub.handle(RenameToDoList(user, oldName, newName))
    }
}

// tooling.HttpActions
data class HttpActions(val env: String = "local") : ZettaiActions {
    // other methods....
    private fun renameListForm(newName: ListName): Form =
        listOf("newListName" to newName.name)

    override fun renameList(
            user: User,
            oldName: ListName,
            newName: ListName) {
        val response = submitToZettai(
                renameListUrl(user, oldName), renameListForm(newName))
        expectThat(response.status).isEqualTo(Status.SEE_OTHER)
    }
}

// webserver.Routes.ks
"/todo/{user}/{listname}/rename" bind POST to ::renameList

private fun renameList(request: Request): Response {
    val user = request.extractUser()
            .onFailure { return Response(BAD_REQUEST).body(it.msg) }
```

```
    val listName = request.extractListName()
            .onFailure { return Response(BAD_REQUEST).body(it.msg) }

    val newListName = request.form("newlistname")
            ?.let(ListName.Companion::fromUntrusted)
            ?: return Response(BAD_REQUEST).body("missing new listname in
form")

    return hub.handle(RenameToDoList(user, listName, newListName))
            .transform { Response(SEE_OTHER)
                        .header("Location", todoListPath(user,
newListName)) }
            .recover { Response(UNPROCESSABLE_ENTITY).body(it.msg) }
}

// commands.ToDoListCommandHandler.kt
class ToDoListCommandHandler<CTX>(
    private val contextProvider: ContextProvider<CTX>,
    private val eventStore: ToDoListEventStore<CTX>
) : (ToDoListCommand) -> ToDoListCommandOutcome {

    override fun invoke(command: ToDoListCommand): ToDoListCommandOutcome =
        contextProvider.tryRun(
            when (command) {
                is CreateToDoList -> command.execute()
                is AddToDoItem -> command.execute()
                is RenameToDoList -> command.execute()
            }.bindOutcome(eventStore)
        ).join()
            .transform { storedEvents -> storedEvents.map { it.event } }
            .transformFailure { it as? ZettaiError ?:
ToDoListCommandError(it.msg) }

    // other methods...
    private fun RenameToDoList.execute(): CommandOutcomeReader<CTX> =
        eventStore.retrieveByNaturalKey( UserListName(user, oldName) )
            .transform { listState ->
                when (listState) {
                    is ActiveToDoList -> {
                        ListRenamed(listState.id, user,
newName).asCommandSuccess()
                    }
                    null -> ToDoListCommandError("list $oldName not
found").asFailure()
                    else -> InconsistentStateError(this,
listState).asFailure()
```

```
                }
            }
        // other methods...
}

// events.ToDoListEvent.kt
data class ListRenamed(override val id: ToDoListId,
        val owner: User, val newName: ListName) : ToDoListEvent()
```

- If we add a TODO to the projections that don't know how to project the event, this will compile.

```
// events.ToDoListEventTest.kt
@Test
fun `renaming the list`() {
    val newName = randomListName()
    val events: List<ToDoListEvent> = listOf(
        ListCreated(id, user, name),
        ItemAdded(id, item1),
        ListRenamed(id, user, newName)
    )

    val list = events.fold()

    expectThat(list)
        .isEqualTo(ActiveToDoList(id, user, newName, listOf(item1)))
}

// to pass this we need to add a `ListRenamed` event case to the
// `ActiveToDoList`. events.ToDoListEvent.kt
data class ActiveToDoList internal constructor(
    val id: ToDoListId,
    val owner: User,
    val name: ListName,
    val items: List<ToDoItem>
) :
    ToDoListState() {
    override fun combine(event: ToDoListEvent): ToDoListState =
        when (event) {
            is ItemAdded -> copy(items = items + event.item)
            is ItemRemoved -> copy(items = items - event.item)
            is ItemModified -> copy(items = items - event.prevItem +
event.item)
            is ListPutOnHold -> onHold(event.reason)
            is ListClosed -> close(event.closedOn)
            is ListRenamed -> rename(event.newName)
            else -> this //ignore other events
        }
}
// other methods...
fun ActiveToDoList.rename(newName: ListName) = copy(name = newName)
```

- So we have two projections that need to know how to handle the event. The `ToDoItemProjection` only has two fields in its row - the list ID and the item details - so it doesn't need to do anything. The `ToDoListProjection` need to handle the rename, so we'll write a test for that.

```
// queries.ToDoListProjectionAbstractTest
@Test
fun `findList get a renamed list`() {

    val listName = randomListName()
    val id = ToDoListId.mint()
    val item1 = randomItem()
    val newListName = randomListName()

    val projection: ToDoListProjection = buildListProjection(
        listOf(
            ListCreated(id, user, listName),
            ItemAdded(id, item1),
            ListRenamed(id, user, newListName)
        )
    )

    expectThat(projection.findList(user, listName).expectSuccess()).isNull()
    expectThat(projection.findList(user, newListName).expectSuccess())
            .isNotNull()
            .isEqualTo(ToDoList(newListName, listOf(item1)))
}

// For this to pass, we need to handle the new event in the projector
// queries.ToDoListProjection.kt
data class ToDoListProjectionRow(
        val id: ToDoListId,
        val user: User,
        val active: Boolean,
        val list: ToDoList) {
    // other methods...
    fun rename(newName: ListName): ToDoListProjectionRow =
            copy(list = list.copy(listName = newName))
}

interface ToDoListProjection {

    companion object {
        fun eventProjector(e: ToDoListEvent) =
            when (e) {
                // other events...
                is ListRenamed -> UpdateRow(e.rowId()) { rename(e.newName) }
            }.toSingle()
    }
}
```

- We have completely coded the happy path, but renaming can fail if a list of the same name already exists.

```
// commands.ToDoListCommands.kt
@Test
fun `Rename list fails if a list with same name already exists`() {

    handler(CreateToDoList(user, name)).expectSuccess()

    val newName = randomListName()
    handler(CreateToDoList(user, newName)).expectSuccess()
    val res = handler(RenameToDoList(user, name, newName)).expectFailure()
    expectThat(res).isA<ToDoListCommandError>()
}
```

- Unsurprisingly, this test fails. We need to search for `oldName` and make sure it exits and search for `newName` to make sure it's free.

```
// commands.ToDoListCommandHandler.kt
private fun RenameToDoList.execute(): CommandOutcomeReader<CTX> =
    retrieveOldAndNewListName()
        .transform { (listState, newNameList) ->
            when (listState) {
                is ActiveToDoList -> {
                    if (newNameList != null)
                        ToDoListCommandError("list $newName already exists")
                                .asFailure()
                    else
                        ListRenamed(listState.id, user, newName)
                                .asCommandSuccess()
                }

                null -> ToDoListCommandError("list $oldName not found")
                                .asFailure()
                else -> InconsistentStateError(this, listState).asFailure()
            }
        }

private fun RenameToDoList.retrieveOldAndNewListName():
        ContextReader<CTX, Pair<ToDoListState?, ToDoListState?>> =

    eventStore.retrieveByNaturalKey(UserListName(user, oldName))
        .bind { currList ->
            eventStore.retrieveByNaturalKey(UserListName(user, newName))
                .transform { currList to it }
    }
```

- This is all good except now our DDT is failing. After we rename the list, the commands lose track of it and you can't access it under the new name. The problem is `retrieveIdFromNaturalKey` queries by user and list name and expects to find a `ListCreated` for the user/list name pairing. A renamed list was never created under the new name. Let's capture this with a test:

```
// commands.ToDoListCommandsTest.kt
@Test
fun `RetrieveIdFromNaturalKey consider the most recent ListName`() {
    handler(CreateToDoList(user, name)).expectSuccess()
    val newName = randomListName()
    handler(RenameToDoList(user, name, newName)).expectSuccess()
    val newNewName = randomListName()
    handler(RenameToDoList(user, name, newNewName)).expectSuccess()

    handler(AddToDoItem(user, name, randomItem())).expectFailure()
    handler(AddToDoItem(user, newName, randomItem())).expectFailure()
    handler(AddToDoItem(user, newNewName, randomItem())).expectSuccess()
}

// We can get this test to pass by changing retrieveIdFromNaturalKey
// events.EventstreamerInMemory.kt
override fun retrieveIdFromNaturalKey(key: UserListName) =
    InMemoryEventsReader { events ->
        events.get()
            .mapNotNull {
                when (val e = it.event) {
                    is ListCreated -> if (e.owner == key.user)
                        e.id to e.name else null
                    is ListRenamed -> if (e.owner == key.user)
                        e.id to e.newName else null
                    else -> null
                }
            }
            .groupBy({ it.first }, { it.second })
            .filter { it.value.lastOrNull() == key.listName }
            .keys.singleOrNull()
}

// This make our new test and the domain-only DDT pass, but the HTTP DDT
// is still failing. We're going to test the equivalent SQL query.
// db.ToDoListEventStreamerOnPgTest.kt
@Test
fun `RetrieveIdFromNaturalKey considers only the most recent name of a
list`() {

    val name = randomListName()
    val newName = randomListName()
    val newNewName = randomListName()
    val listId = ToDoListId.mint()
```

```
    val eventsToStore = listOf(
        ListCreated(listId, user, name),
        ListRenamed(listId, user, newName),
        ListRenamed(listId, user, newNewName)
    )

    transactionProvider.doRun {
        +streamer.store(eventsToStore)

        expectThat(
            +streamer.retrieveIdFromNaturalKey(UserListName(user, name))
        ).isNull()

        expectThat(
            +streamer.retrieveIdFromNaturalKey(UserListName(user, newName))
        ).isNull()

        expectThat(
            +streamer.retrieveIdFromNaturalKey(UserListName(user,
newNewName))
        ).isEqualTo(listId)
    }.expectSuccess()
}
```

- To make this pass, we need similar logic to our in-memory streamer but implemented in SQL instead of Kotlin.

```
fun createToDoListEventStreamerOnPg(): ToDoListEventStreamerTx =
    EventStreamerTx(toDoListEventsTable, toDoListEventParser()) {
"""
select
  entity_id
from
  todo_list_events
inner join (
  select
   MAX(id) as maxid
  from
   todo_list_events
  where
   json_data ->> 'owner' = '${it.user.name}'
  and event_type in ('ListCreated', 'ListRenamed')
  group by
   entity_id) lastRename on
  id = lastRename.maxid
where
  json_data ->> 'owner' = '${it.user.name}'
  and (json_data ->> 'name' = '${it.listName.name}'
    or json_data ->> 'newName' = '${it.listName.name}')
""".trimIndent()
  }
```

- This is a good example of letting the compiler and DDT's guide us to add functionality quickly and safely. With these modifications all tests pass!
- Now let's consider the function we recently defined `retrieveOldAndNewListName`:

```
// retrieveOldAndNewListName
eventStore.retrieveByNaturalKey(UserListName(user, oldName))
    .bind { currList ->
        eventStore.retrieveByNaturalKey(UserListName(user, newName))
            .transform { newList -> currList to newList }
    }

// What would it look like if we extracted local variables for each functor
// and the combining functions.
val cr1: ContextReader<CTX, ToDoListState?> =
    eventStore.retrieveByNaturalKey(UserListName(user, oldName))
val cr2: ContextReader<CTX, ToDoListState?> =
    eventStore.retrieveByNaturalKey(UserListName(user, newName))
val f: (ToDoListState?, ToDoListState?) ->
        Pair<ToDoListState?, ToDoListState?> = :: Pair
return cr1.bind { v1 ->
          cr2.transform{ v2 -> f(v1, v2)}
}

// That body looks kinda generic. We will extract it to `transform2`
fun <CTX, A, B, R> transform2(
    first: ContextReader<CTX, A>,
    second: ContextReader<CTX, B>,
    f: (A, B) -> R
): ContextReader<CTX, R> =
    first.bind { a ->
        second.transform { b ->
            f(a, b)
        }
    }

// It becomes even clearer if we inline `bind`, but for Outcome
fun <ERR : OutcomeError, A, B, R> transform2(
    first: Outcome<ERR, A>,
    second: Outcome<Err, B>,
    f: (A, B) -> R
): Outcome<ERR, R> =
    when (first) {
        is Failure -> first
        is Success -> second.transform {b -> f(first.value, b) }
}

// Consider the function signatures:
// transform
// (Outcome<ERR, A>, f: (A) -> R) -> Outcome<ERR, R>
```

```
// transform2
// (Outcome<ERR, A>, Outcome<ERR, B>, f: (A ,B) -> R) -> Outcome<ERR, R>

private fun RenameToDoList.retrieveOldAndNewListName() =
    transform2(
        eventStore.retrieveByNaturalKey(UserListName(user, oldName)),
        eventStore.retrieveByNaturalKey(UserListName(user, newName)),
        ::Pair
    )
```

- Now consider validation. Lists should have names at least 3 characters long, no longer than forty characters, and shouldn't contain spaces, slashes, or other characters not valid in URLs. We will write each check as a separate function and collect the results and report all errors. Below is the current name validation we have and the first test prompting us to make it better.

```
fun fromUntrusted(name: String): ListName? =
    if (name.matches(pathElementPattern) && name.length in 1..40)
        fromTrusted(name) else null

@Test
fun `Names longer than 40 chars are not valid`() {
    stringsGenerator(validCharset, 41, 200)
        .take(100)
        .forEach {
            val failure = ListName.fromUntrusted(it).expectFailure()
            expectThat(failure.msg).contains("is too long")
        }
}

// We can write these really easy with a `discardUnless` helper function.
// It returns nul if a Boolean condition is false.
fun <T> T.discardUnless(predicate: T.() -> Boolean): T? =
    takeIf { predicate(it) }

fun nameTooShort(name: String): Outcome<ValidationError, String> =
    name.discardUnless { length >= 3 }
        .failIfNull(ValidationError("Name is too short!"))

fun nameTooLong(name: String): Outcome<ValidationError, String> =
    name.discardUnless { length <= 40 }
        .failIfNull(ValidationError("Name ${name} is too long"))

fun nameWithInvalidChars(name: String): Outcome<ValidationError, String> =
    name.discardUnless { matches(pathElementPattern) }
        .failIfNull(
            ValidationError(
                "Name ${name} contains illegal characters:" +
                " only letters, digits, and hyphen are allowed"
            )
        )
```

- We need a way to return multiple errors from combined failures. One good way to get started is write the function signature and then try to implement it.

```
fun <E: OutcomeError, T> combineFailures(
        first: Outcome<E, T>,
        second: Outcome<E, T>): Outcome<E, T> =
    when (first) {
        is Success<*> -> second
        if Failure<E> -> when (second) {
            is Success<*> -> first
            is Failure<E> -> ??? a failure with first.error + other error
        }
    }

// We would like to be able to pass a combining function to put the errors
// together as one. But looking at this and the signature, it looks like
// transform2 for Outcome we were playing with earlier except we want to
// combine two errors instead of two successes.
fun <ER : OutcomeError, E1 : ER, E2 : ER, T> transform2Failures(
    first: Outcome<E1, T>,
    second: Outcome<E2, T>,
    f: (E1, E2) -> ER
): Outcome<ER, T> =
    when (first) {
        is Success<*> -> second
        is Failure<E1> -> when (second) {
            is Success<*> -> first
            is Failure<E2> -> f(first.error, second.error).asFailure()
        }
    }

// Now we need a reduce for failures since we have a way to combine two.
// We also need a special `ValidationError` to store them.
fun <E : OutcomeError, T> List<Outcome<E, T>>
        .reduceFailures(f: (E, E) -> E): Outcome<E, T> =
    reduce { acc, r -> transform2Failures(acc, r, f) }

data class ValidationError(val errors: List<String>) : ZettaiError() {
    constructor(error: String) : this(listOf(error))

    override val msg: String = errors.joinToString()

    fun combine(other: ValidationError): ValidationError =
        ValidationError(errors + other.errors)
}

// And let's write a convenience extension function called `validateWith`
fun <T, E: OutcomeError> T.validateWith(
```

```
        validations: List<(T) -> Outcome<E, T>>,
        combineErrors: (E, E) -> E): Outcome<E, T> =

    validations
        .map { it(this) }
        .reduceFailures(combineErrors)

typealias ListNameValidation = (String) -> Outcome<ValidationError, String>
typealias ListNameOutcome = Outcome<ValidationError, ListName>

data class ListName internal constructor(val name: String) {
    companion object {
        // other methods...
        fun fromUntrusted(name: String): ListNameOutcome =
            name.validateListName(
                ::nameTooShort,
                ::nameTooLong,
                ::nameWithInvalidChars)
    }
}

fun String.validateListName(vararg validations: ListNameValidation) =
    validateWith(validations.toList(), ValidationError::combine)
        .transform { ListName.fromTrusted(it) }

// Now we need to return an Outcome from `extractListNameFromForm`
private fun Request.extractListNameFromForm(formName: String) =
    form(formName)
        .failIfNull(InvalidRequestError("missing list name in form!"))
        .bind { ListName.fromUntrusted(it) }

private fun createNewList(request: Request): Response {

    val user = request.extractUser()
        .onFailure { return Response(BAD_REQUEST).body(it.msg) }

    val listName = request.extractListNameFromForm("listname")
        .onFailure { return Response(BAD_REQUEST).body(it.msg) }

    hub.handle(CreateToDoList(user, listName))
        .transform { allListsPath(user) }
        .transform { Response(SEE_OTHER).header("Location", it) }
        .recover { Response(UNPROCESSABLE_ENTITY).body(it.msg) }
}

// We can handle `renameList` in a similar fashion
```

```
private fun renameList(request: Request): Response {
    val user = request.extractUser()
        .onFailure { return Response(BAD_REQUEST).body(it.msg) }
    val listName = request.extractListName()
        .onFailure { return Response(BAD_REQUEST).body(it.msg) }
    val newListName = request.extractListNameFromForm("newListName")
        .onFailure { return Response(BAD_REQUEST).body(it.msg) }
    return hub.handle(RenameToDoList(user, listName, newListName))
        .transform { Response(SEE_OTHER)
            .header("Location", todoListPath(user, newListName)) }
        .recover { Response(UNPROCESSABLE_ENTITY).body(it.msg) }
}
```

- So this is good but a little noisy and repetitive with all the `onFailure` calls.

```
// This has less repetition, but we can do better!
fun <ERR : OutcomeError, A, B, C, R> transform3(
    first: Outcome<ERR, A>,
    second: Outcome<ERR, B>,
    third: Outcome<ERR, C>,
    f: (A, B, C) -> R
): Outcome<ERR, R> =
    transform2(first,
        transform2(second, third) { b, c -> f.partial(c).partial(b) })
            { a: A, fa: (A) -> R -> fa(a) }

private fun renameList(request: Request): Response =
    transform3(
        request.extractUser(),
        request.extractListName(),
        request.extractListNameFromForm("newListName"),
        ::RenameToDoList
    ).bind { cmd ->
        hub.handle(cmd)
            .transform { Response(SEE_OTHER)
                .header("Location", todoListPath(cmd.user, cmd.newName)) }
    }.recover { Response(UNPROCESSABLE_ENTITY).body(it.msg) }
```

- An *applicative functor* can do transformations using functions with multiple parameters. Mapping a collection of validation functions over the same input and collecting the errors into a single success or combination of failures is called *traverse*. We started with a collection of functor builders and traversed it to produce a single functor with a collection inside. Type that support traverse are called traversables.

```
// traverse
fun <A, B> Iterable<A>.traverse(f: (A) -> Functor<B>): Functor<List<B>>
// similar to map where the target type is Functor<B>
fun <A, B> Iterable<A>.map(f: (A) -> Functor<B>): List<Functor<B>>

// So let's define transform2 for Holder then use it to write traverse
data class Holder<T>(private val value: T) {
    fun <U> transform(f: (T) -> U): Holder<U> = Holder(f(value))

    companion object {
        fun <A, B, R> transform2(
            first: Holder<A>,
            second: Holder<B>,
            f: (A, B) -> R
        ): Holder<R> =
            Holder(f(first.value, second.value))
    }

    fun <A, B> Iterable<A>.traverse(f: (A) -> Holder<B>): Holder<List<B>> =
        fold(Holder(emptyList())) { acc, e ->
            transform2(acc, f(e)) { list, el -> list + el }
        }

    // swapWithList, known as sequence in Haskell, Swaps a List of Functors
    // for a Functor of a List: (List<Functor<A>>) -> Functor<List<A>>
    fun <A> Iterable<Holder<A>>.swapWithList(): Holder<List<A>> =
        traverse { it }
}

// So back to Outcome
fun <T, ERR: OutcomeError, U> Iterable<T>.foldOutcome(
    initial: U,
    operation: (acc: U, T) -> Outcome<ERR, U>
): Outcome<ERR, U> =
    fold(initial.asSuccess() as Outcome<ERR, U>)
        { acc, el -> acc.bind { operation(it, el) } }

fun <E: OutcomeError, T, U> Iterable<T>.traverseOutcome(
    f: (T) -> Outcome<E, U>
): Outcome<E, List<U>> =

    foldOutcome(mutableListOf()) { list, e ->
        f(e).transform { list.add(it) }
    }
```

```
fun <E: OutcomeError, T> Iterable<Outcome<E, T>>.swapWithList(
): Outcome<E, List<T>> =
    traverseOutcome {it}

typealias sequentialApplication<A, B> =
    (Applicative<(A)->B>) -> (Applicative<A>) -> Applicative<B>
// apply is taken in Kotlin, so we will call this andApply
fun <A, B> List<(A)->B>.andApply(a: List<A>): List<B> = flatMap { a.map(it)
}
// We could rewrite this without the it for clarity to point out that
// the function is what is being mapped over the value, since andApply
// reverses positions of receiver/argument, and that is why you would end
// up with a list of lists if you weren't flattening with flatMap:
fun <A, B> List<(A)->B>.andApply(vals: List<A>): List<B> =
    flatMap { f -> vals.map(f) }
val l = listOf(Int::toString)
        .andApply( listOf(1,1,2,3,5,8) )
println(l.joinToString()) // 1, 1, 2, 3, 5, 8

// It doesn't look all that useful above, but it works. We're going to try
// a function that takes two parameters, but we need to curry it first.
fun <A, B, R> ((A, B) -> R).curry(): (A) -> (B) -> R =
        { a -> { b -> invoke(a, b) } }
fun <A, B, C, R> ((A, B, C) -> R).curry(): (A) -> (B) -> (C) -> R =
        { a -> { b -> { c -> invoke(a, b, c) } } }
// etc. It''s seldom worth currying more than five or six argument functions.
val l = listOf( String::plus.curry() )
        .andApply( listOf("hmm", "ouch", "wow") )
        .andApply( listOf("?", "!") )
println(l.joinToString()) // hmm?, hmm!, ouch?, ouch!, wow?, wow!
// applying a list of 3 to a list of 2 gives us 6 combinations.
// Rather than put our function in a list, we can start with a list of
// strings and partially apply the function with a regular map,
// then `andApply` the result list of partially applied functions:
val l = listOf("hmm", "ouch", "wow")
        .andApply(String::plus.curry())
        .andApply( listOf("?", "!") )
println(l.joinToString()) // hmm?, hmm!, ouch?, ouch!, wow?, wow!
// The code was more contrived but also more clear with a list of functions,
// so we are going to define a `transformAndCurry` function that starts with
// the function and both curries it and maps it over a list of values.
fun <A, B, R> ((A, B) -> R)
        .transformAndCurry(other: List<A>): List<(B) -> R> =
    other.map { curry()(it) }
// We could make this a bit more clear with `this`. But you get the idea.
// `curry` is being called on the receiver, the function we called
```

```
// `transformAndCurry` from. It returns a curried version of the same
function,
// the the `map` supplies it's first argument from the values of `other`.
fun <A, B, R> ((A, B) -> R)
        .transformAndCurry(other: List<A>): List<(B) -> R> =
    other.map { valA -> this.curry()(valA) }
// Let's define some others.
fun <A, B, C, R> ((A, B, C) -> R)
        .transformAndCurry(other: List<A>): List<(B) -> (C) -> R> =
    other.map { curry()(it) }
fun <A, B, C, D, R> ((A, B, C, D) -> R)
        .transformAndCurry(other: List<A>): List<(B) -> (C) -> (D)-> R> =
    other.map { curry()(it) }
val l = String::plus
        .transformAndCurry( listOf("hmm", "ouch", "wow") )
        .andApply( listOf("?", "!") )
println(l.joinToString()) // hmm?, hmm!, ouch?, ouch!, wow?, wow!
// To make this even more convenient, we are going to define shorthand
// operators. These correspond to the Haskell `fmap` or `<$>` operator
// which takes a plain function and wraps it to act on a contextual first
// argument (it's already curried because Haskell), and the `ap` or `<*>`
// operator that takes a wrapped function and applies it to a wrapped
// argument returning a wrapped result.
infix fun <A, B, C, R>
        ((A, B, C) -> R).`!`(other: List<A>): List<(B) -> (C) -> R> =
    transformAndCurry(other)
infix fun <A, B> List<(A) -> B>.`*`(a: List<A>): List<B> =
    andApply(a)
val l = String::plus `!` listOf("hmm", "ouch", "wow") `*` listOf("?", "!")
println(l.joinToString()) // hmm?, hmm!, ouch?, ouch!, wow?, wow!
```

- So to define an applicative functor, we need a type constructor to create the applicative from any type and an implementation of `andApply` or `transform2`. You can always write `transform` in terms of `transform2` or `andApply`, but not vice versa. All applicatives are functors, but not all functors are applicatives. Applicatives must obey four laws. The first two are the identity law and the homomorphism law. The identity law says if we apply an applicative with the identity function to another applicative, the result is equal to the second applicative alone. The homomorphism law says that if you apply the applicative of a function to the applicative of a value, the result must be equal to the applicative of the function applied to the value. The last two laws are the interchange and composition law. They are enforcing the same idea that the sequential application must be entirely transparent. The book doesn't explain them, but I have copied the tests that verify them from the repository for completeness.

```
val x = randomString(text, 1, 10)
val id: (String) -> String = { it }

@Test
fun `Identity law`() {
    expectThat(listOf(id) `*` listOf(x)).isEqualTo(listOf(x))
}

@Test
fun `Identity law verbose`() {
    expectThat(
        listOf(id).flatMap { f -> listOf(x).map(f) }
    ).isEqualTo(listOf(x))
}

val f: (String) -> Int = { it.length }

@Test
fun `Homomorphism law`() {
    expectThat(listOf(f) `*` listOf(x)).isEqualTo(listOf(f(x)))
}

@Test
fun `Interchange law`() {

    expectThat(
       listOf(f) `*` listOf(x)
    ).isEqualTo(
       listOf({ fx: (String) -> Int -> fx(x) }) `*` (listOf(f))
    )
}


@Test
fun `Composition law`() {

    fun <A, B, C> composeFn(f: (B) -> C): ((A) -> B) -> (A) -> C =
        { fab -> { a -> f(fab(a)) } }

    val g: (LocalDate) -> String = { it.toString() }
    val d = LocalDate.now()

    expectThat(
        listOf({ fx: (String) -> Int -> composeFn<LocalDate, String, Int>(fx)
})
```

```
        `*` listOf(f) `*` listOf(g) `*` listOf(d)
    ).isEqualTo(
        listOf(f) `*` (listOf(g) `*` listOf(d))
    )
}
```

- You'll notice in Interchange the initial function is a closure that closes over the `x` value and waits for a function to apply to `x`. the law is saying it doesn't matter whether you treat f as "the function side" and x as "the value side," or flip it around by packaging "apply to x" as its own function and treating f as the value being applied to that. The composition law simple states composing two functions and applying the composition should equal applying each function in sequence.

- With monads we set up a chain of operations where each can influence the next. With applicatives we can execute the operations independently and then combine the results. Applicatives are in a sense less powerful, but with monads the first failure would have prevented the others, so applicatives allow us to do something we cannot with monads. Not all applicatives are monads, but all monads are applicatives, because you can implement `andApply` and `transform2` with `bind`. However, when you define these via `bind` the applicative inherits the monad's first failure semantics rather than collecting errors. Consider this for `bind` of `Outcome`. This defines a perfectly good applicative, but it fails to collect multiple errors.

```
fun <E, A, B> Outcome<E, A>.bind(f: (A) -> Outcome<E, B>): Outcome<E, B> =
    when (this) {
        is Success -> f(value)
        is Failure -> this
    }

fun <E, A, B> Outcome<E, (A) -> B>.andApply(a: Outcome<E, A>): Outcome<E, B>
=
    bind { fn -> a.transform(fn) }

fun <E, A, B, R> transform2(first: Outcome<E, A>, second: Outcome<E, B>,
        f: (A, B) -> R): Outcome<E, R> =
    first.bind { a -> second.transform { b -> f(a, b) } }
```

- So all theory aside, let's get back to our application. How would we rewrite `renameList` with our new applicative infix operators.

```
fun renameList(request: Request): Response =
    ( :: RenameToDoList `!` request.extractUser()
                        `*` request.extractListName()
                        `*` request.extractNewListName())
        .bind { cmd ->
            hub.handle(cmd)
                .transform { Response(SEE_OTHER)
                .header("Location", toDoListPath(cmd.user, cmd.newName)) }
        }.recover { Response(UNPROCESSABLE_ENTITY).body(it.msg) }
```

- We actually have to do this for a lot of our requests, so we can extract a function to make it more convenient. And an `errorToResponse` function.

```
fun <C: ToDoListCommand>
    executeCommand(command: ZettaiOutcome<C>): ZettaiOutcome<C> =
        command.bind(hub::handle)

// This is not quite up to the repo version yet. We plan to handle
// `ValidationError` by displaying the error list.
fun errorToResponse(referrer: String?): ZettaiError.() -> Response = {
    when (this) {
        is ValidationError -> Response(BAD_REQUEST).body(msg)
        is InvalidRequestError -> Response(NOT_FOUND).body(msg)
        is ZettaiParsingError -> Response(BAD_REQUEST).body(msg)
        is QueryError, is ToDoListCommandError,
        is InconsistentStateError, is ZettaiRenderError ->
            Response(UNPROCESSABLE_ENTITY).body(msg)
    }
}

// And with this, we finally reach our repo version of `renameList`
private fun renameList(request: Request): Response =
    executeCommand(
        ::RenameToDoList
                `!` request.extractUser()
                `*` request.extractListName()
                `*` request.extractListNameFromForm("newListName")
    )
        .transform { Response(SEE_OTHER).header(
            "Location", todoListPath(it.user, it.newName)) }
        .recover(errorToResponse(request.referer))

// And we can rewrite some of our other commands this way too
private fun createNewList(request: Request): Response =
    executeCommand(
        ::CreateToDoList
                `!` request.extractUser()
                `*` request.extractListNameFromForm("listname")
    ).transform { allListsPath(it.user) }
        .transform { Response(SEE_OTHER).header("Location", it) }
        .recover(errorToResponse(request.referer))
```

- It's been a while since we've looked at our UI, and we need to add list renaming to it to complete the story. At this time we're going to branch out from large HTML strings to include static content like CSS files, fonts, and scripts. We will also support HTML templates to generate dynamic content.

```
// It all starts with a place for static assets.
val httpHandler = routes(
    "/" bind GET to ::homePage,
    "/todo/{user}" bind GET to ::getAllLists,
    "/todo/{user}" bind POST to ::createNewList,
    "/todo/{user}/{listname}" bind GET to ::getTodoList,
    "/todo/{user}/{listname}" bind POST to ::addNewItem,
    "/todo/{user}/{listname}/rename" bin POST to ::renameList,
    "/whatsnext/{user}" bind GET to ::whatsNext,
    "/static" bind static(ClassPath("/static"))
)
```

- The advantage of putting static content in our resources folder is that it will always be deployed together with our application, making deployment simpler. The disadvantage is that static content can't change independently of our application. For this small project it is worth the trade off.

- So we are going to define our own template engine rather than use an overly complicated off the shelf library. The main function is `renderTemplate`, which takes the template string and a map of tags that will substitute the special markers in the template with the correct values. We need three types of tag: `StringTag` for simple string replacement, `ListTag` to handle collections, and `BooleanTag` to show or hide a block of text:

```
typealias Template = CharSequence
typealias TagMap = Map<String, TemplateTag>

sealed class TemplateTag
data class StringTag(val text: String?) : TemplateTag()
data class ListTag(val tagMaps: List<TagMap>) : TemplateTag()
data class BooleanTag(val bool: Boolean) : TemplateTag()

infix fun String.tag(value: String?): Pair<String, TemplateTag> =
    this to StringTag(value)
infix fun String.tag(value: List<TagMap>): Pair<String, TemplateTag> =
    this to ListTag(value)
infix fun String.tag(value: Boolean): Pair<String, TemplateTag> =
    this to BooleanTag(value)

fun Template.renderTemplate(data: TagMap): TemplateOutcome =
    applyAllTags(data).checkForUnappliedTags()

fun renderTemplatefromResource( fileName: String,
                                data: TagMap): ZettaiOutcome<Template> =
    TemplateTag::class.java.getResource(fileName)
        .failIfNull( TemplateError("Template not found $fileName"))
        .transform(URL::readText)
        .bind { it.renderTemplate(data) }
        .transformFailure { ZettaiRenderError(it) }

// So now let's update our functions that render HTML.
// The repository has similar functions for `renderListsPage` and
// `renderWhatsNextPage` along with changes to HTML parsing in the DDTs.
fun renderListPage(
    user: User,
    toDoList: ToDoList,
    errors: String? = null
): ZettaiOutcome<HtmlPage> =
    mapOf(
        "user" tag user.name,
        "listname" tag toDoList.listName.name,
        "items" tag toDoList.items.toTagMaps(),
        "errors" tag errors,
        "if_error" tag (errors != null)
    ).renderHtml("/html/single_list_page.xhtml")

fun TagMap.renderHtml(fileName: String): ZettaiOutcome<HtmlPage> =
    renderTemplatefromResource(fileName, this)
        .transform(Template::toString)
```

```
        .transform(::HtmlPage)

private fun List<ToDoItem>.toTagMaps(): List<TagMap> =
    map {
        mapOf(
            "description" tag it.description,
            "dueDate" tag it.dueDate?.toIsoString().orEmpty(),
            "status" tag it.status.toString()
        )
    }
```

- Flash attributes let us display a one-time notification and would be perfect to communicate our validation errors. We only need a `FlashAttributesFilter` in front of routes and a `withFlash` to our error handler:

```
// webserver.Routes.kt
val httpHandler = FlashAttributesFilter.then(
    routes( /* routes definition here */ ))

fun errorToResponse(referrer: String?): ZettaiError.() -> Response = {
    when (this) {
        is ValidationError -> Response(SEE_OTHER)
                .header("Location", referrer)
                .withFlash("Validation error: $msg")
        // rest of the errors...
    }
}
```

- The main exercise in this chapter were actually pretty simple. Implement a `bind` for `Outcome` that combines successes and use it in `reduce`, and implement the applicative operators for `ContextReader`. There were too other bigger exercises to implement `EditListItems` and `DeleteListItems`. They are done in `zettai_step7_monitoring` of the code repository.

```
fun <E : OutcomeError, T, U> Outcome<E, T>.bind(
          f: (T) -> Outcome<E, U>): Outcome<E, U> =

    when (this) {
        is Success -> f(value)
        is Failure -> this
    }

fun <E : OutcomeError, S, T : S> List<Outcome<E, T>>.reduceSuccess(
        f: (S, T) -> T): Outcome<E, S> =

    reduce { acc, r ->
        acc.bind { a: T ->
            r.transform { b ->
                f(a, b)
            }
        }
    }

@Suppress("DANGEROUS_CHARACTERS")
infix fun <CTX, A, B> ContextReader<CTX, (A) -> B>.`*`(
        a: ContextReader<CTX, A>): ContextReader<CTX, B> =
    bind { a.transform(it) }


fun <CTX, A, B, C> ((A, B) -> C).transformAndCurry(
        other: ContextReader<CTX, A>): ContextReader<CTX, (B) -> C> =
    other.transform { curry()(it) }

infix fun <CTX, A, B, C> ((A, B) -> C).`!`(
        other: ContextReader<CTX, A>): ContextReader<CTX, (B) -> C> =
    transformAndCurry(other)


infix fun <CTX, A, B, C, D> ((A, B, C) -> D).transformAndCurry(
        other: ContextReader<CTX, A>): ContextReader<CTX, (B) -> (C) -> D> =
    other.transform { curry()(it) }


infix fun <CTX, A, B, C, D> ((A, B, C) -> D).`!`(
        other: ContextReader<CTX, A>): ContextReader<CTX, (B) -> (C) -> D> =
    transformAndCurry(other)
```

Chapter 12 - Monitoring and Functional JSON

- Most modern day logging is done with aggregator platforms, but we aren't a logging platform book, so we will assume someone else set all that up and just write to a local file that will somehow be aggregated.

> Log as much as you can when things go wrong, log as little as you dare when things go well.
>
> --Kirk Pepperdine

- There are two possible things to log: what we did if everything went well so we can collect metrics, and exactly what went wrong with errors so we can fix them.
- If logging is an important characteristic of our application, which it is, it should be covered by tests. They can also guide us how to best design the logging functionality. Assuming we design our logger functionally, it should be as easy to test as everything else.

```
internal class LoggerTest {

    val failingHub = TODO()
    val logger = TODO()
    val logs: List<String> = emptyList()

    @Test
    fun `commands log when there is a failure`() {
        Zettai(failingHub, logger)
            .httpHandler(Request(Method.POST, "/todo/AUser"))

        expect {
            that(logs.size).isEqualTo(1)
            that(logs[0]).startsWith("error!")
        }
    }
}
```

- Logging isn't a business domain concern. No user cares if your website has good monitoring. Since it's a non-functional requirement, it shouldn't enter our domain, so should not be passed to the hub. Since the hub methods will always return the outcome with all the error information, it is safe to log from the outside.

- What to log? Typically, pure functions need little to no logging. A basic calculation isn't worth logging. We are usually interested in things at the edges of the system: where we received a command from a user, sent a notification to an external system, or stored or retrieved something from a database. Basically, what is done by adapters needs logging. Not domain actions.
- Our logger will be a function of course. It takes an Outcome and returns Unit. It is impoosible to avoid an impure operation with a Logger, but we will minimize the opaque bits.

```
// We need a typealias for our logger and to provide it to Zettai:
typealias ZettaiLogger = (Outcome<*, *>) -> Unit

class Zettai(val hub: ZettaiHub, val logger: ZettaiLogger) : HttpHandler {
    // ...

// And we need a failingHub for tests that always fails.
val failingHub = object : ZettaiHub {
    override fun <C : ToDoListCommand> handle(command: C) =
        InvalidRequestError("failing test").asFailure()

    override fun getList(userListName: UserListName) =
        InvalidRequestError("failing test").asFailure()

    override fun getLists(user: User) =
        InvalidRequestError("failing test").asFailure()

    override fun whatsNext(user: User) =
        InvalidRequestError("failing test").asFailure()
}

// And a minimal implementation that passes the tests.
val logs = mutableListOf<String>()

val logger: ZettaiLogger = {
    val msg = when(it) {
        is Failure -> "error! $it"
        is Success -> "success! $it"
    }
    logs.add(msg)
}

// Our test is still failing, so we need to use the logger in Zettai
class Zettai(val hub: ZettaiHub, val logger: ZettaiLogger): HttpHandler {

    // other methods...

    fun <C: ToDoListCommand> executeCommand(command: ZettaiOutcome<C>) =
        command.bind(hub::handle)
            .logIt()

    fun <QP, QR> executeQuery(
        queryParams: ZettaiOutcome<QP>,
        query: (QP) -> ZettaiOutcome<QR>
    ): ZettaiOutcome<Pair<QP, QR>> =
```

```
        queryParams.bind { qp -> query(qp).transform { qp to it } }
            .logIt()

    private fun <T> ZettaiOutcome<T>.logIt(): ZettaiOutcome<T> =
        also { logger(it) }
}

// So we have ways to log success and failure; so we need tests for logging
// of success and failure. Rather than complex mocks, we already have a
// `failingHub` that always fails, and we'll add a `happyHub` that always
// succeeds:
val happyHub = object : ZettaiHub {
    override fun <C : ToDoListCommand> handle(command: C) =
        command.asSuccess()

    override fun getList(userListName: UserListName) =
        randomToDoList().asSuccess()

    override fun getLists(user: User) =
        listOf(randomListName()).asSuccess()

    override fun whatsNext(user: User) =
        randomToDoList().items.asSuccess()
}

// So now we need more tests
internal class LoggerTest {
    // declarations...

    @Test
    fun `commands log when there is a failure`() {
        Zettai(failingHub, logger)
            .httpHandler(Request(Method.POST, "/todo/AUser"))

        expect {
            that(logs.size).isEqualTo(1)
            that(logs[0]).startsWith("error!")
        }
    }

    @Test
    fun `commands always log successful calls`() {
        val times = Random.nextInt(5, 10)
        val zettai = Zettai(happyHub, logger)
        repeat(times) {
            zettai.httpHandler(
```

```
                Request(Method.POST, "/todo/AUser")
                    .form("listname", "newList")
            )
        }
        expect {
            that(logs.size).isEqualTo(times)
            that(logs[0]).startsWith("success!")
        }
    }

    @Test
    fun `queries log when there is a failure`() {
        Zettai(failingHub, logger)
            .httpHandler(Request(Method.GET, "/todo/AUser"))

        expect {
            that(logs.size).isEqualTo(1)
            that(logs[0]).startsWith("error!")
        }
    }

    @Test
    fun `queries always log successful calls`() {
        val times = Random.nextInt(5, 20)
        val zettai = Zettai(happyHub, logger)
        repeat(times) {
            zettai
                .httpHandler(Request(Method.GET, "/todo/AUser"))
        }
        expect {
            that(logs.size).isEqualTo(times)
            logs.forEach {
                that(it).startsWith("success!")
            }
        }
    }
}

// So let's start with a StreamLogger that can write to a file or StdOut
data class StreamLogger(val stream: OutputStream): ZettaiLogger {

    val writer = PrintWriter(stream, true)

    override fun invoke(outcome: Outcome<*, *>) {
        when (outcome) {
            is Success -> "success! ${outcome.value}"
```

```
            is Failure -> "error! ${outcome.error.msg}"
        }.let(writer::println)
    }
}

fun stdOutLogger() = StreamLogger(System.out)

// Let's also modify logIt on ZettaiOutcome to include additional info
fun <T> ZettaiOutcome<T>.logIt(request: Request): ZettaiOutcome<T> =
    also {
        logger(
            transformFailure {
                LoggerError(it, request) // we log request on failure
            }
        )
}

// Since executeCommand lacks the Request, we need to move the logIt calls
// to every route, but that has other advantages.
class Zettai(val hub: ZettaiHub, val logger: ZettaiLogger) : HttpHandler {
    // ...
    private fun createNewList(request: Request): Response =
        executeCommand(
            ::CreateToDoList
                    `!` request.extractUser()
                    `*` request.extractListNameFromForm("listname")
                .logIt(request)
        ).transform { allListsPath(it.user) }
            .transform { Response(SEE_OTHER).header("Location", it) }
            .recover(errorToResponse(request.referer))
    // ...

// Now let's wire out logger into the main application:
fun main() {
    // ... setup
    val zettai = Zettai(hub, stdOutLogger())

    zettai.asServer(Jetty(8080)).start()

    println("Server started at http://localhost:8080/todo/username")
}
```

- Our logs are simple but enough for many purposes. Logging levels are a commonly seen feature that overcomplicates things. However, we would like more structure so we are going to support JSON logging.

```
// This is a sealed interface and make it easy to share some fields in
// all sealed classes.
// logger.LogEntry.kt
sealed interface LogEntry {
    val time: Instant
    val hostname: String
    val msg: String
    val logContext: LogContext
}

data class LogSuccess(
    override val time: Instant,
    override val hostname: String,
    override val msg: String,
    override val logContext: LogContext
) : LogEntry

data class LogFailure(
    override val time: Instant,
    override val hostname: String,
    override val msg: String,
    override val logContext: LogContext
) : LogEntry

// logger.LogContext.kt
data class LogContext(
    val desc: String,
    val kind: OperationKind,
    val user: User?,
    val listName: ListName?
)

enum class OperationKind { Command, Query }

// logger.JsonLogger.kt
// We're going to go back to our StreamLogger and make it abstract and
// add some methods:
abstract class StreamLogger(stream: OutputStream) : ZettaiLogger {

    abstract fun onSuccess(value: Any?, logContext: LogContext): String
    abstract fun onFailure(error: OutcomeError, logContext: LogContext):
String

    val writer = PrintWriter(stream, true)
```

```
    override fun invoke(outcome: Outcome<*, *>, logContext: LogContext) {
        outcome.transform { onSuccess(it, logContext) }
            .recover { onFailure(it, logContext) }
            .let(writer::println)
    }
}

// Then define our JSON Logger extending that.
data class JsonLogger(
    private val stream: OutputStream,
    private val clock: Clock,
    private val hostName: String
) : StreamLogger(stream) {

    override fun onSuccess(value: Any?, logContext: LogContext) =
        LogSuccess(
            clock.instant(),
            hostName,
            value.toString(),
            logContext
        ).toJson()

    override fun onFailure(error: OutcomeError, logContext: LogContext) =
        LogFailure(
            clock.instant(),
            hostName,
            error.toString(),
            logContext
        ).toJson()

    // We don't know how to do JSON yet, so punting on this...
    private fun LogEntry.toJson(): String = this.toString()
}

// So how would we define a FileLogger now?
fun fileLogger(fileName: String) =
    JsonLogger(
        FileOutputStream(fileName),
        Clock.systemUTC(),
        getLocalHost().hostName
    )

// But now we need some more parameters for LogIt
private fun<T> ZettaiOutcome<T>.logIt(
    kind: OperationKind,
    description: String,
```

```
    request: Request,
    describeSuccess: (T) -> String
): ZettaiOutcome<T> =
    also {
        logger(
            transform(describeSuccess)
                .transformFailure {
                    LoggerError(it, request)
                },
            LogContext(
                description,
                kind,
                request.extractUser().orNull(),
                request.extractListName().orNull(),
            )
        )
    }

// Here's some examples of using our new logger
private fun getAllLists(request: Request): Response =
    executeQuery(
        request.extractUser(),
        hub::getLists,
    ).logIt(Query, "getAllLists", request) { "Found ${it.second.size} lists"
}

private fun addNewItem(request: Request): Response =
    executeCommand(
        ::AddToDoItem
                `!` request.extractUser()
                `*` request.extractListName()
                `*` request.extractItem()
    ).logIt(Command, "addNewItem", request) { "Item added ${it.item}" }
        .transform { Response(SEE_OTHER).header(
                        "Location", todoListPath(it.user, it.name))
        }.recover(errorToResponse(request.referer))
```

- That's all for logging, but we do need to handle the JSON.

```
// Let's start with the simplest functions
fun toJson(value: T): String
fun fromJson(json: String): Outcome<JsonError, T>

// So we want these in interfaces and more generic
interface ToJson<T, S> {
    fun toJson(value: T): S
}

interface FromJson<T, S> {
    fun fromJson(json: S): JsonOutcome<T>
}

// So a JSON converter needs to implement these two interfaces
interface JsonConverter<T> : ToJson<T, String>, FromJson<T, String>

// So, we have a JSON library. Something homegrown and half-baked.
// Kondor JSON. You define a type for each class we plan to serialize,
// usually an object. Consider serializing User and ListName. They are
// just String values with a wrapper, so we don't want a JSON object.
// logger.JLogEntry.kt
object JUser : JStringRepresentable<User>() {
    override val cons: (String) -> User = :: User
    override val render: (User) -> String = User::name
}

object JListName : JStringRepresentable<ListName>() {
    override val cons = ::ListName
    override val render = ListName::name
}

// What about for a normal data class like LogContext
object JLogContext: JAny<LogContext>() {

    private val kind by str(LogContext::kind)
    private val description by str(LogContext::desc)
    private val user by str(JUser, LogContext::user)
    private val list_name by str(JListName, LogContext::listName)

    override fun JsonNodeObject.deserializeOrThrow() =
        LogContext(
            desc = +description,
            kind = +kind,
            user = +user,
            listName = +list_name,
```

```
        )
    }

// Sealed classes are usually hard to convert in JSON, but we have a
// discriminator field to know which class to use.
object JLogEntry : JSealed<LogEntry>() {

    override val discriminatorFieldName: String = "outcome"

    override val subConverters = mapOf(
        "success" to JLogSuccess,
        "failure" to JLogFailure
    )

    override fun extractTypeName(obj: LogEntry): String =
        when (obj) {
            is LogSuccess -> "success"
            is LogFailure -> "failure"
        }

}

object JLogSuccess : JAny<LogSuccess>() {

    private val hostname by str(LogSuccess::hostname)
    private val time by str(LogSuccess::time)
    private val log_context by flatten(JLogContext, LogSuccess::logContext)
    private val log_message by str(LogSuccess::msg)

    override fun JsonNodeObject.deserializeOrThrow(): LogSuccess =
        LogSuccess(
            hostname = +hostname,
            logContext = +log_context,
            msg = +log_message,
            time = +time
        )
}


object JLogFailure : JAny<LogFailure>() {

    private val hostname by str(LogFailure::hostname)
    private val time by str(LogFailure::time)
    private val log_context by flatten(JLogContext, LogFailure::logContext)
    private val error by str(LogFailure::msg)
```

```
    override fun JsonNodeObject.deserializeOrThrow(): LogFailure =
        LogFailure(
            msg = +error,
            hostname = +hostname,
            logContext = +log_context,
            time = +time
        )
}

// Now we go back and fix our `toJson` function in `JsonLogger`
// logger.JsonLogger.kt
private fun LogEntry.toJson(): String = JLogEntry.toJson(this)
```

- `FromJsonF<T>`'s parse: `(String) -> JsonOutcome<T>` produces a `T`. `transform` is covariant in `T`: it uses a forward-facing `(T) -> U` to turn a `FromJsonF<T>` into a `FromJsonF<U>`, composing `f` onto the output side after parsing. This is ordinary `map`. The input side, `String`, is fixed in `FromJsonF` — there's nothing to vary there, so this specific type can't demonstrate contravariance itself, even though the underlying interface it implements is written generically enough to support it.

```
data class FromJsonF<T>(
      val parse: (String) -> JsonOutcome<T>): FromJson<T> {

  override fun fromJson(json: String): JsonOutcome<T> = parse(json)

  fun <U> transform(f: (T) -> U): FromJson<U> =
      FromJsonF { fromJson(it).transform(f) }
```

- We can't do the same type of `transform` on `f: (T) -> U` with `ToJsonF`, because it consumes a `T` and produces a `String`. So we can't run `f` on the result of it's operation. If we had an `f: (U) -> T` though, we could apply it and then call `toJson: T -> String` on the result to serialize it. This is a contravariant functor.

```
data class ToJsonF<T>(val toJson: (T) -> String) : ToJson<T> {

    fun toJson(value: T): String = toJson(value)

    fun <U> contraTransform(f: (U) -> T): ToJson<U> =
        ToJsonF { toJson(f(it)) }
}
```

- Intuitively, a covariant functor contains or produces another type. `Outcome` contains a result and `ContextReader` produces the read value and you can transform it with a forward facing function `f: (T) -> U`. A contravariant functor consumes an arbitrary type to produce a given type. You give it a backwards pointing function `f: (U) -> T` to adapt it to take a U and still produce the given type. For our serializer, it consumes a type to produce a String. Predicates are another example. They consume a type to produce a Boolean. To adapt a `Predicate<T>` into a `Predicate<U>` you would need an `f: (U) -> T` and it would still produce a `Boolean`.

- Couples of covarniant and contravariant functors are quite common and called *profunctors*. So the full profunctor picture, now with both halves concretely demonstrated:

  - `FromJsonF<T>` — covariant in the output. `transform(f: (T) -> U)` composes `f` after parsing, same direction as the data flow.
  - `ToJsonF<T>` — contravariant in the input. `contraTransform(f: (U) -> T)` composes `f` before serializing, reversed direction, adapting the new input type into the old one first.

- By definition, a profunctor comes with two types, here `A` and `B`, and it's contravariant on `A` and covariant in `B`. A profunctor has a `dimap` function and in Kondor JSON is defined by the following interface:

```
interface Profunctor<A, B> {

    fun <S, T> dimap(contraMap: (S) -> A, coMap: (B) -> T): Profunctor<S, T>

    fun <S> lmap(f: (S) -> A): Profunctor<S, B> =
        dimap(contraMap = f, coMap = { it })
    fun <T> rmap(g: (B) -> T): Profunctor<A, T> =
        dimap(contraMap = { it }, coMap = g)
}
```

- `{ it }` is the identity function. `lmap`'s implementation calls `dimap` with your `f` as `contraMap` and identity as `coMap` — meaning the output side `(B -> T)` is left untouched, since `lmap` only wants to change the input type. These interface methods (`dimap/lmap/rmap`) don't do any actual work themselves — they build and return a new profunctor whose internal logic composes your supplied function(s) around the original operation. The mapping only actually runs later, when you call something like `toJson/fromJson` on the result. So: `existingSerializer.lmap { wrapped -> wrapped.unwrap() }` builds `newSerializer`, a `ToJson<WrappedDomainClass>`. Nothing

happens yet. Only when you later call `newSerializer.toJson(someWrapped)` does the chain actually run: `wrapped.unwrap()` first converts it to a `DomainClass`, then the original serializer's `toJson` runs on that, producing the `String` with `coMap`'s identity meaning that `String` is returned as-is. Note if we defined our serialize with `lmap` it could serialize the `WrappedDomainClass` but not deserialize it without us setting `rmap`.

- To define a concrete instance of this, we must define the `dimap` function that accepts two mapping functions and returns a new profunctor with the new types, in this case `S` and `T`.
- In our JSON converters, `A` and `B` are the same type - the domain class, and `S` and `T` are the same type - the string representation of JSON.
- Intuitively, a profunctor creates a relation between two types, converting from one to the other an back.
- So far all our logging has been at the HTTP layer. We also need to log other external adapters such as the database adapter. We used the stdout logger that is part of the Exposed library previously. Fortunately, Exposed has a nice `SqlLogger` interface. First let's add a kind.

```
enum class OperationKind { Command, Query, SqlStatement }

// logger.SqlJsonLogger - SqlLogger is defined in Exposed
data class SqlJsonLogger(val logger: ZettaiLogger) : SqlLogger {
    override fun log(context: StatementContext, transaction: Transaction) {
        logger(
            context.expandArgs(transaction).asSuccess(),
            LogContext(
                "txId: ${transaction.id} duration: ${transaction.duration}",
                OperationKind.SqlStatement,
                null,
                null
            )
        )
    }
}

// So we'll pass our logger to the TransactionProvider
// db.jdbc.TransactionProvider.kt
data class TransactionProvider(
    private val dataSource: DataSource,
    private val logger: SqlLogger,
    val isolationLevel: TransactionIsolationLevel,
    val maxAttempts: Int = 10
) : ContextProvider<Transaction> {
    override fun <T> tryRun(reader: TxReader<T>): Outcome<ContextError, T> =
        inTopLevelTransaction(
            db = Database.connect(dataSource),
            transactionIsolation = isolationLevel.jdbcLevel,
            repetitionAttempts = maxAttempts
        ) {
            addLogger(logger)
      // rest of the method.

// And we can wire it into the main method.
// webserver.Main.kt
fun main() {

    val logger = fileLogger("zettai-${System.currentTimeMillis()}.log")

    val dataSource = prepareProductionDatabase()

    val streamer = createToDoListEventStreamerOnPg()
    val eventStore = ToDoListEventStore(streamer)
    val txProvider = TransactionProvider(
```

```
            dataSource = dataSource,
            logger = SqlJsonLogger(logger),
            isolationLevel = TransactionIsolationLevel.Serializable
        )
        // more of main
        val zettai = Zettai(hub, logger)

        zettai.asServer(Jetty(8080)).start()

        println("Server started at http://localhost:8080/todo/username")

    }
```

Chapter 13 - Designing a Functional Architecture

- The real challenge isn't to design a system that works perfectly with the current requirements, but to design it in a way that can be easily changed when necessary.

Appendices

A1 - What Is Functional Programming

- The core goal of functional programming is referential transparency.
  - *Purity:* The same inputs produce the same results and do not cause observable side effects.
  - *Immutability:* Prefer data structures that are not changed in place.
  - *Totality:* Functions return a result for every input without throwing exceptions.
- Think in Morphisms - Functional Programming Heuristics
  1. *Treat functions as data.* Define behavior combining simpler functions together. Pass and return functions.
  2. *Define your types precisely.* Make the signature of your function meaningful.
  3. *Prefer a declarative style.* Express what you want to achieve with your code rather than the how.
  4. *Try to be lazy.* Defer calculations until the last possible moment.

```
data class BowlingGameFP( val rolls: List<Pins>,
                          val scoreFn: (List<Pins>) -> Int) {
    val score by lazy { scoreFn(rolls) }
    fun roll(pins: Pins): BowlingGameFP = copy(rolls = rolls + pins)
    companion object {
        fun newBowlingGame() = BowlingGameFP(emptyList(), ::calcBowlingScoreRec)
        fun calcBowlingScoreRec(rolls: List<Pins>): Int {
            val lastFrame = 10
            val noOfPins = 10
            fun List<Int>.isStrike(): Boolean = first() == noOfPins
            fun List<Int>.isSpare(): Boolean = take(2).sum() == noOfPins
        fun calcFrameScore(frame: Int, rolls: List<Int>): Int =
            when {
                frame == lastFrame || rolls.size < 3 ->
                    rolls.sum()
                rolls.isStrike() ->
                    rolls.take(3).sum() + calcFrameScore(frame + 1,
rolls.drop(1))
                rolls.isSpare() ->
                    rolls.take(3).sum() + calcFrameScore(frame + 1,
rolls.drop(2))
                else ->
                    rolls.take(2).sum() + calcFrameScore(frame + 1,
rolls.drop(2))
            }
            return calcFrameScore(1, rolls.map(Pins::number))
        }
    }
}

enum class Pins(val number: Int) {
    zero(0),
    one(1),
    two(2),
    three(3),
    four(4),
    five(5),
    six(6),
    seven(7),
    eight(8),
    nine(9),
    ten(10),
}
```

A2 - About Functional Kotlin

- A fairly good but brief review of Kotlin. I especially liked the infix operator examples.

A3 - A Pinch of Theory

- A category is a mathematical construct consisting of some dots, which are called objects, and some arrows connecting them, called morphisms. These morphisms must satisfy certain properties, including the existence of an identity morphism for each object, and the ability to compose morphisms in a way that's associative. We can consider a category as something similar to a set, but instead of being interested in its elements, we're mostly interested in how those elements relate to each other.
- It is possible to create a category with all the types in a programming language: the objects are the different types, and the morphisms are pure total functions between two types, one as input and one as output. Most static-type computer languages form a Cartesian Closed-Category (CCC), which means they have terminal objects, binary products, and exponentiation. When we talk about functors in functional programming, we always mean endofunctors, functors from a category to the same category. As we know, List is a functor. List is also an endofunctor. It takes elements of our language (Ints, Strings, functions from Int to String, List<String>, etc) and maps them to a subset of elements of our language (to Lists).